

DentaVision: Automating Intraoral Scanning & Digital 3D Dental Modeling with Robotics

Marvin Kraus Sebastian Nagles Irmak Damla Özdemir Manuel Stöth

Pascal Meißner Frank Deinzer

{marvin.kraus | sebastian.nagles | irmakdamla.oezdemir | manuel.stoeth}@study.thws.de

{pascal.meissner | frank.deinzer}@thws.de

Faculty of Computer Science and Business Information Systems (FIW)

Technical University of Applied Sciences Wuerzburg-Schweinfurt (THWS)

Sanderheinrichsleitenweg 20, 97074 Würzburg, Germany

Abstract—Intraoral scanning is commonly performed manually by trained dental personnel, making the process costly, time-consuming, and dependent on operator experience. At the same time, automating this task is challenging, because a robot must move safely around complex dental geometry while capturing sufficient data without causing any harm to the patient. This project investigates whether existing robotics software, scanning, and computer vision technologies can be combined into an automated system for intraoral dental scanning and modeling.

The project aims to create a pipeline that encompasses the process of scanning dental geometry and creating a 3D model of said dental geometry into a continuous integration, to evaluate if automation of this process is feasible. The concrete setup is roughly as follows: A simulation of an UR5e collaborative robot, equipped with a camera toolhead creates synthetic images of dental geometry using preexisting datasets. This is then operated through a multitude of preexisting components and libraries, that are chained together in order to recreate the initial jaw model using stereo reconstruction and registration.

The scope of the project includes a PyBullet simulation that renders multiple image pairs belonging to specific jaw sides using Blender, an analysis of different methods for achieving a 3D reconstruction from the rectified image pairs and exporting them as point clouds in .ply format, a point-cloud registration and 3D model reconstruction pipeline that compares five pairwise registration methods, performs sequential multiway registration and point-cloud fusion, and reconstructs the final dental surface using the Ball Pivoting Algorithm, as well as a prototype of a touchscreen user interface, built with Qt Creator, that functions as a control panel, tracking a patient's head position and orientation through a camera and estimating the target coordinates a robotic arm would need to reach a selected tooth during a dental scanning procedure.

Evaluation is currently performed separately for the individual components using external or generated datasets, as the complete pipeline has not yet reached a stage at which an end-to-end evaluation is possible. For the registration and model reconstruction stage, controlled synthetic datasets and five distinct

dental model sets comprising ten jaw meshes were used to compare different registration and surface reconstruction approaches. The evaluated components were designed to interface with each other, so that a future complete pipeline could be assessed by comparing the original dental geometry with the reconstructed and registered 3D model.

The resulting system may provide a basis for further investigating automated intraoral scanning under controlled conditions. The simulated image generation for example could be improved by using gazebo, with a much better ROS integration instead of pybullet, which in turn could allow using a real ur5e for image generation. Similarly, The head position and orientation estimation could be improved by replacing the current 2D landmark-based approximation with a real 3D calculation, using a calibrated camera matrix together with a method such as `solvePnP`. And in the future, the results of this project may be used to create a specialized medical robot, designed to be certified to work with real humans, thus making trained personnel available for other tasks.

Index Terms—UR5e, Simulation, Stereo Reconstruction, 3D Registration, Qt

I. SIMULATION-BASED IMAGE GENERATION

Authors: Manuel Stöth

A. Introduction

To evaluate stereo-based 3D reconstruction algorithms, a large dataset of closely spaced image pairs with known relative camera poses is required. That is difficult to obtain in practice, since jaw scans involve complex geometries and limited access paths. This section presents a simulation environment that automatically generates such image datasets from pre-scanned dental arches of the Bits2Bites dataset [1], controlling the relative pose between adjacent images precisely. The environment combines a UR5e collaborative robot [2] with a custom camera tool in PyBullet [3], using a modified IK solver for arbitrary pose planning. High-quality images are rendered via a separate Blender instance that mirrors the simulation state, with customizable rendering settings [4].

B. Design Decisions and Implementation

1) *Simulation with PyBullet:* PyBullet [3] was chosen as the physics and robot kinematics backend. This decision was made after trying to use NVIDIA URSim, which proved to be too difficult to set up in Linux due to specific driver dependencies, which weren't bundled in any easily accessible repository for Fedora Linux, and MuJoCo, which I set aside after finding a GitHub repository [5] that provided a working UR5e model including mesh files and a URDF description file for the exact UR model we were targeting, which was implemented with PyBullet. It loads the UR5e robot from a URDF description, provides forward and inverse kinematics, collision detection and a GUI for inspection. All quantities are kept in SI units (metres) so that camera poses exported for the reconstruction pipeline can be used without scaling.

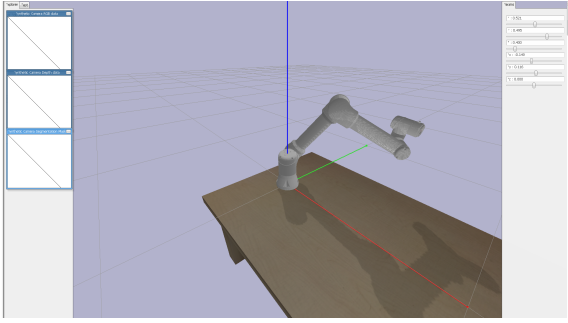


Figure 1: Basic UR5e implementation sitting on a table in PyBullet [5].

2) *Robot model and camera tool:* The modeled UR5e collaborative robot arm [2] consists of six revolute joints, based on the aforementioned repository [5]. The custom camera tool, which is modeled after the Sirona Omnicam[6] scanner, is mounted onto wrist_3 of the robot. The arm parts are colored based on a gradient from green to red, which shows how far they are away from their joint limits.

The preexisting IK solver has been modified in a few ways. One such change was changing the joint limits in order to allow or disallow certain IK solutions. The elbow_joint lower limit, for example, was set to 0° to prevent IK solutions, where the forearm would bend upwards, while the wrist_3_joint limits were increased from 180° to the full hardware possible amount of 360° in both directions, to allow IK solutions for positions that previously resulted in unrecoverable positions. Additionally, it is possible to set the seed for the IK solver to either a global seed for deterministic positioning or to the current position in order to achieve a solution close to the current one. To reach the dense waypoint sweeps around the jaw, the inverse kinematics is used in a modified fashion. First, the damped least-squares solution (PyBullet

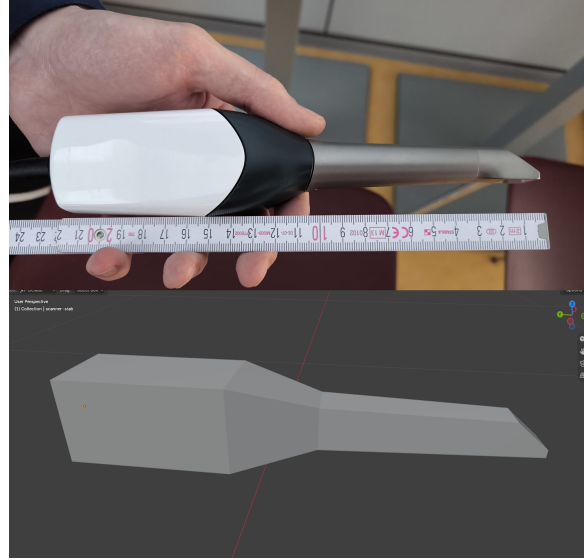


Figure 2: The Sirona Omnicam scanner device and the STL model used in the URDF description

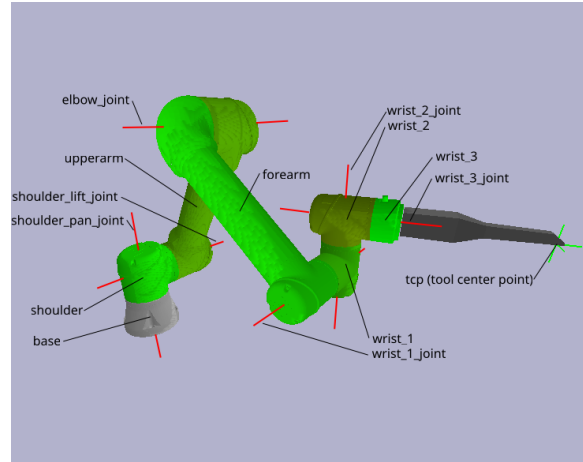


Figure 3: Description of the complete UR5e model

Table I: Joint limit values

Name	lower_deg	upper_deg	rest_deg
shoulder_pan	-180	180	0
shoulder_lift	-180	180	-90
elbow	0	180	90
wrist_1	0	360	180
wrist_2	-180	180	-90
wrist_3	-360	360	0

IK_DLS) is only a starting point: starting from a seeding joint configuration, the solver iterates up to ten times, computing the tool position and orientation error and applying a damped Jacobian update (analytic Jacobian) until the tool pose converges onto the requested target (position tolerance $1e-6$ m). Second, since a single IK result may collide with the jaw, the solver is retried with different joint-space seed configurations and the first collision-free solution is kept. Movement between waypoints is planned with the *pybullet-planning* library [7] (RRT-Connect with smoothing and restarts, fixed random seed for reproducible paths).

3) *Realistic rendering with Blender*: Physics-based rendering with PyBullet’s built-in renderer is not photo-realistic enough for validating stereo reconstruction algorithms. Blender [4] was chosen as the renderer because of prior experience with the software and its Cycles path tracer, which offers physically based materials and GPU acceleration. In practice this means that a separate Blender instance builds the scene procedurally from the same URDF data and renders left/right stereo pairs with the Cycles engine. The scene is set up entirely by script (*rig.py*): The

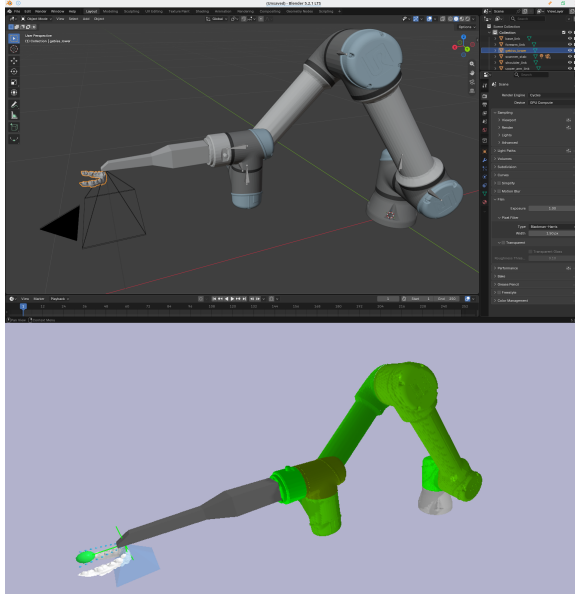


Figure 4: A position mirrored from PyBullet to Blender

URDF link meshes are imported, re-oriented and parented to a bone armature constructed from the joint data, and the scanner tool, both pinhole cameras and the scanner light are attached to the *wrist_3* frame. Default camera and light parameters consist of: a full-frame 16:9 sensor (36×20.25 mm), a horizontal field of view of 87° , a stereo baseline of 1.7 mm lateral offset, and near/far clipping at 1/40 mm. The scanner light is a point light offset a few millimeters forward from the TCP, to prevent it from clipping

in the scanner STL, ensuring the same lighting for both render pairs. Rendering runs with Cycles on the GPU (NVIDIA RTX 3060) at 1920×1080 (16:9); the Blender render properties use 4096 samples with denoising enabled at the high and *Accurate* output quality.

So that the rendered images correspond exactly to the simulated camera poses, the simulation mirrors its state to Blender over a TCP socket (*blender_link.py* on the simulator side, *mirror.py* in Blender, polled every 50 ms). Each message carries the six joint angles and the tool-center-point position; Blender applies the joint angles to the armature bones and moves both cameras and the scanner light to the mirrored TCP position. A render request includes the target file paths for the left and right image. Blender renders both views sequentially and reports its completion. The jaw model is loaded in both applications from the same STL files with identical transforms (world scale $S = 1$ m per Blender unit). Its material is set to a roughness/specular pair of 0.8/0.2, and backface culling makes the mesh one-sided: the inside of the dental model becomes transparent, so the underside of the jaw never appears in the rendered images.

Each currently defined scan start position in *config.py* moves along 21 waypoints on a parabola over one jaw side and captures 21 stereo pairs, i.e. 42 single images per scan. Rendered at 4096 samples, one such jaw side takes roughly 2–3 hours on the RTX 3060, depending on the complexity of the visible scene. The outer side of a jaw for example takes longer than the top side of a jaw.

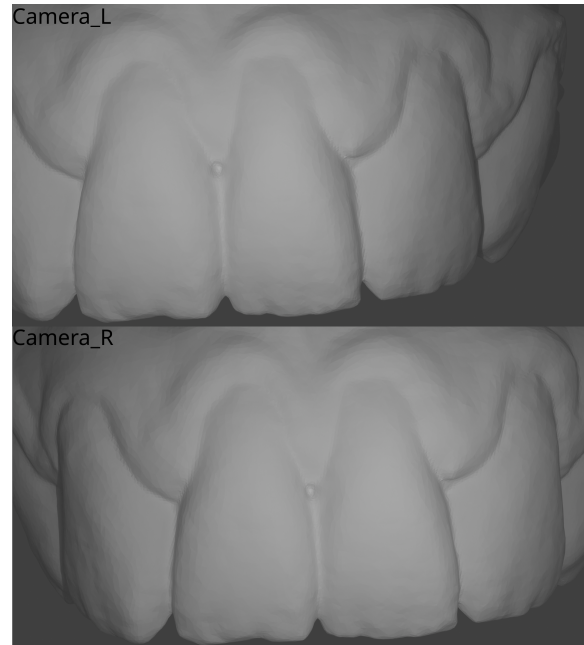


Figure 5: Render pair example

4) *Generated datasets in render/*: Every scan is stored in its own folder under the `render/` directory. The folder name encodes the scan geometry: jaw type, jaw folder, start position, render resolution and stereo baseline, e.g. `lower_1_all_1920x1080_lat1.7mm`. For each waypoint that is scanned, the folder contains the two rendered images (`iL_render.png` and `iR_render.png`) and a pose file `i_pose.json`; a single `render_settings.json` documents the whole scan. A render failure aborts the scan so that a folder is never left incomplete.

Each `i_pose.json` stores the waypoint index and label together with the left and right camera pose as position (in metres) and quaternion (x, y, z, w) in the jaw frame. Because all quantities are kept in SI units, the reconstruction pipeline can consume these poses directly without any scaling. The metadata of the JSON files is summarized in Table II. Together with the measured render times (e.g. 9731 s $\approx$ 2.7 h for the recorded scan above), a recorded folder is fully self-describing.

The `visualize_scan.py` tool reads the recorded pose files to redraw all camera poses in the jaw frame; this was used to verify that the stereo baseline remains constant across the rendering process and that the cameras follow the intended path.

C. Package/Code Structure

1) *Component architecture*: The simulation is implemented as the Python package `ur5e_bullet` (Figure 6), split into a simulation core, an interactive driver and the Blender side.

All parameters live in `config.py`, sorted by domain (paths, robot and kinematics, camera model, rendering, scan positions). It is standalone-loadable (no `pybullet` import, no relative imports), so the Blender scripts import exactly the same configuration inside Blender’s own Python environment; it also defines the joint limits and rest poses used as IK null space and the scale factor S shared by all geometry. `math_utils.py` provides pure-Python quaternion arithmetic (multiplication, conjugation) and the world-to-jaw-frame transform, and – being free of `pybullet` imports – is likewise usable in Blender.

The simulation core is `sim.py`, whose `UR5Sim` class loads the robot URDF and the jaw STL as a collision object, implements the IK variant (damped least squares refined by the analytic Jacobian), collision checking and RRT-Connect path planning, and executes trajectories in `PyBullet`. `waypoints.py` generates the parabola waypoints per start position from the parameters in `config.py` (21 waypoints, orientation anchors interpolated with `slerp`, optional `look_at_jaw` re-aiming). `camera.py` derives the left/right camera poses in the jaw frame from the actual scanner_link transform (TCP pose in the

Table II: Metadata of the JSON files in a scan folder.

Field	Content
<i>render_settings.json — info</i>	
<code>start_position</code>	scan start position, e.g. all
<code>jaw_folder</code>	folder number in data/meshes_jaws/[8]
<code>jaw_type</code>	upper/lower jaw
<code>jaw_pos</code>	world frame position (m)
<code>jaw_euler_deg</code>	world frame orientation (deg)
<code>jaw_shading</code>	flat/smooth/auto-smooth shading of the jaw mesh
<code>jaw_auto_smooth_angle_deg</code>	auto-smooth angle threshold (deg), null for flat
<code>render_w</code>	image resolution width
<code>render_h</code>	image resolution height
<code>render_engine</code>	currently only CYCLES
<code>render_device</code>	GPU/CPU
<code>world_unit</code>	m
<code>timestamp</code>	start time
<code>render_duration_s</code>	total render time
<i>render_settings.json — reconstruction</i>	
<code>camera_fov_deg</code>	opening angle of the camera
<code>camera_lens_mm</code>	lens size based on render settings
<code>camera_lateral_offset</code>	distance between camera and TCP
<code>baseline_m</code>	distance between both cameras
<code>camera_far_m</code>	render far clipping plane (0.04 m)
<code>rectified</code>	currently always true
<code>lens_distortion</code>	currently always false
<i>render_settings.json — reconstruction.camera_intrinsic</i>	
<code>fx_px</code>	focal length in x direction
<code>fy_px</code>	focal length in y direction
<code>cx_px</code>	principal point (optical center) in x
<code>cy_px</code>	principal point (optical center) in y
<code>sensor_w_mm</code>	physical sensor width
<code>sensor_h_mm</code>	physical sensor height
<i>i_pose.json (per waypoint)</i>	
<code>waypoint</code>	waypoint index
<code>label</code>	waypoint name e.g. W10
<code>camera_left</code>	pos. (m) and quat. (x, y, z, w) of left camera
<code>camera_right</code>	pos. (m) and quat. (x, y, z, w) of right camera

scanner frame plus/minus the 1.7 mm lateral baseline offset) and provides the intrinsic parameters in pixels, while `demo.py` writes them as `<n>_pose.json` together with a `render_settings.json` per scan.

The interactive control loop also lives in `demo.py`: a console parses the commands defined in `commands.py`; `viz.py` draws waypoint

Simulation environment: component architecture

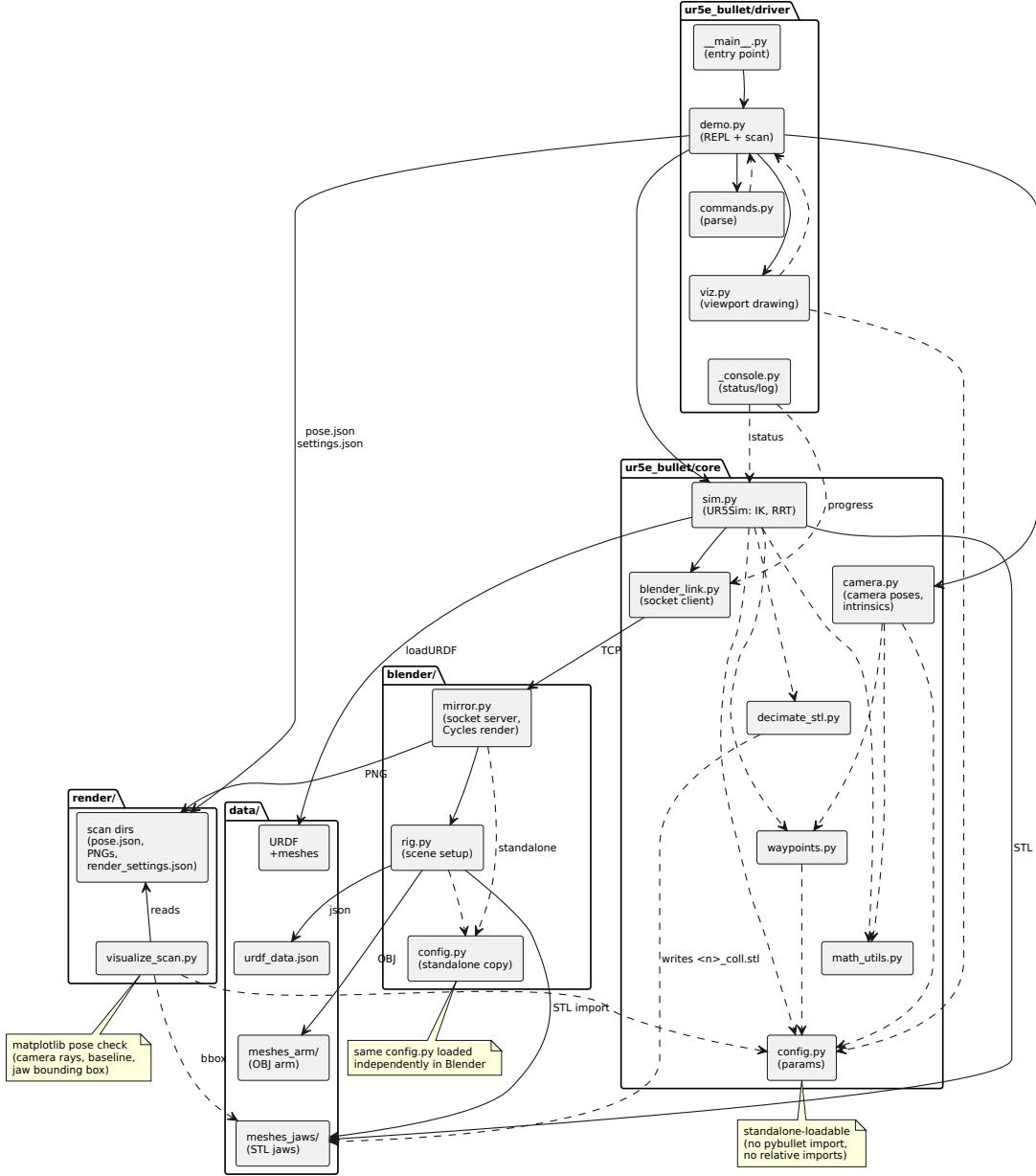


Figure 6: Component architecture of the simulation environment.

markers, look targets and cross-hairs in the PyBullet viewport, and `preview_and_move` plans a collision-free path and shows it before execution. `blender_link.py` is the TCP client that launches the Blender GUI instance, mirrors joint states and TCP position, forwards the render requests and waits for the completion messages. `decimate_stl.py` reduces the jaw STL to a voxel-decimated collision mesh for PyBullet.

On the Blender side, `rig.py` builds the whole scene procedurally (URDF meshes, armature, scanner, cameras, light, jaw import with backface culling, Cycles settings), and `mirror.py` runs the socket

server in Blender: it applies the received joint angles and TCP position, executes the requested left/right Cycles renders per waypoint and reports progress.

2) *Configuration*: All parameters are collected in `src/ur5e\_bullet/config.py` and sorted by domain (paths, robot and kinematics, camera model, rendering, PyBullet GUI, visualization, jaw material, Blender, scan positions). The file is the sole editing place for the parameters, which are forwarded into the live PyBullet session and, through the standalone load, into Blender. Table III lists the parameters together with their meaning and current values.

Table III: Configuration parameters in config.py.

Parameter	Meaning	Current value
<i>Paths</i>		
PROJECT_ROOT	project root directory	<repo>/src/ur5e_bullet
ROBOT_URDF_PATH	UR5e robot description	data/robot_description/urdf/ur5e.urdf
JAWS_DIR	jaw STL model directory	data/meshes_jaws
ARM_MESH_DIR	robot arm meshes directory	data/meshes_arm
BLENDER_URDF_DATA_JSON	URDF data exported for Blender	data/urdf_data.json
SCANNER_STAB_STL	scanner rod mesh	data/robot_description/meshes/scanner-stab.stl
RENDER_DIR	output directory for scans	render
<i>Robot and kinematics</i>		
TOOL_OFFSET_POS	TCP offset that changes the TCP position from wrist_3 to the top of the scanner	[0.213, 0, -0.006]
TOOL_OFFSET_ORN	tool offset orientation (quaternion, unused)	[0, 0, 0, 1]
SCANNER_BASE_ORN_DEG	fixed scanner base rotation (deg); must stay consistent across camera, demo and Blender	[0, -90, 0]
GEBISS_COLL_CELL	voxel cell size for collision mesh decimation (mm)	1.5
IK_LAMBDA	damping factor of the IK solver	0.05
IK_TOLERANCE	IK position tolerance (m)	0.08
RRT_RESTARTS	RRT planning restarts	30
RRT_SMOOTH	RRT path smoothing steps	30
RRT_SEED	deterministic planning seed	0
JOINTS	per-joint limits and rest poses, see Table I	—
<i>Camera / sensor (RealSense D455)</i>		
CAMERA_ROLL_DEG	roll of the camera rig around the viewing axis (deg)	-90
CAMERA_SENSOR_W_MM	physical sensor width (mm)	36.0
CAMERA_SENSOR_H_MM	physical sensor height (mm), 9/16 of the width	20.25
CAMERA_FOV_DEG	horizontal field of view (deg)	87
CAMERA_LENS_MM	focal length derived from sensor width and FOV (mm)	18.97
CAMERA_LATERAL_OFFSET	lateral offset of one camera from the scanner axis (m); baseline_m = 2 x offset	0.0017
CAMERA_NEAR_M	Blender near clipping plane (m)	0.001
CAMERA_FAR_M	Blender far clipping plane (m)	0.04
CAMERA_DISPLAY_M	frustum display range (m)	0.2
<i>Rendering</i>		
RENDER_W, RENDER_H	render resolution (px)	1920 / 1080
RENDER_ENGINE	Blender render engine	CYCLES
RENDER_DEVICE	Blender render device	GPU
RENDER_TIMEOUT	timeout per render pair (s), abort on exceedance	900
RENDER_TRANSPARENT	toggles transparency of the rendered images	false
<i>PyBullet GUI</i>		
PB_CAMERA_DISTANCE	PyBullet GUI camera distance to target (m)	1.0
PB_CAMERA_YAW	PyBullet GUI camera yaw (deg)	70
PB_CAMERA_PITCH	PyBullet GUI camera pitch (deg)	-25
PB_CAMERA_TARGET_POS	PyBullet GUI camera target position (m)	[0.6, 0, 0.4]
PREVIEW_PAUSE	pause after path preview (s)	0.6
WAYPOINT_MARKER_RADIUS	radius for the points that show where the scanner moves to around a jaw (m)	0.001
BOOT_START	robot pose on program start; None disables	pos:[0.85, 0, 0.38], ori:[0, 0, 0]
<i>Visualization / debug</i>		
DRAW_CAMERA_FRUSTUM	draw camera frustum for the active start position (shows the space the cameras render)	true
CAMERA_FRUSTUM_COLOR	frustum color (RGB)	[0.0, 0.55, 1.0]
CAMERA_FRUSTUM_ALPHA	frustum transparency	0.2
LOOK_TARGET_RADIUS	look-target marker radius (the look marker is a point to which the view direction of a waypoint can be corrected to, in order to better the view angle) (m)	0.008
LOOK_TARGET_COLOR	look-target marker color (RGBA)	[0.1, 1.0, 0.3, 0.95]

Table III: Configuration parameters in `config.py` (continued).

Parameter	Meaning	Current value
<i>Jaw / material</i>		
GEBISS_SCALE	jaw mesh scale (mm to m)	[0.001, 0.001, 0.001]
GEBISS_ROUGHNESS	Principled BSDF roughness of the jaw material	0.8
GEBISS_SPECULAR	specular IOR level of the jaw material	0.2
GEBISS_SHADING	Blender jaw shading mode: flat/ smooth/ auto_smooth	auto_smooth
GEBISS_AUTO_SMOOTH_ANGLE_DEG	auto-smooth angle threshold (deg)	30
<i>Blender</i>		
S	global scale, 1 Blender unit = 1 m	1
LIGHT_POWER	scanner light power	0.0009
LIGHT_OFFSET	light offset relative to the scanner (m)	[0.008, 0.025, 0.213]
ENABLE_BLENDER_SYNC	enable TCP robot-state sync to the Blender GUI	true
MIRROR_SCRIPT	path to the Blender mirror script	blender/mirror.py
SOCKET_BUFFER	socket buffer size (bytes)	4096
SOCKET_POLL_INTERVAL	blender_link.py socket poll interval (s)	0.05
ATTACH_VIEWPORT_TO_CAMERA	attaches the viewport to the <code>scene.camera</code>	true
<i>Scan positions</i>		
START_POSITIONS	start poses, approach sequences and parabola way- point generators; described in the usage section (Section I-D2)	—

D. Usage

1) *Getting started/Basic Commands:* The project is available under the dentavision GitHub organisation [8]. It requires Python ≥ 3.14 , Poetry $\geq 2.x$ and a GUI-capable Blender build available as `blender` on the `PATH`. The Poetry-managed simulation dependencies (PyBullet, `pybullet-planning`, NumPy, matplotlib) are installed in the project directory with `poetry install`, which installs the dependencies from `pyproject.toml`. After that, the program can be started by running either `poetry run ur5e` or `poetry run python -m ur5e_bullet` in the project root. On boot the arm moves to the `BOOT_START` pose without a jaw model loaded. The `>` prompt accepts the basic move and inspection commands:

- `m x y z [rx ry rz]` moves the TCP via RRT (asks for confirmation; orientation defaults to 0 0 0). The position values should not exceed 1 m in any direction due to the operational range of the robot arm (will throw an error) and the `x` value should be kept out of the negative range as the IK solutions tend to become somewhat unpredictable

in the area behind the robot. Accepted inputs for the command would be for example `m 0.85 0 0.4` or `m 0.6 0.3 -0.4 180 90 0`.

- `o x y z` sets the tool offset. This is only useful for testing the offset value of new robot attachments instead of the current scanner model, as the correct offset of the current scanner is loaded from `config.py`.
- `s <speed>` sets the global move speed (default 0.5). This influences the speed with which the robot arm drives along the RRT path of a movement.
- `@` resets the robot to the last joint configuration. This is needed, because the PyBullet GUI can't be configured to not let the mouse cursor interact with scene objects. So if clicking on the robot throws it off its position, this is for resetting that.
- `jaw <nr> [upper|lower]` loads the jaw mesh from `data/meshes_jaws/<nr>` into PyBullet and Blender. This can only be done when a jaw has already been loaded by a start position.
- `q` quits the interactive loop and closes the simulation including Blender.
- `start <name>` drives the robot to the start position `name`, if defined in `START_POSITIONS` in `config.py`.

```
X 0.850 Y -0.000 Z 0.380 pa -14° ll -60° el +90° w1 +141° w2 -70° w3 -0°
[mirror] Szene aufgebaut
[mirror] Verbunden mit Pybullet (Port 46693)
[ok] (0.850, -0.000, 0.380) rx=0 ry=0 rz=0 (0 mm)
Boot: Startposition erreicht (Gebiss nicht geladen)
— UR5e Demo —
Move: 'm x y z [rx ry rz]' (RRT, Default-Orientierung 0 0 0)
Tool-Offset: 'o x y z' (z. B. 0 0 0.15)
Geschw.: 's 0.5' (global, Default 0.5)
Reset: '@' (nach manuellem Ziehen)
Gebiss: 'jaw <nr> [upper|lower]' (z. B. jaw 3 upper)
Start: 'start <Ausser|Oben|Innen>' (Startposition anfahren)
Waypoints: '+/-/-' naechster/vorheriger Waypoint
Scan: 'scan [n]' (alle bzw. nur die ersten n Waypoints + L/R rendern)
Scan-All: 'scan-all <start1> [start2 ...]' (Startpositionen der Reihe nach scannen)
>
```

Figure 7: Console view after program start

2) *Defining start positions:* A start position bundles a target TCP pose, the jaw placement, orientation anchors, the camera view and further parameters for a waypoint generator. The entire goal of this composition is to define a function, that creates waypoints along a parabola, which are suited to the specific side of the jaw that has to be scanned from them. The `config.py` in the repository [8] already contains start positions for the first jaw, meaning

waypoint parabola configurations for the outer, inner and upper side of its upper and lower jaw halves.

Listing 1: start position example

```
{
  "all": {
    "tcp_pos": [0.616, 0, 0.296],
    "tcp_ori_deg": [180, 90, 0],
    "approach": [
      {"tcp_pos": [0.85, 0, 0.38],
       "tcp_ori_deg": [0, 0, 0],
       "label": "1",
       "use_current_seed": False},
      {"tcp_pos": [0.616, 0, 0.296],
       "tcp_ori_deg": [0, 90, 0],
       "label": "2",
       "use_current_seed": True}],
    "jaw_pos": [0.65, 0, 0.3],
    "jaw_euler_deg": [0, 0, 90],
    "jaw_folder": 1,
    "jaw_type": "lower",
    "generator": parabola_waypoints,
    "look_at_jaw": True,
    "look_target": [0.662, 0.0],
    "parabola": {
      "x0": 0.616,
      "a": 0.047,
      "z": 0.296,
      "n": 21,
      "y_max": 0.037,
      "power": 4},
    "ori_anchors": {
      "start": [90, 0, 0],
      "mid": [180, 90, 0],
      "end": [-90, 0, 0]},
    "view": {
      "apply": True,
      "distance": 0.1,
      "yaw": 90.0,
      "pitch": -89.0,
      "target": [0.65, 0.0, 0.3]},
  }
}
```

The following points describe how the configuration entries can be used to influence the waypoint generation:

- `tcp_pos` and `tcp_ori_deg`: These two values define the position and orientation the robot drives to before the jaw gets loaded. The position is supposed to be located at the origin of the parabola (Figure 8).

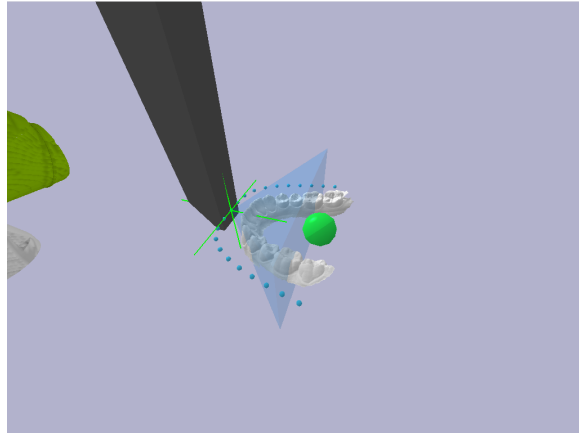


Figure 8: Scanner at the `tcp_pos` and `tcp_ori_deg` of a start position

- `approach`: Optional list of intermediate poses. These are driven to before the position from the values above. The main reason was to be able to set a default point from which the `tcp_pos` is always reachable in order to prevent weird IK solutions based on the position of the robot before driving to the start position.
- `jaw_pos` / `jaw_euler_deg`: Placement of the jaw mesh in the PyBullet scene and Blender.
- `jaw_folder` / `jaw_type`: Jaw mesh directory under `data/meshes_jaws/` and side (upper / lower).
- `generator`: Callable that computes the waypoints from this config (e.g. `parabola_waypoints`); `waypoints` is a static list fallback when no generator is set.
- `parabola.x0`: This is the world x position of the origin of the parabola. Should equal the x value of `tcp_pos`.
- `parabola.a`: This is the multiplier of the parabola. With this it's possible to stretch the height of the parabola along the world x axis.
- `parabola.z`: Corresponds to the height at which the parabola lies flat down. Should equal the z value of `tcp_pos`.
- `parabola.n`: Configures the amount of waypoints that are generated along the parabola. The number should be odd so the center waypoint lands on the `tcp_pos`.
- `parabola.y_max`: This configures the span between the two outermost waypoints, or in other words, the range between the two ends of the parabola.
- `parabola.power`: With this it's possible to choose the power of the parabola. Mostly only powers 2 and 4 are needed here.
- `ori_anchors`: The point of the orientation anchors is to define 3 orientations at the origin and at each end of the parabola. The orientation of the

robot at the waypoints between these anchors is interpolated with these values. The images here show the start 9, mid 8 and end 10 orientation anchor positions.

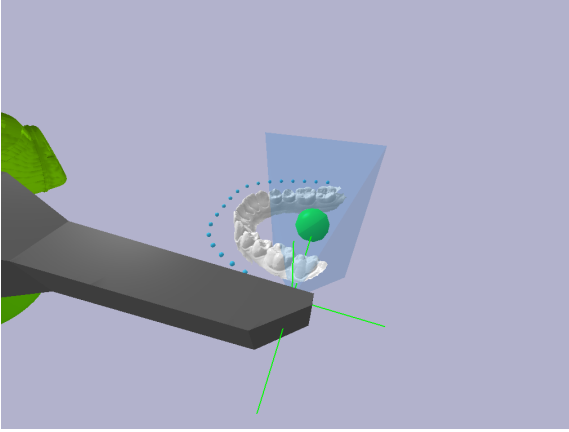


Figure 9: Scanner at W_{min} with the orientation of the start anchor

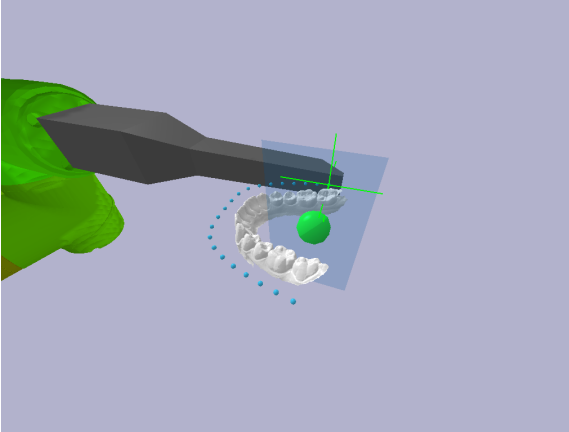


Figure 10: Scanner at W_{max} with the orientation of the end anchor

- `look_at_jaw`: This is an additional toggle that changes the orientation generation additionally to the interpolation of the orientation anchors. It changes the orientation slightly, so the view direction looks exactly at the point defined in `look_target`, which ensures that the gradient of the view direction lies on the same level as the parabola. This helps to create a more direct viewing angle on the tooth surface, but is only useful for the scan of the outer side of a jaw.
- `view`: Here the PyBullet GUI camera viewpoint can be customized for a start position, so the user doesn't have to manually drag the camera there.
- `view.apply`: This is a toggle, with which the camera position change can be turned on or off.
- `view.distance`: This defines the distance of the camera from the target position.

- `view.yaw/pitch`: Yaw and pitch of the camera. Pitch can't be 90° because a PyBullet bug lets the whole scenery disappear when it has that value.
- `view.target`: The x y z position onto which the camera looks and revolves around.

The default configuration ships six positions for the three lower (`all`, `oll`, `ill`) and three upper arches (`alu`, `olu`, `ilu`).

3) *Start Position Commands*: Once a start position is active, the following additional commands can be used:

- `+/-`: Can be used either only as `+/-` to let the robot drive to the next upper/lower waypoint or as `+/- <n>`, which drives the robot by n waypoints in the chosen direction.
- `scan <n>`: This allows rendering a specific number n of waypoints, which was mostly used to test the metadata generation of the scan process.
- `scan-all <start1> [start2 ...]`: This command allows rendering multiple start positions sequentially, which makes it possible to automate the hour-long rendering process of multiple start positions.

The process of fully rendering a start position is visualized in Figure 11. First, a scan folder is created and the initial `render_settings.json` is written; the robot then drives to the first approach pose (RRT) and sequentially moves along the waypoints to W_0 without rendering. For each further waypoint it repeats the same sequence: the robot moves to the next waypoint, the left/right camera poses are computed in the jaw frame and written to `<n>_pose.json`, and a render request for the left and right image is sent to Blender. Blender renders both views via Cycles on the GPU, stores the PNGs in the scan folder and reports completion. After the last waypoint, `render_settings.json` is updated with the measured `render_duration_s`.

4) *Rendered datasets*: All six start positions were rendered into complete scan datasets. Each scan produces a folder under `render/` whose name encodes the scan geometry (jaw type, jaw folder, start position, render resolution and stereo baseline), e.g. `lower_1_all_1920x1080_lat1.7mm`, containing 21 pose files (`0_pose.json` through `20_pose.json`), 42 rendered PNGs (left/right per waypoint) and a single `render_settings.json`; the field and file layout is detailed in Section I-B4. Table IV summarizes the six datasets together with their measured render durations.

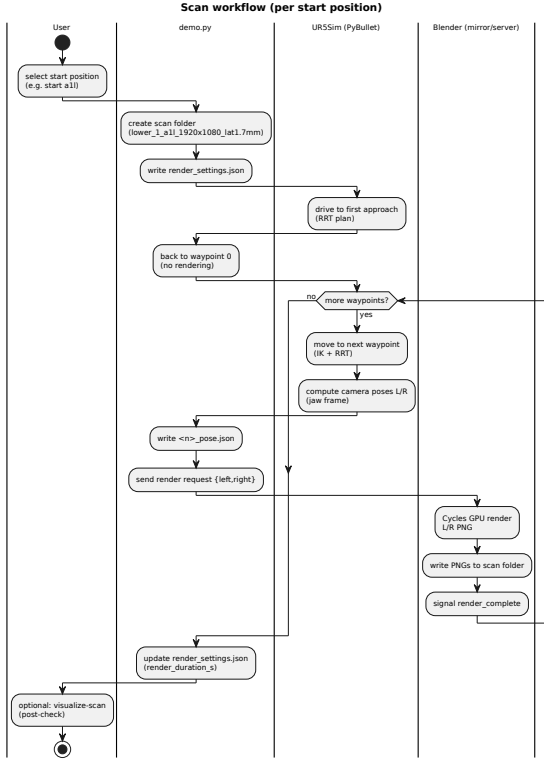


Figure 11: Scan workflow for one start position.

Table IV: Rendered datasets for the six start positions.

Start position	Jaw	Waypoints	Total render time
a11	lower	21	2.7 h
o11	lower	21	2.3 h
i11	lower	21	4.3 h
alu	upper	21	2.8 h
olu	upper	21	2.6 h
ilu	upper	21	5.4 h

The reconstruction parameter block exported by each dataset feeds the stereo pipeline directly: `baseline_m = 0.0034`, `camera_fov_deg = 87`, `camera_lens_mm` ≈ 18.97 , `camera_far_m = 0.04`, `rectified = true`, `camera_intrinsic` with `fx_px = fy_px = 1011.63`, `cx_px = 960`, `cy_px = 540` on a $36\text{mm} \times 20.25\text{mm}$ sensor. The jaw meshes rendered by the simulation are real intra-oral scans from the Bits2Bites dataset [1], i.e. 200 registered pairs of upper and lower dental arches in STL format, acquired with Carestream and 3Shape TRIOS scanners. All datasets together cover both the lower and the upper arch from three view sides each, so that the stereo registration and model construction sections are supplied with synthetic image pairs whose relative camera poses are known precisely.

The exported `pose.json` files encode the camera poses purely as position and quaternion (x, y, z, w) in

the jaw frame. As a reference implementation for how these poses are to be interpreted, `visualize-scan` reads the pose files and reconstructs the full camera geometry from them: it rotates the view vector $(0, 0, -1)$ by each camera’s quaternion to draw the camera rays (length `camera_far_m`) from the left (red) and right (blue) camera centers with `matplotlib`, verifies the stereo baseline by measuring $|\mathbf{L} - \mathbf{R}|$ (expected constant `baseline\_m`) and plots the sweep order and the jaw bounding box. Any consumer of the exported data can therefore use this tool as a reference for the pose convention before feeding the poses into the reconstruction pipeline.

5) *Reconstruction reference*: The exported stereo pairs with known intrinsics and extrinsics are the input to the subsequent image acquisition and 3D reconstruction section. The rectified left/right pairs enable standard stereo matching (e.g. SGM [9] or learned methods such as IGEV-Stereo [10]) followed by point triangulation with the stated baseline; the `camera_far_m` value bounds the penetration of the viewing rays into the jaw surface and can be used as a near-plane limit when triangulating intra-oral geometry. All pose files together realize the photo-granular pose ground-truth that is otherwise hard to obtain for real intra-oral scans.

II. IMAGE ACQUISITION AND 3D RECONSTRUCTION

Authors: Sebastian Nagles

III. SIRONA OMNICAM IMAGE ACQUISITION INVESTIGATION

A. Objective and Initial Approach

The initial objective of this investigation was to obtain image data directly from the **Sirona Omnicam** for use within the DentaVision project. Access to the scanner’s image stream was considered an important prerequisite for eventually integrating the scanner into the reconstruction pipeline and avoiding dependence on externally provided datasets or pre-existing scan data.

The initial approach was based on the assumption that the Omnicam could expose its camera or acquisition interface as a conventional USB device. Therefore, the first step was to investigate how the scanner was connected to the computer and whether its USB interface could provide a direct path to the acquired images. Both the Windows Device Manager and USBTreeView were used for this purpose. The connected devices and USB interfaces were inspected in an attempt to identify an entry corresponding to the Sirona Omnicam, or a device with a name or hardware information that could be associated with the scanner. USBTreeView was additionally used to inspect the available USB device information, interfaces, and endpoints in greater detail.

This initial investigation did not provide a straightforward way to access the scanner’s image data. In particular, identifying the device at the USB level did not imply that the image stream was exposed through a standard camera interface that could be accessed independently of the manufacturer’s software. This led to a change in the investigation strategy.

Instead of treating the Omnicam as a conventional USB camera, the investigation was subsequently expanded to the hardware, camera interfaces, drivers, acquisition SDK, and CEREC software stack. The purpose was to determine how the scanner was actually exposed to the operating system and how the CEREC software interacted with the underlying hardware. This ultimately led to the investigation of the Matrix Vision camera components and, later, to reverse engineering of the software and DLLs used by **CEREC SW 5.1**.

B. Camera and Acquisition Interface Investigation

After the initial USB-level investigation, the focus shifted towards identifying the camera hardware used by the Omnicam and determining whether it could be accessed independently of the CEREC software. The investigation revealed that the scanner contained an industrial camera from MATRIX VISION, identified as a SiCamCCT camera belonging to the mvBlueCOUGAR family. The device was identified with serial number 27323555 and exposed a GenICam interface. The camera used an ICX424 sensor and communicated through GigE Vision.

The identification of the camera was significant because it provided a potential acquisition interface that could be investigated independently of the higher-level CEREC software. The Matrix Vision ecosystem includes tools and libraries for configuring and acquiring images from compatible devices. Therefore, the investigation focused on the mvIMPACT Acquire SDK and the associated wxPropView utility. The wxPropView interface used during this investigation is shown in the appendix for reference (see Figure 80).

1) Camera Detection and SDK Investigation:

The mvIMPACT Acquire SDK was used to investigate whether the camera could be detected and accessed directly. The device could be identified through the Matrix Vision acquisition software, confirming that the underlying camera hardware was accessible at this level. Several Matrix Vision components were also identified as being associated with the device, including libraries such as `mvDeviceManager.dll`, `mvBlueCOUGAR.dll` and `mvPropHandling.dll`. The wxPropView application was additionally used to inspect and configure the available camera properties. This provided access to acquisition-related parameters and helped determine whether the camera could be operated

outside the normal CEREC acquisition workflow. The investigation also identified several image formats supported by the camera interface, including:

- RGB8Packed
- RGB20p12
- BGR8Packed
- BGR10V2Packed

The availability of these formats indicated that the camera interface itself was capable of providing image data in standard color representations. However, this did not necessarily mean that the Omnicam used these formats during its normal scanning operation.

2) *Acquisition Attempts:* Despite successfully detecting the camera and accessing its configuration through the Matrix Vision tools, obtaining a usable image through direct acquisition initially proved unsuccessful. One of the main observed results was a black image being returned during acquisition attempts. This indicated that device detection and communication with the camera did not necessarily imply that the complete acquisition process used by the Omnicam had been correctly reproduced.

Acquisition parameters, particularly exposure-related settings, were investigated and adjusted in an attempt to determine whether the black image was caused by an incorrect configuration. A newer version of the mvIMPACT Acquire SDK was also installed and tested. After experimenting with different exposure settings and other available acquisition parameters, it was eventually possible to obtain an image in grayscale. However, this result did not provide a reliable acquisition path suitable for integration into an automated scanning pipeline. In addition, the mvIMPACT Acquire SDK did not provide a mechanism to control the scanner’s projector, which was an important limitation because image acquisition alone would not be sufficient to reproduce the structured-light scanning process used by the Omnicam.

3) *Result and Limitations:* The investigation established that the Omnicam contained an identifiable Matrix Vision industrial camera that could be detected and accessed at the SDK level. The GenICam, GigE Vision, and mvIMPACT Acquire interfaces therefore provided a real hardware-level access point for further investigation. However, direct access to the camera through these interfaces was not sufficient to reproduce the scanner’s normal image acquisition. Although a grayscale image could eventually be obtained, the acquisition process could not be demonstrated to provide the complete set of functionality required for automated scanning. In particular, the inability to control the scanner’s projector through the available SDK represented a significant limitation. At this stage, the investigation therefore shifted from attempting to operate the camera solely through the Matrix Vision acquisition interface towards understanding how the

CEREC software and its associated components controlled and interacted with the scanner.

4) *Investigation Method: Software Interaction and Reverse Engineering:* After the direct acquisition approach through the Matrix Vision interfaces failed to provide a complete and reliable way of obtaining the images required for the project, the investigation shifted towards understanding how the original CEREC software interacted with the scanner.

The main objective of this phase was to identify the software components responsible for communicating with and controlling the Omnicam and to determine whether the scanner's acquisition process could be reproduced or controlled outside the original CEREC environment. Particular attention was given to the possibility that the CEREC software performed additional operations that were not exposed through the standard Matrix Vision acquisition SDK.

The investigation began by examining the components used by CEREC SW 5.1 during scanner operation. Process Monitor was used to observe the activity of the software while interacting with the scanner. This provided information about the processes, dynamic-link libraries, files, and other system resources involved during the acquisition workflow. The collected information was used to identify potentially relevant components for further analysis. The identified binaries were subsequently investigated using Ghidra. An example of the reverse-engineering environment used during this analysis is provided in Appendix A. The purpose of the reverse-engineering process was not to fully reconstruct the entire CEREC software, but to identify relevant functions, classes, data structures, and relationships that could provide information about the scanner's acquisition pipeline. Particular attention was given to the following DLLs:

- `acquire.dll`
- `CAI2_CameraCCT.dll`
- `SiAcquireCtrl.dll`
- `mvPropHandling.dll`
- `MHT_ScannerLib.dll`

The analysis focused on finding evidence of functionality related to several stages of the scanning process. These included scanner initialization, starting and stopping the scanning process, image acquisition, and the transfer or processing of acquired data. In addition, the investigation specifically looked for code that could provide information about the structured-light system, particularly functions or components related to projector control, pattern generation or sequencing, and synchronization between the projected patterns and camera acquisition.

The latter aspects were particularly relevant because obtaining camera frames alone would not necessarily reproduce the Omnicam's scanning process. If the

scanner relies on structured-light patterns, the acquired images would need to be synchronized with the corresponding projected patterns in order to reproduce the same type of measurement outside the original software. The reverse-engineering results were therefore used to construct a partial model of the scanner's software and data flow. Because this analysis was based primarily on decompiled binaries and indirect relationships between functions and components, the resulting architecture was treated as a set of working hypotheses rather than as a definitive description of the scanner's internal implementation.

The analysis was also subject to significant practical limitations. The CEREC software consisted of numerous interconnected binaries and libraries, and not all components could be interpreted reliably or easily from their decompiled representation. Indirect calls, complex dependencies, compiler-generated structures, and the amount of available code made it difficult to follow the complete execution path. Consequently, the investigation was able to identify several relevant components and data flows, but it was not possible to reconstruct the complete scanner acquisition and reconstruction pipeline with certainty.

C. Reverse Engineering Findings and Working Hypotheses

The reverse-engineering analysis provided evidence of several components and data paths involved in the Omnicam's scanning process. However, the following reconstruction should not be understood as a definitive description of the scanner's internal architecture. It was derived primarily from decompiled binaries, identified class and function relationships, and the interpretation of data structures and buffers. Consequently, some elements are directly supported by the analyzed code, while others represent working hypotheses based on the observed relationships.

1) *Identified Components:* One of the main components identified during the analysis was `ScannerProxy`, which appears to provide a high-level abstraction of the scanner hardware. The analyzed code contains functions such as `ScannerProxy::StartScanning(bool enable)`, `ScannerProxy::CreateUsbScanData2D(...)`, and `ScannerProxy::CreateUsbScanData25D(...)`. The class also appears to provide access to a `MainFpgaProxy` object through a virtual function call. Based on the analyzed relationships, `ScannerProxy` was interpreted as a component connecting several parts of the acquisition process, including camera acquisition, FPGA-related processing, and the creation of scan-data objects. `MainFpgaProxy` appears to represent an interface to the scanner's internal FPGA. Among the identified functions are `IsInterpolatedPeaksUsbData()`, `motor_hard_stop_front_in_microme`

ters(), and motor_hard_stop_back_in_micrometers(). The presence of these functions suggests that the FPGA interface is associated not only with data processing but also with aspects of the scanner's mechanical control.

The analysis also identified a hierarchy of scan-data objects. ScanData appears to act as a high-level container holding references to two different types of data, which were interpreted as ScanData2D and ScanData25D. The constructors observed in the decompiled code copy two independent shared_ptr objects, which is consistent with such a structure, although the complete original class definition could not be recovered. ScanData2D appears to represent the two-dimensional camera data. The function ScanDataHelper::GetRawTwoDScanData() was identified together with parameters corresponding to image width, height, and a raw data buffer. The analysis also identified. TwoDCameraProcess::DebayerBayeredImage(), providing evidence for a Bayer image processing stage. ScanData25D, in contrast, appears to represent processed data associated with the scanner's three-dimensional acquisition. The analyzed code distinguishes between raw and interpolated peak data. Depending on the selected mode, the data appears to be encapsulated in either UsbScanData25D or UsbScanData25DInterpolatedPeaks. Several constructors associated with these objects were also identified:

- FUN_180177750 → UsbScanData25D
- FUN_180177850 → UsbScanData25DInterpolatedPeaks
- FUN_180177960 → UsbScanData2D
- FUN_180183310 → UsbScanData2D

These functions were interpreted as object constructors rather than as functions performing the actual three-dimensional reconstruction.

2) *Identified Data Paths:* The analysis provided stronger evidence for the two-dimensional camera-data path than for the complete three-dimensional reconstruction pipeline. The identified camera configuration and the observed TwoDCameraProcess::DebayerBayeredImage() call suggest that the scanner can acquire image data in a Bayer mosaic representation before converting it to RGB through software processing. The corresponding data path can therefore be represented as:

Camera → RAW Bayer → Debayer → RGB

This interpretation is supported by the identification of the Bayer acquisition mode and the explicit debayering function in the analyzed code.

The creation of ScanData2D also provided evidence of a raw byte buffer being passed into the

scanner's data structures. In particular, ScannerProxyQcc61::CreateUsbScanData2D(...) accepts a shared_ptr<vector<unsigned char>> together with image dimensions. During the construction process, an ROI object was also observed, suggesting that the useful image region may be defined at this stage. The exact role and contents of all associated metadata, however, could not be established with certainty.

A separate data path appears to originate from the FPGA. The analyzed code distinguishes between raw peaks and interpolated peaks, controlled by the result of MainFpgaProxy::IsInterpolatedPeaks UsbData(). The corresponding conceptual flow was interpreted as:

FPGA → Raw or Interpolated Peak Data → ScanData25D

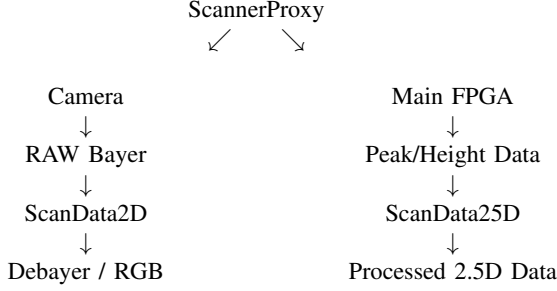
The observed object sizes were approximately 12,776 bytes (0x31E8) for the normal representation and 10,760 bytes (0x2A08) for the interpolated representation. These values provide evidence that different internal data representations are used, although their complete binary structure was not determined.

Overall, the analyzed code suggests that the scanner handles at least two related data streams: a two-dimensional camera stream and a second stream containing peak or height-related information. These streams are subsequently represented through the ScanData2D and ScanData25D structures.

3) *Partial Reconstruction of the Acquisition Flow:* Based on the identified functions and relationships, a partial reconstruction of the acquisition flow was developed. When the user initiates a scan, ScannerProxy::StartScanning(bool enable) appears to initiate or enable the scanning process. During this process, the ScannerProxy is associated with both camera-related functionality and the MainFpgaProxy. The analyzed relationships suggest the following conceptual flow:

Scan Initiation → ScannerProxy::StartScanning() → $\begin{cases} \text{Camera Acquisition} \\ \text{FPGA-related Processing} \end{cases}$

The camera-related path appears to produce RAW Bayer data that is subsequently processed through software debayering and represented as ScanData2D. In parallel, the FPGA-related path appears to produce peak or height-related data that is represented as ScanData25D. This can be summarized as:



This represents a working model rather than a fully verified implementation. In particular, the analysis did not establish that `StartScanning()` itself directly controls every stage of the acquisition process. Instead, it appears to initiate a larger sequence in which other modules perform the camera, FPGA, and potentially optical operations. The presence of the message “Generating faked volume data!” during the analysis of `StartScanning()` also suggests that the software contains an internal simulation or testing mode capable of generating artificial volume data. This provides additional evidence that the scanner software contains different internal acquisition and processing paths, although the exact purpose and activation conditions of this mode could not be fully established.

4) *Inferred Working Model of the Acquisition and Data Flow:* Based on the available evidence, the following model represents the most plausible interpretation of the scanner’s acquisition architecture at the time of the investigation:

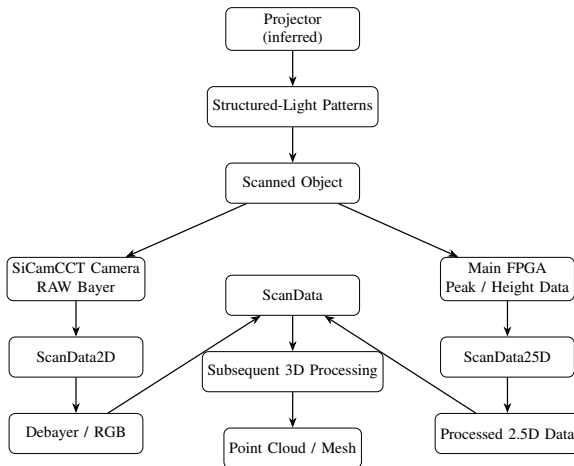


Figure 12: Inferred data flow of the Omnicam scanning pipeline.

The camera, FPGA, `ScanData2D`, and `ScanData25D` relationships shown in this model are supported to varying degrees by the analyzed code. The projector, structured-light pattern generation, synchronization, and final three-dimensional

processing are included as inferred components rather than confirmed findings.

5) *Unresolved Components and Limitations:* Despite identifying several important components, significant parts of the acquisition pipeline remained unresolved. The most important unresolved component was the projector control. The analysis did not identify explicit functions or modules clearly associated with terms such as `Projector`, `Pattern`, `GrayCode`, `PhaseShift`, `DLP`, `Sequence`, `Projection`, `LED`, or `Laser`. Consequently, no direct evidence was found showing where the structured-light patterns were generated or how the projector was controlled. Similarly, the synchronization mechanism between the projector and camera could not be identified. A complete automated acquisition process would likely require coordination between the projected pattern and the corresponding camera exposure, but the specific mechanism responsible for this synchronization remained unknown. The exact format of the internal data buffers was another unresolved aspect. Although several buffers were identified as `vector<unsigned char>` objects and their approximate sizes could be determined, their internal fields could not be fully reconstructed. Elements such as headers, frame identifiers, timestamps, peak values, calibration information, or coordinates could therefore not be confirmed. The final processing stages were also not fully identified. Although the presence of `ScanData25D` suggests that data related to three-dimensional acquisition is generated before later processing, the exact component responsible for transforming this information into a point cloud, mesh, or final three-dimensional model was not established.

Finally, the reverse-engineering process itself imposed substantial limitations. The CEREC software consisted of numerous interconnected DLLs, and several binaries were difficult to interpret reliably using decompilation. Indirect calls, compiler-generated structures, dependencies between modules, and the complexity of the assembly code made it difficult to trace complete execution paths. As a result, the analysis was sufficient to identify several relevant components and construct a plausible working model, but not to verify the complete internal architecture of the Omnicam with certainty or to identify a self-contained function that could be invoked from custom software to initiate and reproduce the complete scanning process.

D. UDP Communication and Data Capture Investigation

1) *Initial UDP Communication and API Prototype:* During the investigation of the scanner control commands, a small prototype API was developed in parallel to explore how a programmable scanner interface could be structured if a suitable command

for initiating the scan could be identified. At this stage, it was still assumed that such a command might be available and that it could potentially be integrated into a custom acquisition interface. The intended concept was to provide an interface capable of connecting to the scanner, initiating a scan through a command identified during the reverse-engineering process, receiving the resulting data, and subsequently forwarding it to the DentaVision processing pipeline. The objective was therefore not to provide a complete scanner implementation, but to investigate the software structure that could be used for such an interface. The prototype introduced an abstract hardware interface with operations for connecting to the scanner, projecting a pattern, capturing an image, and disconnecting from the device. A mock implementation was used to simulate these operations and to verify the intended control flow. A higher-level `Scanner` class could then execute a sequence consisting of connecting to the hardware, projecting several patterns, capturing the corresponding images, and disconnecting. This prototype was developed as a preliminary representation of the intended scanner-control architecture rather than as a functional interface to the physical Omnicam. Once the investigation progressed and no suitable command for independently initiating the scanning process could be identified, the prototype was no longer pursued as a means of controlling the physical scanner.

2) *Investigation and Reproduction of GVCP Commands:* As the reverse-engineering analysis had not identified a directly usable function for starting the scan from custom software, the network communication between the scanner and the CEREC software was investigated. The motivation for this approach came from observing the network activity of the system during a scan. In the Windows Task Manager, Ethernet usage increased noticeably when a scan was initiated in the CEREC software, suggesting that a significant amount of data could be exchanged through the network interface. This raised the possibility that the scanner might communicate with the CEREC software over the network, leading to the use of Wireshark to capture and investigate the corresponding traffic. Wireshark was used to observe the UDP traffic generated during scanner operation. Particular attention was given to communications occurring before and around the beginning of a scan, with the objective of identifying commands that could potentially be reproduced independently of the original software.

Based on this investigation, a C++ implementation of a UDP communication layer was developed. The `UDPClient` class provided basic UDP packet transmission and reception, while `GvcpClient` implemented read and write operations for registers using the observed GVCP command structure. A

`GvcpSequence` class was additionally implemented to execute a series of register operations in a defined order. Several register addresses were investigated and sequences containing read and write operations were tested. The scanner responded to some of the transmitted commands, demonstrating that communication with the device could be established through the implemented UDP interface. However, reproducing the complete sequence required to initiate the scanning process was not achieved. Consequently, although the experiment demonstrated that the scanner could respond to custom UDP/GVCP communication, it did not provide a reliable method for starting an acquisition independently from the CEREC software.

3) *Direct Capture of Scanner Data:* After the attempt to initiate the scanning process through reproduced commands was unsuccessful, a different approach was investigated. Instead of attempting to control the scanner directly, the CEREC software was used to initiate the scan manually while the custom program listened for the data transmitted by the scanner. A UDP receiver was implemented in C++ specifically for this purpose. The `GvspReceiver` class opened a UDP socket on the investigated GVSP port and waited for incoming datagrams. The received packets were inspected to extract information interpreted as block identifiers, packet identifiers, and packet types. The receiver attempted to distinguish between leader, data, and trailer packets and to group packets belonging to the same block. Data packets were then processed by removing the interpreted packet header and appending the remaining bytes to an output buffer. The accumulated data were subsequently written to a binary file named `frame.raw`. In addition, a separate `GvspAnalyzer` component was implemented to inspect previously captured data. It divided the captured stream into fixed-size packets and reported the interpreted block and packet identifiers, allowing the structure and continuity of the received data to be investigated. This implementation therefore demonstrated that data transmitted by the scanner could be captured independently of the CEREC application's normal display path and stored for further analysis.

4) *Observed Behaviour During Data Capture:* An important observation was made while performing the direct data-capture experiment. When the custom receiver was connected to the investigated communication port and started receiving the scanner's transmitted data, the live image that normally appeared in the CEREC software stopped being displayed. This behaviour was considered potentially significant because it suggested that the captured traffic could be related to the data stream used by the scanner during acquisition. However, the observation alone was not sufficient to establish that the received packets

represented the camera images directly.

In particular, the internal architecture analysis had already indicated that the scanner may exchange multiple types of data associated with the acquisition and 3D reconstruction process. Therefore, the captured stream could potentially contain image data, processed 3D-related data, metadata, or a combination of different data types. The possibility of additional encoding or an undocumented internal data format also could not be excluded. Consequently, the disappearance of the live image was treated as an indication that the captured traffic was relevant to the scanner's acquisition process, rather than as definitive evidence that the received bytes constituted directly usable image frames.

5) *Data Interpretation and Limitations:* Although the C++ receiver successfully captured and stored data transmitted by the scanner, the complete structure and meaning of the received stream could not be established. The captured binary data could not be reliably reconstructed into usable image frames. The investigation therefore did not establish whether the received data consisted exclusively of camera images, contained a combination of image and 3D-related information, or used an additional encoding or proprietary representation. The packet-level analysis provided information about the apparent structure of the communication, but it was insufficient to reconstruct the complete payload semantics. As a result, the `frame.raw` output should be understood as a capture of received binary data rather than as a successfully reconstructed image.

Combined with the lack of a reliable method for initiating the scan programmatically, the limited available documentation, and the difficulty of determining the internal data representation, this approach was ultimately considered impractical for obtaining image data from the Omnicam for the DentaVision pipeline. The scanner was therefore not used as the image source for the subsequent development. Instead, image acquisition was moved to the project's virtual/simulated pipeline, where the required camera input could be provided while allowing the remaining DentaVision processing and reconstruction pipeline to be developed independently of the limitations encountered with the physical scanner.

Examples of the output produced by the C++ UDP/GVSP implementation during the data capture experiments are provided in Appendix A. The four captures show representative outputs obtained during the communication and packet-analysis experiments.

E. Conclusion and Future Considerations

The investigation successfully identified the main camera hardware and acquisition interfaces, provided access to relevant device information, examined the available SDKs, identified several relevant software

components, and allowed parts of the scanner's acquisition and data flow to be inferred through reverse engineering. It was also possible to establish UDP communication with the scanner and capture data transmitted during an acquisition. However, no reliable method was found to obtain directly usable images for the DentaVision pipeline, fully control the scanner, identify the synchronization mechanism of the structured-light system, or completely interpret the captured data.

As a result, the investigation of direct image acquisition from the Omnicam was concluded, and the scanner was not used as the image source for the subsequent development of DentaVision. Nevertheless, the work provided a partial understanding of the scanner's internal architecture and identified several relevant acquisition and processing paths. For future investigations, the use of a programmable scanner with documented access to its acquisition process would be preferable.

IV. 3D RECONSTRUCTION

A. Theoretical Background

Three-dimensional reconstruction from images can be approached through different computer vision paradigms. The methods investigated in this project range from feature-based reconstruction and Structure-from-Motion (SfM) to classical stereo matching and learning-based stereo estimation. Although these approaches differ in their assumptions and internal processing, they all rely on establishing relationships between different observations of the same scene in order to infer three-dimensional information.

The following sections introduce the main concepts required to understand the reconstruction methods implemented and evaluated in this work. The discussion begins with feature-based correspondence using the Scale-Invariant Feature Transform (SIFT), followed by Structure-from-Motion, classical stereo vision and disparity estimation, and finally learning-based stereo matching using IGEV-Stereo.

1) Feature-Based Reconstruction and SIFT:

Feature-based reconstruction relies on identifying distinctive visual structures in one or more images and establishing correspondences between observations of the same physical points. These correspondences provide the basis for estimating geometric relationships between images and, ultimately, recovering three-dimensional information about the observed scene.

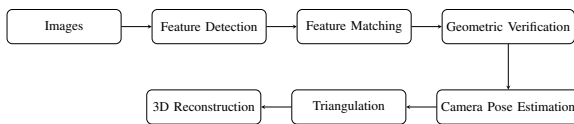
A typical feature-based pipeline consists of two main stages: feature detection and feature description. First, visually distinctive locations, commonly referred to as *keypoints*, are identified within each image. A descriptor is then computed for the local image region surrounding each keypoint. The descriptors can subsequently be compared between images to

determine which keypoints are likely to represent the same physical location. One of the most widely used feature extraction methods is the Scale-Invariant Feature Transform (SIFT), introduced by Lowe [11]. SIFT detects distinctive image features across different scales and computes a descriptor that characterizes the local appearance around each detected keypoint. An important property of the method is its invariance to image scale and rotation, while also providing robustness to moderate changes in viewpoint, illumination, noise and affine distortion [11].

After features have been extracted from multiple images, their descriptors can be compared to establish candidate correspondences. These correspondences are subsequently filtered using geometric constraints in order to remove incorrect matches. The resulting set of reliable correspondences can then be used by higher-level reconstruction algorithms to estimate camera motion or recover the three-dimensional position of observed points. In the context of three-dimensional reconstruction, SIFT therefore does not directly generate a three-dimensional model. Instead, it provides the feature correspondences required by subsequent geometric operations such as pose estimation and triangulation. This distinction is particularly relevant to the reconstruction approaches investigated in this project, where SIFT was used as the basis for both pairwise reconstruction and Structure-from-Motion pipelines.

An inherent limitation of feature-based reconstruction is its dependence on the availability of sufficiently distinctive visual features. Regions with little texture, homogeneous appearance or repetitive structures may provide fewer reliable keypoints and correspondences. As a result, the performance of feature-based approaches can vary considerably depending on the visual characteristics of the observed scene. This limitation motivated the subsequent evaluation of alternative reconstruction approaches based on stereo disparity estimation.

2) *Structure-from-Motion*: Structure-from-Motion is a computer vision approach that estimates the three-dimensional structure of a scene together with the relative positions and orientations of the cameras from multiple images. Starting from correspondences between image features, SfM jointly recovers the geometry of the observed scene and the motion or pose of the cameras that captured it [12], [13]. A simplified SfM pipeline can be described as follows:



The process begins by detecting and matching features between images. The resulting correspon-

dences are used to estimate the geometric relationship between different camera views. In the case of calibrated cameras, this relationship can be represented through the essential matrix, which encodes the relative rotation and translation between two camera views. Once the relative camera poses have been estimated, corresponding image observations can be triangulated to estimate the three-dimensional position of the observed points. The reconstructed three-dimensional points are commonly referred to as *landmarks*. In multi-view reconstruction, the same landmark may be observed in several images. Maintaining the association between these individual observations and their corresponding three-dimensional points requires the management of *tracks*, which represent groups of feature observations that are assumed to correspond to the same physical point. As additional images are incorporated into the reconstruction, the estimated camera poses and three-dimensional landmarks are refined through bundle adjustment. Bundle adjustment jointly optimizes the reconstruction parameters in order to minimize the reprojection error between the observed image features and the projections of the estimated three-dimensional points [12].

Although the individual operations involved in SfM can be implemented using general-purpose computer vision libraries, constructing a complete and robust SfM pipeline requires managing feature correspondences, tracks, camera poses, landmarks and iterative optimization. Modern SfM systems therefore integrate these operations into specialized reconstruction pipelines. For example, Schönberger and Frahm describe an incremental Structure-from-Motion pipeline designed to improve robustness, accuracy, completeness and scalability when reconstructing scenes from unordered image collections [13]. This complexity was directly relevant to the development of the reconstruction pipeline in this project. An initial manual implementation demonstrated that performing individual operations such as feature matching, pose estimation and triangulation was not sufficient to construct a complete multi-view reconstruction system without additionally managing the relationships and consistency between all observations. Consequently, established SfM and multi-view reconstruction tools were later investigated as an alternative to implementing the complete pipeline manually.

3) *Stereo Vision and Disparity*: Stereo vision estimates three-dimensional information by observing the same scene from two different viewpoints. In a stereo camera configuration, two cameras observe the scene from slightly different positions. Consequently, the projection of the same physical point appears at different image locations in the left and right images. After the stereo images have been rectified, corresponding points are typically located along the

same horizontal image coordinate. This simplifies the correspondence problem by reducing the search for matching points primarily to the horizontal direction. The horizontal displacement between corresponding points is referred to as *disparity*. For a rectified stereo pair, disparity can be expressed as

$$d = x_L - x_R \quad (1)$$

where x_L and x_R represent the horizontal coordinates of corresponding points in the left and right images, respectively. The estimated disparity is inversely related to the depth of the observed point. For an ideal stereo configuration, depth can be calculated as

$$Z = \frac{fB}{d} \quad (2)$$

where Z represents the depth, f is the focal length, B is the baseline between the two camera centers, and d is the disparity. Therefore, points with a larger disparity are generally located closer to the cameras, whereas points with a smaller disparity are located farther away [14], [15].

The main challenge in stereo reconstruction is the correspondence problem: determining which pixel or image region in the left image corresponds to the same physical point in the right image. Stereo matching algorithms estimate these correspondences and generate a *disparity map*, in which each valid pixel is associated with an estimated horizontal displacement. Classical stereo matching methods generally combine a matching cost with spatial regularization. The matching cost evaluates the similarity between candidate image regions, while the regularization process encourages geometrically consistent disparity estimates across neighboring pixels. One influential approach is Semi-Global Matching (SGM), proposed by Hirschmüller [9]. SGM approximates global energy optimization by aggregating matching costs along multiple paths through the image while incorporating smoothness constraints. This allows the method to balance local image similarity with spatial consistency without performing a computationally expensive global optimization.

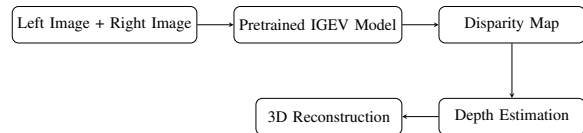
The Stereo Semi-Global Block Matching implementation provided by OpenCV, referred to as StereoSGBM, follows this general class of stereo matching approaches and was used in this project to generate disparity maps from rectified stereo image pairs. The resulting disparity estimates were subsequently used to reconstruct depth and three-dimensional point clouds. Unlike Structure-from-Motion, which estimates camera motion and scene structure from multiple observations, stereo reconstruction can directly estimate depth from the disparity between two views when the relevant camera parameters and stereo geometry

are known. This makes stereo matching particularly suitable for evaluating the reconstruction of individual image pairs.

4) *Learning-Based Stereo Matching and IGEV*: Classical stereo matching methods rely on explicitly designed similarity measures and optimization strategies to establish correspondences between stereo images. Their performance can be affected by ambiguous correspondences, low-texture regions, repetitive patterns and other image characteristics that make it difficult to identify corresponding pixels reliably. Learning-based stereo matching approaches address the correspondence problem by using neural networks trained on stereo data. Instead of relying exclusively on manually designed image features or matching costs, these methods learn feature representations and correspondence relationships from training data. During inference, a trained model receives a stereo image pair and estimates a disparity map representing the relative displacement between corresponding pixels.

One such method is IGEV-Stereo, proposed by Xu et al. [10]. IGEV-Stereo introduces the Iterative Geometry Encoding Volume, a representation designed to combine geometric and contextual information with local matching information. The resulting representation is used iteratively to refine the estimated disparity map. The method builds upon iterative stereo estimation approaches but introduces additional geometric information into the representation used during disparity refinement. According to Xu et al., the geometry encoding volume is designed to provide information that is not sufficiently represented by conventional all-pairs correlation alone, particularly in ambiguous or ill-posed image regions [10].

An important distinction between the learning-based approach used in this project and the development of a neural network from scratch is the use of a pretrained model. Training a stereo network requires a substantial amount of stereo data, computational resources and optimization time. Instead, the implementation evaluated in this work used a pretrained IGEV model to perform inference on the selected stereo image pairs. The reconstruction process can therefore be summarized as



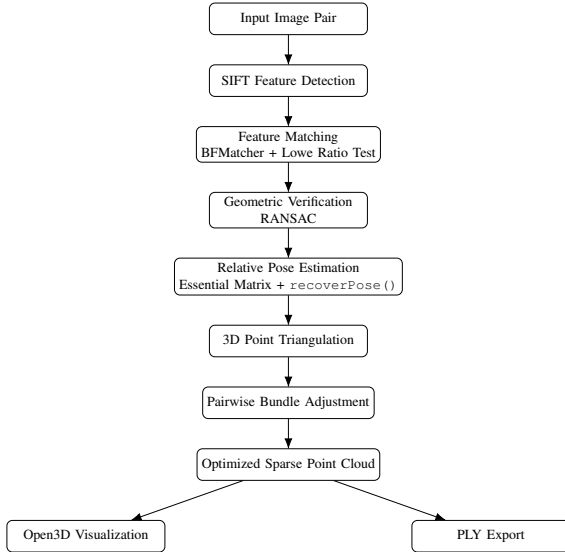
This approach allowed the learning-based stereo method to be integrated into the reconstruction pipeline without performing model training. The resulting disparity maps could then be evaluated using the same underlying stereo geometry used for the classical StereoSGBM approach.

Together, feature-based reconstruction, Structure-from-Motion, classical stereo matching and learning-

based stereo estimation represent different approaches to recovering three-dimensional information from images. The following sections describe how these approaches were implemented and evaluated within the project, beginning with feature-based pairwise reconstruction using SIFT.

B. Pairwise Feature-Based Reconstruction with SIFT

One of the reconstruction approaches investigated in the project is a feature-based pairwise reconstruction pipeline using the **Scale-Invariant Feature Transform (SIFT)**. The purpose of this approach was to investigate whether sparse 3D geometry could be reconstructed directly from corresponding features between individual image pairs. This approach represents only one of the reconstruction strategies explored in the project. It was selected as a relatively simple implementation of two-view Structure from Motion (SfM), allowing the individual geometric components of the reconstruction process to be investigated independently. The resulting sparse point clouds were also suitable for subsequent processing, such as point cloud registration. Unlike a complete multi-view SfM system, the implementation does not attempt to maintain a globally consistent reconstruction across an entire image sequence. Instead, each image pair is processed independently. This substantially reduces the complexity of the reconstruction problem while retaining the fundamental geometric operations required for two-view reconstruction. The resulting workflow is:



The following subsections describe the individual components of this pairwise reconstruction approach.

1) *Motivation for a Pairwise Approach:* Implementing a complete Structure from Motion system from low-level computer vision primitives is considerably more complex than implementing the individual reconstruction operations in isolation. While operations such as feature detection, feature matching,

essential matrix estimation, camera pose recovery, and triangulation are available in OpenCV, a complete multi-view SfM system additionally requires persistent management of camera poses, feature observations, feature tracks, 3D landmarks, geometric constraints, scale ambiguity, outlier rejection, and global optimization. A complete implementation would therefore require a substantially larger architecture than the individual two-view reconstruction operations. For this reason, the feature-based implementation uses an independent pairwise reconstruction model. Each consecutive image pair can be processed without requiring information from previous or subsequent pairs. This makes the individual reconstruction stages easier to inspect, test, and debug while still providing the essential geometry of a two-view SfM problem. This decision does not imply that pairwise reconstruction is equivalent to a complete multi-view SfM system. Rather, it provides a controlled intermediate approach between individual two-view operations and a globally optimized multi-view reconstruction. In particular, the implementation does not maintain a global camera trajectory, persistent feature tracks, global landmarks, or a globally consistent coordinate system. These aspects would be required for a complete multi-view SfM implementation and were therefore outside the scope of this specific reconstruction approach.

2) *Input and Pairwise Processing:* The reconstruction starts with two input images. At the current stage of the project, images are loaded from a dataset stored on disk by the `Camera` class. Each loaded image is represented by a `Frame` object. The `Frame` stores the original image together with the information generated during subsequent processing stages. This includes the image filename and path, SIFT keypoints, SIFT descriptors, and camera-related information associated with the reconstruction.

For an individual reconstruction, two frames are selected as an image pair. For a sequence of images,

$$I_1, I_2, \dots, I_N, \quad (3)$$

the implementation can process consecutive pairs:

$$(I_1, I_2), (I_2, I_3), \dots, (I_{N-1}, I_N). \quad (4)$$

For a sequence containing N images, this results in $N - 1$ independent image pairs. Each pair is reconstructed separately and produces its own sparse point cloud. Although consecutive pairs share an image, their reconstructions are not automatically aligned and therefore remain in independent coordinate systems.

The use of the `Camera` abstraction keeps image acquisition separated from the reconstruction stages. Consequently, replacing the current dataset-based input with another image source does not require the higher-level feature and geometry components to be redesigned.

The high-level `Reconstructor` orchestrates the geometric reconstruction once a `MatchResult` has been prepared. The `MatchResult` contains the two frames, their feature correspondences, geometrically valid inliers, and the estimated relative camera pose. The `Reconstructor` passes this information to the `Triangulator`, which reconstructs the valid correspondences as 3D points and returns a `TriangulationResult`. The resulting sparse `PointCloud` can be used directly after triangulation without requiring Bundle Adjustment. If additional optimization is desired, the triangulated points and their image observations can instead be passed to the separate `BundleAdjustment` component. The `Reconstructor` does not maintain state across image pairs. No global landmark database, camera trajectory, or persistent feature-track structure is required. This allows individual pairs to be inspected and evaluated independently while keeping the implementation simpler than a complete multi-view SfM system.

3) *SIFT Feature Detection*: The first processing stage is the detection of local image features using SIFT. Each image is processed independently. SIFT identifies distinctive local structures in an image and represents each detected feature using a keypoint and a descriptor. The keypoint contains the location and scale of the detected feature, while the descriptor encodes the local image appearance. SIFT was selected because its feature representation is designed to be relatively robust to changes in scale and rotation and to moderate changes in illumination and viewpoint. This makes it suitable for establishing correspondences between images acquired from different camera positions. The detected keypoints and their corresponding descriptors are stored in the respective `Frame` objects. These descriptors form the input to the subsequent feature matching stage. The feature detector is implemented as a separate `FeatureDetector` component. It can operate on individual frames or on a sequence of frames, allowing feature extraction to remain independent from the subsequent geometric reconstruction.

4) *Feature Matching*: Once the SIFT descriptors have been extracted, descriptors from the two images are compared to identify potential correspondences. The implementation uses OpenCV's `BFMatcher`. For each descriptor in the first image, the matcher searches for the closest descriptors in the second image according to the descriptor distance. The two nearest candidate matches are used by the **Lowe Ratio Test**. A correspondence is accepted only when the best match is sufficiently better than the second-best match. This removes many ambiguous correspondences in regions where multiple image features have similar descriptors. The resulting correspondences are stored

in a `MatchResult`. The object therefore becomes the main data structure passed from feature matching to the geometric stages of the reconstruction. At this point, however, the matches are only appearance-based correspondences. Some may still be geometrically incorrect.

5) *Geometric Verification and Outlier Rejection*: Feature matching alone does not guarantee that all correspondences originate from the same physical scene geometry. Incorrect correspondences can lead to invalid camera poses and incorrect 3D points. The implementation therefore performs geometric verification using RANSAC-based estimation of the Essential Matrix. The Essential Matrix describes the relative geometry between two calibrated camera views. RANSAC repeatedly evaluates subsets of the available correspondences and identifies a model that is consistent with the largest possible set of observations. The resulting correspondences are separated into geometrically consistent *inliers* and rejected *outliers*. Only the inliers are used by the subsequent pose estimation and triangulation stages. The `MatchResult` stores both the matching information and the geometric information required by the later stages, including the inlier matches and the RANSAC mask. This separation between appearance-based matching and geometric verification is important because it allows the quality of the correspondences to be evaluated before attempting 3D reconstruction.

6) *Relative Camera Pose Estimation*: After geometric verification, the relative camera motion between the two images is estimated. The Essential Matrix is computed using OpenCV's `cv2.findEssentialMat()`, followed by relative pose recovery using `cv2.recoverPose()`. The recovered pose consists of a rotation matrix R and a translation vector t . The implementation uses the world-to-camera transformation convention

$$X_{\text{camera}} = RX_{\text{world}} + t. \quad (5)$$

For each pairwise reconstruction, the first camera defines the reference coordinate system. Its pose is therefore

$$R_1 = I, \quad t_1 = 0. \quad (6)$$

The second camera is represented by the relative transformation estimated from the image pair:

$$X_2 = RX_1 + t. \quad (7)$$

The resulting rotation and translation are stored in the `MatchResult` and are subsequently used by triangulation and, when applicable, by dense stereo reconstruction.

An important limitation of monocular two-view reconstruction is that the translation recovered from

the Essential Matrix has an unknown absolute scale. Consequently, the resulting 3D reconstruction is defined only up to an arbitrary scale factor. The pairwise point cloud therefore describes the relative geometry of the scene rather than providing metric dimensions.

7) *3D Point Triangulation*: Once the relative camera pose has been estimated, the geometrically valid correspondences are triangulated to obtain their three-dimensional positions. The projection matrix of the first camera is constructed as

$$P_1 = K \begin{bmatrix} I & 0 \end{bmatrix}, \quad (8)$$

while the second camera is represented as

$$P_2 = K \begin{bmatrix} R & t \end{bmatrix}, \quad (9)$$

where K denotes the camera intrinsic matrix.

The implementation uses OpenCV's `cv2.triangulatePoints()` to reconstruct the 3D position of each corresponding pair of image observations. The resulting homogeneous coordinates are then converted into Euclidean 3D coordinates. The `Triangulator` is responsible for this operation. It receives a `MatchResult` together with the camera intrinsic matrix and returns a `TriangulationResult`. The result contains the reconstructed `PointCloud` as well as the corresponding image observations from both images. Keeping the image observations together with the reconstructed points is important because the same correspondences are required later by Bundle Adjustment. The triangulation stage additionally performs geometric validity checks. Reconstructed points must satisfy the cheirality condition, meaning that they must lie in front of both cameras. Points with excessive reprojection error are also rejected.

For every retained point, the implementation therefore keeps the 3D position, its observation in image 1, its observation in image 2, and its optional RGB color information aligned by index. RGB colors can be sampled from the first input image at the corresponding feature locations. These colors are stored together with the reconstructed points in the resulting `PointCloud`.

8) *Pairwise Bundle Adjustment*: Triangulation provides an initial estimate of the scene geometry, but both the estimated camera pose and the reconstructed point positions are affected by image measurement noise and errors in the initial geometric estimation. Bundle Adjustment was therefore implemented as an additional optimization stage that can be applied to the pairwise reconstruction when required. However, it is not a mandatory component of the `Reconstructor` and is kept separate from the main sparse reconstruction path. This distinction was made because Bundle Adjustment can become computationally expensive for image pairs producing a large number of

reconstructed points. During experimentation, some pairs required considerable optimization time while providing only a limited improvement in reprojection error. Consequently, applying Bundle Adjustment to every image pair was not considered necessary or efficient. When Bundle Adjustment is enabled, the first camera remains fixed as the reference camera,

$$R_1 = I, \quad t_1 = 0, \quad (10)$$

while the optimization variables consist of the pose of the second camera and the 3D coordinates of the reconstructed points. For each 3D point, its estimated position is projected into both images. The difference between the projected positions and the original feature observations forms the reprojection residual. The optimization therefore minimizes the combined reprojection error of the two views. Conceptually, the objective is

$$\min \left(\sum_i \|\hat{x}_{1,i} - x_{1,i}\|^2 + \sum_i \|\hat{x}_{2,i} - x_{2,i}\|^2 \right), \quad (11)$$

where $x_{1,i}$ and $x_{2,i}$ are the observed image points and $\hat{x}_{1,i}$ and $\hat{x}_{2,i}$ are their corresponding projected points.

The optimization is represented by the `BAPProblem` class. This object contains the second camera's rotation and translation, the reconstructed 3D points, and the corresponding observations in both images. The `BundleAdjustment` class performs the numerical optimization using SciPy's `least_squares`. A sparse Jacobian structure is used because an individual observation depends only on the parameters of the second camera and its corresponding 3D point. This avoids constructing unnecessary dependencies between unrelated points. When used, Bundle Adjustment transforms the initial triangulated reconstruction into an optimized pairwise solution consisting of an optimized camera pose and optimized 3D point positions. The resulting 3D coordinates can then be represented as a `PointCloud` for visualization or export.

9) *Point Cloud Representation and Output*:

The reconstructed geometry is represented by the `PointCloud` class. The point cloud contains the reconstructed 3D positions and, when available, RGB colors. The point positions are represented as an $N \times 3$ array,

$$\mathbf{P} = \begin{bmatrix} X_1 & Y_1 & Z_1 \\ X_2 & Y_2 & Z_2 \\ \vdots & \vdots & \vdots \\ X_N & Y_N & Z_N \end{bmatrix}. \quad (12)$$

The point cloud provides the main output of this reconstruction approach and can be passed to sub-

sequent processing stages. For qualitative inspection, the `Visualizer` converts the internal representation into an Open3D point cloud and displays it interactively. When RGB information is available, the point colors are preserved during visualization. For persistent storage, the `PointCloudWriter` converts the internal representation into an Open3D point cloud and exports it in PLY format. This allows the reconstructed geometry to be inspected using external 3D visualization and processing tools. It also prepares the point cloud for the subsequent registration module.

10) Limitations of the Pairwise SIFT Reconstruction: The pairwise SIFT reconstruction provides the fundamental geometry required for two-view sparse reconstruction, but its deliberate simplification also introduces limitations. The implementation does not maintain a globally consistent camera trajectory or coordinate system. Persistent feature tracks are not constructed across multiple image pairs, and reconstructed landmarks are not associated between different pairwise point clouds. Consequently, two consecutive reconstructions can contain overlapping physical regions while remaining independent in their coordinate systems. The point cloud reconstructed from (I_1, I_2) and the point cloud reconstructed from (I_2, I_3) are therefore not automatically expressed in a common global frame. Similarly, Bundle Adjustment is restricted to the individual image pair. There is no global optimization capable of jointly refining all camera poses and landmarks across an entire image sequence.

These limitations are a direct consequence of the chosen pairwise architecture. They are acceptable for investigating individual reconstruction stages and for producing local sparse geometry, but a globally consistent SfM reconstruction would require additional components such as persistent feature tracks, landmark management, camera-pose composition, inter-pair registration, and global Bundle Adjustment.

11) Evaluation on Palm Desert: The pairwise SIFT reconstruction was initially evaluated using the Palm Desert Micro dataset. The evaluation focused on the image pair `DJI_0042.JPG` and `DJI_0043.JPG`. The results demonstrate that the pipeline can establish a large number of reliable feature correspondences and reconstruct a substantial sparse set of 3D points under favorable visual conditions.

Table V: Results of the pairwise SIFT reconstruction on the Palm Desert image pair.

Metric	Result
Image 1 keypoints	20,001
Image 2 keypoints	20,000
Good matches	3,476
RANSAC inliers	2,959
Inlier ratio	85.13%
Reconstructed points	2,959
Reprojection error before BA	0.1936 px
Reprojection error after BA	0.1876 px
BA improvement	3.11%

Bundle Adjustment reduced the mean reprojection error from 0.1936 px to 0.1876 px, corresponding to an improvement of 3.11%. The optimization reached the configured maximum number of function evaluations, so the improvement was relatively limited despite the additional computational cost. Overall, the Palm Desert experiment provided an initial validation of the pairwise reconstruction workflow under conditions with sufficient visual structure.



Figure 13: Sparse 3D reconstruction obtained from the Palm Desert image pair. The reconstructed points capture recognizable scene structure, including the main hillside, while remaining distributed throughout the surrounding scene.

As shown in Figure 13, the resulting point cloud is sparse and does not form a continuous surface. Nevertheless, recognizable scene structure, including the main hillside visible in the input images, can be identified in the reconstruction. Additional reconstructed points are distributed across the surrounding scene, reflecting the feature-based nature of the approach rather than a dense surface reconstruction.

12) Experimental Dense Reconstruction: In addition to the sparse SIFT-based reconstruction, an experimental dense stereo module was investigated. This module is intentionally treated as an optional component rather than as a required stage of the main feature-based reconstruction pipeline.

The `DenseReconstructor` operates on a single image pair and reuses the relative rotation and translation already estimated during the feature-based pipeline. The images are first downsampled to reduce

computational cost and their intrinsic matrix is scaled accordingly. The pair is then rectified using OpenCV’s `cv2.stereoRectify()`, after which dense disparity is estimated using StereoSGBM. The resulting disparity map is converted into 3D coordinates using `cv2.reprojectImageTo3D()` and the reprojection matrix obtained during rectification. The module additionally performs validity checks and robust depth filtering based on the Median Absolute Deviation (MAD). RGB values are sampled from the rectified reference image and associated with the resulting 3D points. This approach can produce substantially denser geometry than the sparse feature-based reconstruction because depth is estimated over a large number of image pixels rather than only at selected SIFT correspondences.

However, experiments showed that the dense reconstruction was not equally reliable for all image pairs. Its quality depended strongly on the relative camera motion, image overlap, scene texture, disparity distribution, and quality of the estimated relative pose. Some pairs produced meaningful dense geometry, whereas other pairs generated distorted or geometrically incorrect structures. For this reason, the dense stereo implementation remains experimental. It was not considered necessary for the subsequent processing stages because the sparse feature-based reconstruction already provides a set of 3D points suitable for operations such as point cloud registration. The dense module can therefore be evaluated or improved independently without becoming a dependency of the main reconstruction workflow.

C. OpenMVG/OpenMVS Reconstruction Pipeline

To evaluate an established Structure-from-Motion (SfM) and multi-view stereo (MVS) reconstruction backend, a second reconstruction implementation was developed in `src_v2/`. Unlike the custom OpenCV-based reconstruction pipeline described previously, this implementation does not reproduce the individual SfM algorithms directly. Instead, Python provides the configuration, process orchestration, file management, and visualization interface, while the computational reconstruction stages are delegated to OpenMVG and OpenMVS. This pipeline is intentionally maintained as a separate reconstruction alternative. In particular, it is not an additional registration stage of the main DentaVision reconstruction pipeline: OpenMVG performs its own camera registration and SfM reconstruction as part of the established pipeline.

1) Motivation for Using an Established SfM Pipeline: The original reconstruction implementation was developed using OpenCV in order to expose and control the individual components of the SfM process, including SIFT feature extraction, feature matching, geometric verification, relative pose estimation, track construction, triangulation, landmark management,

and bundle adjustment. This approach is useful for understanding the reconstruction process and for experimenting with individual algorithms, but it also requires substantial implementation and maintenance effort. In contrast, OpenMVG provides an established implementation of the complete sparse SfM workflow, including feature processing, matching, geometric filtering, camera estimation, triangulation, and optimization. The purpose of introducing OpenMVG was therefore not to replace the custom implementation, but to provide an alternative reconstruction backend in which the project could evaluate a more complete and established SfM implementation without having to reimplement the underlying reconstruction algorithms.

This distinction is also relevant when interpreting later experiments. Although OpenMVG provides the complete SfM framework, the feature extraction stage used by the project remains SIFT. Consequently, the use of an established SfM implementation does not eliminate limitations originating from the input features. In particular, scenes or images containing insufficient distinctive features can still present difficulties for feature matching and subsequent reconstruction. This makes the OpenMVG pipeline useful not only as a reconstruction backend, but also as an independent reference for evaluating the limitations of feature-based reconstruction.

2) OpenMVG Pipeline: The OpenMVG integration is implemented as a thin Python wrapper around the OpenMVG command-line applications. The project configuration is centralized in `src_v2/config/settings.py`, where the input image directory, OpenMVG installation, sensor-width database, output directories, feature extraction parameters, matching parameters, camera configuration, and SfM configuration are defined. Machine-dependent paths, including the input image directory and external software installations, are loaded from the project’s `.env` file, while the OpenMVG executable directory and sensor-width database are derived from `OPENMVG_ROOT`. This keeps the reconstruction code independent of a particular installation path and allows camera- and dataset-dependent parameters to be changed without modifying the reconstruction pipeline itself.

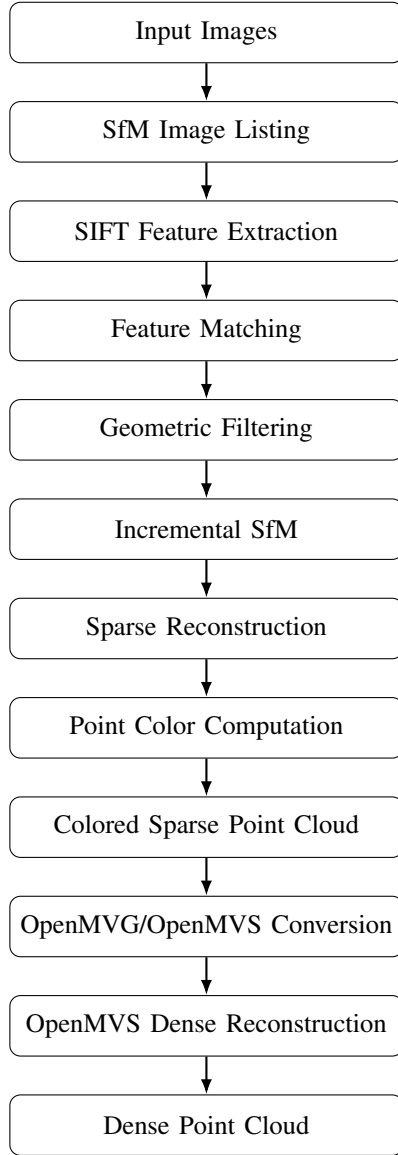


Figure 14: End-to-end OpenMVG/OpenMVS reconstruction workflow.

The sparse stage is coordinated by `ReconstructionPipeline.run_sparse()`. First, `openMVG_main_SfMInit_ImageListing` creates the initial `sfm_data.json` from the input images and the configured camera parameters. Camera-dependent values are centralized in `settings.py`. When explicit intrinsic parameters are available, the configured focal length and principal point are passed directly to OpenMVG. If explicit focal-length information is not provided, OpenMVG can instead use its sensor-width database together with the available image metadata to initialize the camera intrinsics. This allows the same reconstruction pipeline to be used with different cameras and image sets without requiring changes to the reconstruction logic.

Feature extraction is then performed by

`openMVG_main_ComputeFeatures`. The project uses SIFT with the NORMAL descriptor preset and configures 16 processing threads. The resulting feature regions and descriptors are subsequently used by `openMVG_main_ComputeMatches`, with a matching ratio configured in `settings.py`. The resulting `matches.putative.bin` file contains the initial feature correspondences.

These matches are subsequently passed to `openMVG_main_GeometricFilter`, which uses the geometric model configured by the project to remove correspondences that are inconsistent with the estimated camera geometry. The selected geometric model is therefore also treated as a configuration parameter rather than being hard-coded into the reconstruction pipeline. The filtered correspondences are stored as `matches.f.bin`.

The verified matches are then provided to `openMVG_main_SfM`. The configured SfM engine is INCREMENTAL, so OpenMVG progressively reconstructs the camera poses and sparse 3D structure from the available image correspondences. The reconstruction produces the OpenMVG SfM representation under `output/openmvg/sfm/`, including `sfm_data.bin`. Finally, `openMVG_main_ComputeSfM_DataColor` is used to generate the colored sparse point cloud `cloud_and_poses.ply`. The visualization layer can consume this point cloud without being coupled to the internal OpenMVG representation.

The Python implementation deliberately keeps process execution separate from reconstruction logic. The generic `run_command()` helper validates executable paths, constructs and prints the generated command, and invokes the external process using `subprocess.run(..., check=True)`. Consequently, OpenMVG failures are propagated to the Python pipeline instead of allowing subsequent stages to operate on invalid intermediate data. The same abstraction is reused for the OpenMVS stage.

3) *Dense Reconstruction with OpenMVS*: OpenMVG provides the sparse reconstruction, whereas dense reconstruction is delegated to OpenMVS. The implementation separates this process into two independent operations: `prepare_dense()` converts the existing OpenMVG reconstruction into an OpenMVS scene, while `run_dense()` performs the actual dense reconstruction. This separation allows an existing scene to be reused without repeating the sparse reconstruction or the scene conversion.

The scene preparation stage uses `openMVG_main_openMVG2openMVS.exe`. Its primary input is the OpenMVG reconstruction `sfm_data.bin`, and it generates `output/openmvs/scene.mvs` together with the associated undistorted image data. The resulting

scene acts as the interface between the sparse OpenMVG representation and the OpenMVS dense reconstruction stage. The implementation also ensures that the image references required by the scene remain accessible when OpenMVS loads the scene; the working directory used for the dense reconstruction is therefore set to the directory containing the scene.

Dense reconstruction is performed by `DensifyPointCloud.exe`. The generated point cloud is stored as `output/openmvs/dense/pointcloud.ply`. The implementation supports CUDA processing while retaining a CPU fallback. With the current configuration, `OPENMVS_CUDA_DEVICE=None`, OpenMVS is first invoked with automatic GPU selection (-1); if that attempt fails, the operation is retried using CPU processing (-2). A specific CUDA device can alternatively be configured, in which case the selected device is attempted first, followed by automatic GPU selection and CPU processing. If a failed processing attempt leaves an output file behind, that file is removed before the next attempt so that an incomplete reconstruction cannot be mistaken for a successful result.

The resulting architecture therefore separates the responsibilities of the two reconstruction backends: OpenMVG performs feature-based sparse SfM and produces camera poses and sparse 3D structure, while OpenMVS uses that calibrated scene to generate a substantially denser point cloud. Camera configuration and other dataset-dependent parameters remain external to the reconstruction logic, allowing the same pipeline implementation to be reused with different input image sets by modifying the project settings. Python remains responsible for coordinating these stages and managing their intermediate files rather than implementing the underlying SfM or MVS algorithms.

4) *3D Reconstruction Results on the Palm Desert Dataset*: The sparse reconstruction generated by OpenMVG was subsequently used as input for dense reconstruction with OpenMVS. The input scene contained 21 calibrated images and 83,416 sparse 3D points. OpenMVS estimated depth maps for all 21 views, enforced geometric consistency between the views, and fused the resulting depth maps into a dense point cloud. After depth-map fusion, filtering, and trimming to the estimated region of interest, the final dense point cloud contained 1,753,190 3D points. This corresponds to approximately 21 times the number of points in the original sparse reconstruction, providing a substantially denser representation of the reconstructed scene.

Table VI: OpenMVG/MVS dense reconstruction results.

Metric	Result
Calibrated images	21
Sparse points	83,416
Dense points	1,753,190
Dense/Sparse ratio	$\approx 21\times$

Figure 15 provides a visual comparison between the sparse reconstruction produced by OpenMVG and the final dense point cloud generated by OpenMVS. In the sparse reconstruction, the green markers represent the estimated camera positions, whose distribution reflects the relative camera motion recovered from the image correspondences. The coherent camera trajectory provides a visual indication that the feature tracks and pose estimation successfully captured the camera movement throughout the sequence. The comparison also illustrates the substantial increase in point density obtained during the dense reconstruction stage while preserving the overall geometry of the reconstructed scene.

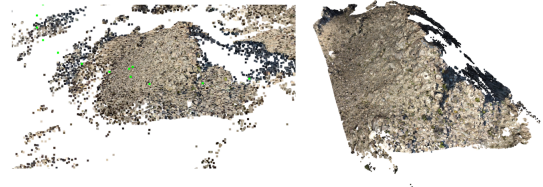


Figure 15: Comparison between the sparse reconstruction generated by OpenMVG (left) and the dense point cloud generated by OpenMVS (right).

D. Evaluation on Low-Texture Scenes

The feature-based reconstruction pipeline was initially developed and evaluated using the Palm Desert Micro dataset provided through the OpenMVG image dataset repository [16]. While this dataset was suitable for developing the reconstruction workflow, it did not sufficiently represent the challenging visual conditions that may be encountered during dental reconstruction, particularly in regions with limited distinctive visual features. Therefore, an additional evaluation was required to investigate how the current reconstruction approach behaves when the available image texture is insufficient to establish a large number of reliable correspondences.

To investigate this limitation, additional experiments were performed using the Middlebury Stereo 2006 datasets [17], [18]. The evaluation focused on four scenes with different visual characteristics: *Midd1*, *Midd2*, *Monopoly*, and *Plastic*. The four scenes were evaluated first using the implemented SIFT-based pairwise pipeline and subsequently using OpenMVG for Structure-from-Motion. This provided both a direct

evaluation of the implemented feature-based approach and an additional reference using an independent SfM implementation.

1) *Motivation for the Middlebury Dataset:* The Middlebury Stereo 2006 dataset was selected because it provides well-established stereo scenes with controlled imaging conditions and available ground-truth disparity data [17], [18]. The images are rectified, and the dataset provides stereo information for specific image views, including `view1.png` and `view5.png`. For the pairwise evaluation, these two views were used as an independent image pair for each scene. Although the Middlebury scenes originate from a multi-view stereo benchmark, the implemented pairwise experiment only receives the selected two images. Consequently, the reconstruction performed by the DentaVision pipeline remains a two-view reconstruction and does not use the additional views available in the dataset. The four selected scenes expose the reconstruction process to different levels of available local image structure. In particular, the *Plastic* scene contains substantially fewer distinctive features than the other evaluated scenes and therefore provides a useful stress case for feature-based reconstruction. This characteristic makes the Middlebury scenes, and particularly *Plastic*, a more relevant approximation of the low-texture conditions that may occur on dental surfaces than the initially used Palm Desert dataset.

The purpose of this experiment was not to evaluate dense stereo reconstruction at this stage. Instead, the initial objective was to establish a baseline for the existing feature-based reconstruction approach and determine whether a lack of detectable and matchable image features could become a limiting factor for sparse 3D reconstruction.

2) *SIFT Reconstruction on Middlebury Images:* The existing pairwise reconstruction pipeline was applied to `view1.png` and `view5.png` for each of the four scenes without changing its feature-based reconstruction architecture. For each image pair, the number of detected SIFT keypoints, the number of matches remaining after the Lowe ratio test, the number of RANSAC inliers, the resulting inlier ratio, and the number of reconstructed 3D points were recorded.

The inlier ratio was computed as

$$\text{Inlier Ratio} = \frac{\text{RANSAC Inliers}}{\text{Good Matches}} \times 100. \quad (13)$$

An important change compared with the initial Middlebury experiment concerns the camera calibration. The original project configuration had been defined for the Palm Desert Micro dataset. The Middlebury experiments were therefore rerun using the appropriate Middlebury camera parameters rather than the Palm

Desert calibration. This prevents the camera model from introducing an additional source of error into the relative pose and triangulation stages.

The resulting measurements are summarized in Table VII.

Table VII: Results of the SIFT-based pairwise reconstruction on the Middlebury scenes.

Dataset	View 1 KP	View 5 KP	Good Matches	Inliers	Inlier Ratio	3D Points
Midd1	621	610	344	311	90.41%	311
Midd2	575	572	333	306	91.89%	306
Monopoly	684	701	414	287	69.32%	287
Plastic	127	67	43	9	20.93%	9

The first three scenes produced several hundred usable correspondences. *Midd1* produced 311 RANSAC inliers from 344 good descriptor matches, corresponding to an inlier ratio of 90.41%. Similarly, *Midd2* produced 306 inliers from 333 good matches, resulting in an inlier ratio of 91.89%. The corresponding triangulation stages produced 311 and 306 reconstructed points, respectively. *Monopoly* produced the largest number of candidate descriptor matches, with 414 good matches. However, only 287 survived geometric verification, resulting in a lower inlier ratio of 69.32%. Nevertheless, the resulting reconstruction still contained 287 3D points. In contrast, *Plastic* showed a substantial reduction in feature availability. Only 127 keypoints were detected in `view1.png` and 67 in `view5.png`. This resulted in only 43 good descriptor matches, of which 9 remained after RANSAC. The resulting inlier ratio was therefore only 20.93%, and the triangulation stage produced only 9 3D points.

These results indicate that the degradation observed for *Plastic* occurs progressively throughout the feature-based reconstruction process. The scene first provides substantially fewer detectable local features, which limits the number of candidate descriptors and correspondences. The reduced number of correspondences is then accompanied by lower geometric consistency, ultimately leaving too few points for a substantial sparse reconstruction.

3) *OpenMVG and OpenMVS Evaluation:* To determine whether the observed limitations were specific to the implemented pairwise reconstruction pipeline, the same four Middlebury scenes were also processed using OpenMVG for Structure-from-Motion reconstruction. Unlike the implemented pairwise pipeline, OpenMVG performs multi-view Structure-from-Motion and therefore requires multiple images to estimate the camera motion and reconstruct the scene. Consequently, all seven available views of each Middlebury scene were used for the OpenMVG evaluation. Therefore, the comparison is not intended to determine which implementation performs better, but rather to assess whether a multi-view SfM approach can provide sufficient sparse structure under the

same challenging scene conditions. The SIFT-based experiment uses only `view1.png` and `view5.png`, whereas OpenMVG uses the complete seven-view sequence available for each scene. The two experiments are therefore evaluated separately, with the OpenMVG results serving as an additional assessment of the sparse reconstruction and its suitability as input for subsequent dense reconstruction. In particular, the number of OpenMVG tracks should not be interpreted as directly comparable to the number of pairwise SIFT inliers. Instead, the OpenMVG results were used to determine whether a sparse reconstruction could be established and whether sufficient sparse structure was available for subsequent dense reconstruction.

OpenMVG successfully produced a sparse reconstruction for all four evaluated scenes. However, the amount of reconstructed structure varied considerably between datasets. The recorded results are summarized in Table VIII.

Table VIII: OpenMVG sparse reconstruction results on the Middlebury scenes.

Dataset	Reconstructed Tracks / 3D Points	Calibrated Cameras
Midd1	129	7 / 7
Midd2	32	7 / 7
Monopoly	4	7 / 7
Plastic	29	3 / 7

The OpenMVG results demonstrate that successful camera calibration and sparse reconstruction do not necessarily imply that a sufficiently dense set of points is available for subsequent processing. For example, *Midd1* resulted in 129 reconstructed tracks with all seven input cameras calibrated, whereas *Monopoly* resulted in only four reconstructed tracks despite all seven cameras being calibrated. For *Plastic*, OpenMVG reconstructed 29 tracks and calibrated three of the seven input cameras. The corresponding residual statistics showed a mean reprojection residual of approximately 0.18 pixels and a median of approximately 0.10 pixels. These low residual values indicate that the surviving tracks were geometrically consistent, but they do not imply sufficient scene coverage. The sparse reconstructions were subsequently passed to OpenMVS for dense reconstruction. OpenMVS was not able to successfully complete the dense reconstruction for the evaluated scenes. The observed behavior was investigated through repeated tests to distinguish a possible implementation problem from a limitation of the input reconstruction. The experiments consistently showed that OpenMVG was able to produce a sparse point cloud before the OpenMVS stage. Therefore, the observed OpenMVS failure was not attributed to the absence of an OpenMVG reconstruction. Instead, the results are consistent with the hypothesis that the sparse reconstructions contained too few reliable points and tracks to provide sufficient geometric

support for successful densification.

This interpretation should be considered an experimental observation rather than a formally isolated causal result. The experiments did not independently vary the number of sparse points while keeping all other conditions constant. Nevertheless, the combination of sparse OpenMVG reconstructions and the subsequent failure of dense reconstruction provides evidence that insufficient sparse scene structure is a plausible limitation of the SfM-to-MVS approach for these scenes.

4) *Implications for Dental Reconstruction:* The Middlebury experiments demonstrate an important limitation of feature-based sparse reconstruction: its performance depends strongly on the availability of distinctive and repeatable visual features. These findings are relevant to dental reconstruction because dental surfaces cannot be assumed to provide uniformly distinctive local image features. Regions with limited local texture may result in fewer reliable feature correspondences, while specular reflections and other appearance variations can further complicate feature matching. A reconstruction approach that relies exclusively on sparse feature correspondences may therefore provide insufficient spatial coverage in some regions of the scene.

The experiments do not establish that SIFT or feature-based reconstruction is generally unsuitable for dental images. Instead, they identify a specific limitation of the current approach: reconstruction quality and coverage can degrade substantially when the available visual features are insufficient. Consequently, the Middlebury evaluation established the SIFT-based sparse reconstruction as a baseline and motivated the investigation of dense stereo correspondence as a complementary reconstruction strategy. The subsequent evaluation of StereoSGBM is therefore presented separately, with the *Plastic* scene providing a particularly useful test case for dense stereo because ground-truth disparity is available.

E. Stereo Reconstruction with SGBM

As an alternative to the feature-based reconstruction pipeline, DentaVision includes a dense stereo reconstruction approach based on OpenCV's Stereo Semi-Global Block Matching (StereoSGBM). This approach was evaluated using the Middlebury 2006 Plastic stereo dataset, which provides a calibrated and already rectified stereo pair together with a ground-truth disparity map. The stereo implementation is organized into separate components for disparity estimation, disparity validation, depth computation, and 3D point cloud reconstruction.

1) *StereoSGBM Implementation:* The disparity estimation stage is implemented by the `StereoMatcher` class in `src/pipeline/stereo/matcher.py`. It encapsulates OpenCV's

`cv2.StereoSGBM_create()` and computes a dense disparity map from a pair of rectified stereo images. The implementation accepts either grayscale or BGR images. BGR images are converted to grayscale before matching, while grayscale images are used directly. The two input images are required to have identical spatial dimensions.

The matcher exposes the main StereoSGBM parameters through its constructor. The default configuration uses a minimum disparity of 0 and searches 128 disparity levels. The matching block size is set to 5 pixels, with a uniqueness ratio of 10, a speckle window size of 100 pixels, a speckle range of 2, and a maximum left-right disparity difference of 1 pixel. The implementation uses the `STEREO_SGBM_MODE_SGBM_3WAY` mode. The smoothness penalties are derived from the block size as $P_1 = 8b^2$ and $P_2 = 32b^2$, where b denotes the selected block size. Input validation also ensures that the number of disparity levels is a positive multiple of 16 and that the block size is a positive odd number.

OpenCV internally stores StereoSGBM disparity values with a fixed scale factor of 16. Therefore, the implementation converts the output to floating-point pixel units by dividing the computed disparity map by 16. The resulting array has the same spatial resolution as the input images and is represented using `float32`.

For the Middlebury evaluation, the images used were the ThirdSize versions of `view1.png` and `view5.png`, with a resolution of 423×370 pixels. The dataset images are already rectified and have radial distortion removed, so no additional stereo rectification or distortion correction is performed by the reconstruction pipeline. The corresponding camera configuration is therefore derived directly from the parameters provided by the dataset. The original focal length of 3740 pixels is scaled according to the ThirdSize image resolution, while the stereo baseline is 160 mm. The principal point is set to the center of the 423×370 image. A corresponding 4×4 reprojection matrix Q is also defined for the Middlebury stereo geometry and supplied to the 3D reconstruction component.

2) *Disparity Validation and Depth Reconstruction:* After disparity estimation, the resulting map is passed to the `StereoValidator`. This component is intentionally separated from the reconstruction stages and currently performs two basic validity checks: disparity values must be finite and strictly greater than zero. The result is a boolean validity mask with the same dimensions as the disparity map. This separation allows additional validation strategies, such as left-right consistency checks, to be introduced later without changing the depth or point cloud reconstruction components.

The validated disparity can subsequently be used to estimate depth. This functionality is implemented by the `DepthReconstructor`, which applies the stereo depth relationship $Z = fB/d$ to all finite, positive disparity values. Invalid values are assigned a depth of zero. The focal length and baseline are provided to the component externally, and the resulting depth map uses the same physical unit as the supplied baseline.

For the Middlebury dataset, an additional disparity offset has to be considered during reconstruction. The ThirdSize images are cropped versions of the original images, and the dataset specifies a minimum disparity value of $d_{\min} = 280$ pixels. Consequently, the disparity estimated by StereoSGBM is offset before it is used for the Middlebury reconstruction:

$$d_{\text{reconstruction}} = d_{\text{SGBM}} + d_{\min}. \quad (14)$$

With the baseline expressed in millimeters, the resulting depth values are correspondingly expressed in millimeters.

The final 3D reconstruction is handled by the `StereoReconstructor`. Rather than explicitly computing the three-dimensional coordinates from individual image coordinates and depth values, this component uses OpenCV's `cv2.reprojectImageTo3D()` function. The function receives the reconstruction disparity and an externally supplied 4×4 reprojection matrix Q . Keeping Q outside the reconstruction class makes the component independent of a specific camera or dataset.

The complete disparity map is first reprojected into 3D space. The validation mask is then used to select only the pixels considered valid for reconstruction. Any resulting 3D coordinates that are not finite are discarded. The corresponding colors are obtained from the left stereo image, converted from BGR to RGB, and normalized to the range $[0, 1]$. The resulting points and colors are stored in the common `PointCloud` representation used by the rest of the reconstruction pipeline.

The resulting processing flow can therefore be summarized as:

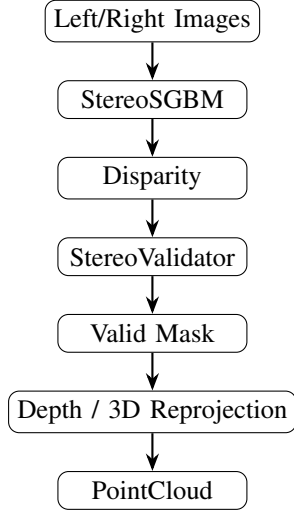


Figure 16: Dense stereo reconstruction pipeline using StereoSGBM.

3) *Quantitative and Qualitative Results*: The StereoSGBM implementation was evaluated quantitatively against the ground-truth disparity provided by the Middlebury Plastic dataset. Three matching block sizes, 3, 5, and 7 pixels, were tested while keeping the remaining StereoSGBM parameters unchanged. The results are summarized in Table IX.

Table IX: StereoSGBM disparity accuracy for different block sizes on the Middlebury Plastic dataset.

Block size	MAE (px)	Median AE (px)	> 1 px	> 2 px	> 5 px
3	2.81	0.60	37.78%	27.27%	18.64%
5	2.91	0.60	39.06%	29.23%	20.13%
7	2.66	0.60	38.30%	28.27%	18.91%

Among the tested configurations, a block size of 7 produced the lowest mean absolute error of 2.66 pixels and was therefore selected for the subsequent reconstruction. Although the improvement over the other configurations was small, it provided the best quantitative result, while the median absolute error remained unchanged at 0.60 pixels. Nevertheless, the disparity estimates still contained a considerable number of local errors: 38.30% of the valid comparison pixels had an absolute error greater than 1 pixel, and 18.91% exceeded 5 pixels.

For the subsequent reconstruction, the selected configuration produced a disparity map of 370×423 pixels. The StereoValidator classified 45.94% of the image pixels as valid, excluding disparities that were non-finite or not strictly positive. This value reflects the validity criterion of the implementation rather than the accuracy of the disparity estimates. After applying the Middlebury $d_{\min}$ offset, the reconstructed disparity values corresponded to depths between approximately 1.73 m and 2.13 m, with a mean depth of 1.84 m. The reprojection produced 71,904 valid 3D points, with no additional point loss

caused by non-finite coordinates. Thus, the resulting point cloud contained one 3D point for each valid disparity pixel in this evaluation.

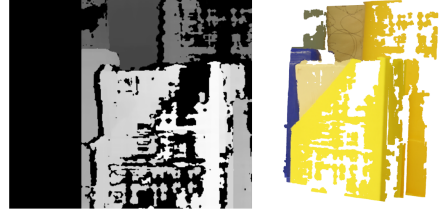


Figure 17: Dense stereo reconstruction of the Middlebury Plastic ThirdSize image pair using StereoSGBM. The left image shows the estimated disparity map, while the right image shows the corresponding colored 3D point cloud reconstructed from the disparity.

As shown in Figure 17, the estimated disparity provides a dense representation of the scene’s depth structure. The main regions of the scene are recovered consistently, while some areas contain invalid or missing disparity values. These regions indicate locations where a reliable disparity estimate was not obtained. The corresponding colored point cloud provides a three-dimensional representation of the reconstructed scene while retaining the color information. The recovered spatial structure is consistent with the disparity estimation, although the density and completeness of the point cloud depend directly on the reliability and spatial coverage of the underlying disparity map.

F. IGEV Stereo Reconstruction with OpenStereo

1) *Overview*: The project integrates the IGEV stereo matching model through the external OpenStereo framework to provide dense stereo disparity for 3D reconstruction. Unlike the custom StereoSGBM implementation, IGEV is a pretrained deep-learning stereo model. OpenStereo provides the IGEV implementation, configuration, preprocessing, and inference functionality, while DentaVision integrates its disparity output into the existing stereo reconstruction pipeline.

2) *Motivation for Using OpenStereo*: The project uses IGEV through the OpenStereo framework rather than integrating the original IGEV implementation directly. The main reason for this choice is to keep the stereo matching component flexible. OpenStereo provides a common framework for multiple stereo matching models, allowing the project to use IGEV while keeping the integration independent from a single model implementation. This makes it possible to evaluate or replace the stereo matching method in the future with other models supported by OpenStereo without requiring major changes to the downstream reconstruction pipeline. The IGEVMatcher isolates the framework-specific details, while the rest of the pipeline only receives a dense disparity map.

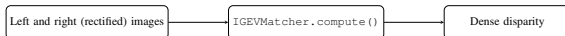
Therefore, OpenStereo is used not only as the implementation source for IGEV, but also as an abstraction layer that keeps the project open to additional deep-learning stereo models in the future.

3) *Preprocessing and Padding*: The evaluation transform configured by OpenStereo can pad the input images to dimensions compatible with the IGEV network. The matcher stores the original image dimensions before preprocessing and removes the added padding after inference. For the configured transform, padding is added to the top and right sides. Consequently, the original image corresponds to the upper-left region of the model output. The matcher removes the padded rows from the top and the padded columns from the right before returning the disparity. The final disparity therefore has exactly the same dimensions as the original input image. This is required so that the disparity remains spatially aligned with the left image during subsequent validation and 3D reconstruction.

4) *OpenStereo Integration*: OpenStereo is kept as an external dependency and is not included in the DentaVision source tree. Both OpenStereo and IGEV checkpoint are configured through the `.env` file. They have separate variables, which keeps the OpenStereo installation and the pretrained checkpoint independent from each other.

The integration is implemented in `src/pipeline/stereo/igev_matcher.py`. The `IGEVMatcher` acts as an adapter between the DentaVision stereo pipeline and OpenStereo. During initialization, it loads the OpenStereo configuration, assigns the configured checkpoint, builds the IGEV model through OpenStereo, and creates the evaluation transform defined by the selected configuration. For each stereo pair, `compute()` validates the input images and their dimensions, converts the OpenCV BGR images to RGB, and applies the OpenStereo evaluation transform. The resulting tensors are batched and moved to the selected device before IGEV inference is performed without gradient computation. The predicted `disp_pred` is then extracted, converted to a `float32` NumPy array, and any padding introduced by the OpenStereo transform is removed before returning the disparity at the original image resolution.

The matcher therefore exposes a simple interface to the rest of the project:



The rest of the reconstruction pipeline therefore does not need to interact directly with the OpenStereo trainer or the IGEV model implementation.

5) *Middlebury Evaluation*: The IGEV implementation was evaluated using the Plastic stereo pair from the Middlebury dataset. The images have a resolution

of 423×370 pixels. The Middlebury ThirdSize ground-truth disparity is stored with a scale factor of three. Therefore, the raw ground-truth values are divided by three before comparison with the IGEV prediction.

The evaluation produced the following results:

Table X: IGEV disparity accuracy on the Middlebury Plastic dataset.

Metric	Result
Disparity minimum	17.73 px
Disparity maximum	64.95 px
Disparity mean	45.10 px
Disparity median	44.11 px
Valid comparison pixels	156267
Mean absolute error	0.42 px
Median absolute error	0.27 px
Maximum absolute error	24.82 px
Mean signed error	-0.17 px
Median signed error	-0.12 px
Absolute error > 1 px	7.64%
Absolute error > 2 px	1.05%
Absolute error > 5 px	0.20%

The mean absolute error of 0.42 pixels and median absolute error of 0.27 pixels show a close agreement between the IGEV prediction and the Middlebury ground truth. Only 0.27% of the compared pixels have an absolute error greater than five pixels.

6) *Dense Reconstruction*: The predicted disparity was passed to the existing `StereoValidator`, `DepthReconstructor`, and `StereoReconstructor` components. The current validation accepted all 156510 pixels, resulting in a validity ratio of 100.00%. The resulting depth values ranged from 3070.72 mm to 11249.35 mm, with a mean of 5050.46 mm and a median of 4522.90 mm. The final 3D reconstruction contained 156510 points. The point coordinates have shape (156510, 3) and the corresponding point colors have shape (156510, 3).

Figure 18 provides a visual representation of the dense stereo result and the resulting 3D reconstruction. The disparity captures the main depth structure of the scene, which is reflected in the spatial organization of the reconstructed point cloud. Qualitatively, the reconstruction appears largely complete, with few visible gaps or inconsistencies relative to the scene represented in the input stereo pair. The resulting point cloud therefore provides a coherent representation of the observed scene structure.

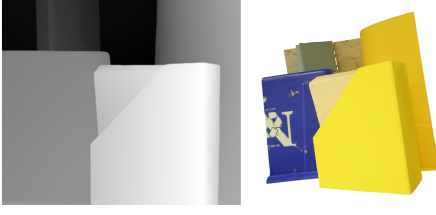


Figure 18: Dense stereo result and 3D reconstruction obtained with IGEV on the Middlebury Plastic stereo pair. The left image shows the estimated disparity map, while the right image shows the corresponding colored 3D point cloud reconstructed from the disparity.

The IGEV integration therefore provides a dense stereo backend that can be used with the existing validation, depth reconstruction, and 3D reconstruction components without requiring model-specific processing in those stages.

G. Comparative Evaluation

The evaluated reconstruction approaches differ in their input requirements and reconstruction principles. Consequently, a single quantitative metric cannot be applied fairly to all four methods. The pairwise SIFT and OpenMVG/OpenMVS approaches are instead considered primarily in terms of their behavior and limitations observed during the low-texture evaluation, while StereoSGBM and IGEV allow a direct quantitative comparison because both estimate dense disparity from the same calibrated stereo pair.

Table XI: Quantitative comparison of dense stereo reconstruction on the Middlebury Plastic dataset. For StereoSGBM, the results correspond to the best-performing block size of 7.

Metric	StereoSGBM	IGEV
Block size	7	—
Mean absolute error (px)	2.66	0.42
Median absolute error (px)	0.60	0.27
Error > 1 px	38.30%	7.64%
Error > 2 px	28.27%	1.05%
Error > 5 px	18.91%	0.20%

The quantitative evaluation shows a clear advantage of IGEV over StereoSGBM on the Middlebury Plastic scene. IGEV achieved a mean absolute disparity error of 0.42 px compared with 2.91 px for StereoSGBM, while the median error decreased from 0.60 px to 0.27 px. The IGEV results also show that only 0.27% of the evaluated pixels have an absolute disparity error greater than 5 px. Both methods produced disparity maps with comparable overall disparity statistics, indicating that the difference is primarily in the accuracy of the estimated disparity rather than in the recovered disparity range.

More broadly, the experiments show that feature-based approaches can provide useful sparse reconstructions when sufficient visual structure is available,

as demonstrated by the initial Palm Desert evaluation. However, the Middlebury experiments exposed substantial limitations under low-texture conditions. The use of OpenMVG’s multi-view SfM formulation does not fundamentally remove this limitation, since it still relies on reliable image correspondences to establish the scene geometry. In contrast, dense stereo methods directly estimate disparity from the calibrated stereo pair and therefore do not require a sparse feature-based scene representation. Between the two evaluated stereo approaches, IGEV achieved substantially lower disparity error than StereoSGBM on the Plastic scene and provided the most promising results for the targeted stereo reconstruction scenario.

H. Conclusion

This work investigated several approaches for reconstructing three-dimensional geometry from image data within the DentaVision project. Feature-based reconstruction using SIFT provided a functional sparse reconstruction pipeline and demonstrated that reliable camera motion and 3D points can be obtained when sufficient visual structure is available. However, the experiments on the low-texture Middlebury Plastic dataset revealed clear limitations of feature-based methods in regions with little distinctive texture. The use of OpenMVG and OpenMVS confirmed that a multi-view SfM pipeline does not fully overcome this limitation when the input images do not provide sufficient visual correspondences. Dense stereo methods proved more suitable for the low-texture scenario. StereoSGBM was able to produce a dense disparity map, but its accuracy remained limited, particularly in regions where local image structure was insufficient. In contrast, the IGEV-based stereo approach achieved substantially lower disparity errors on the same calibrated Middlebury stereo pair. The Plastic stereo pair is particularly relevant to the intended DentaVision application because its predominantly low-texture surfaces provide a scenario that is more representative of the visual characteristics expected from dental surfaces than the highly textured Palm Desert dataset. Based on its strong quantitative performance in this evaluation, IGEV will be used as the basis for the stereo matching component of the DentaVision reconstruction pipeline. Its ability to produce accurate dense disparity in this challenging low-texture scenario makes it a particularly suitable approach for the intended application.

Although an integration with the other components of the DentaVision system could not be completed, the reconstruction stage was designed to provide a clean interface for the subsequent 3D registration stage. The rectified stereo image pair generated by the simulator, together with the relevant camera information, can be used by the selected stereo reconstruction method to generate a 3D point cloud in .ply format. This

output is intended to serve as the input to the subsequent registration stage, allowing the reconstruction component to be integrated into the overall pipeline without requiring changes to its internal processing.

V. 3-D REGISTRATION AND MODEL CONSTRUCTION

Authors: Marvin Kraus

A. Overview and Scope

This part of the project covers two consecutive processing stages: rigid registration of partially overlapping dental point clouds and surface reconstruction of the resulting merged point cloud. Image acquisition and point-cloud generation are handled by the preceding reconstruction stage; the present contribution starts with multiple point-cloud fragments in different local coordinate systems and ends with an explicit triangular surface model.

The development was deliberately organized as an experimental selection process rather than around a single preselected algorithm. Five pairwise registration methods were first screened on controlled synthetic dental-like data. The most promising approaches were then re-evaluated on five distinct dental model sets comprising ten upper- and lower-jaw STL meshes. Each mesh was sampled into a point cloud, split into overlapping fragments, and displaced by known rigid motions. This broader validation stage changed the final registration decision from Fast Global Registration (FGR) to RANSAC-based global registration. Surface reconstruction was evaluated in the same way: Poisson Surface Reconstruction and the Ball Pivoting Algorithm (BPA) were first compared on the synthetic dataset and then re-evaluated across all ten realistic jaw meshes.

B. Registration Methods

A rigid transformation maps a source point $\mathbf{p}$ to a target point $\mathbf{p}'$ by

$$\mathbf{p}' = R\mathbf{p} + \mathbf{t}, \quad (15)$$

where $R \in SO(3)$ is the rotation matrix and $\mathbf{t} \in \mathbb{R}^3$ is the translation vector. The implementation consistently uses the convention source $\rightarrow$ target. This convention is especially important when pairwise transformations are chained during multiway registration.

Five registration variants were implemented. Point-to-Point ICP and Point-to-Plane ICP follow the classical local ICP formulation [19], [20]. Generalized ICP (GICP) extends local matching by incorporating local covariance information [21]. The two global-to-local pipelines first compute Fast Point Feature Histogram (FPFH) descriptors [22], perform either RANSAC-based feature matching or FGR [23], and then refine the resulting transformation with Point-to-Plane ICP [24].

Method	Main advantage	Main limitation
Point-to-Point ICP	Simple and fast	Small capture range
Point-to-Plane ICP	Accurate local surface alignment	Requires normals and a good initial pose
Generalized ICP	Strong local surface adaptation	Still initialization dependent
FPFH + RANSAC + ICP	Large capture range and robust global hypotheses	Stochastic and slower
FPFH + FGR + ICP	Fast global initialization with ICP refinement	Sensitive to ambiguous feature correspondences

Table XII: Implemented registration methods.

Component	Responsibility
Preprocessing	Voxel downsampling, normal estimation, FPFH descriptors
Registration algorithms	P2P ICP, P2Plane ICP, GICP, FPFH+RANSAC+ICP, FPFH+FGR+ICP
Evaluation	Fitness, inlier RMSE, translation/rotation ground-truth error
Multiway registration	Pairwise chaining, pose evaluation, merged cloud export
Mesh reconstruction	Poisson and Ball Pivoting
Mesh analysis	Components, manifold tests, self-intersections, bidirectional distances
Mesh export	Simplification, cleanup, PLY/STL serialization and reload test

Table XIII: Functional structure of the implemented registration and model-construction pipeline.

All algorithms share the same preprocessing whenever possible. Point clouds are voxel-downsampled with a voxel size of 0.5, normals are estimated within a radius of 2.0 voxel sizes using at most 30 neighbors, and FPFH descriptors use a radius of 5.0 voxel sizes and at most 100 neighbors. RANSAC uses three correspondences per hypothesis, up to 100,000 iterations, and a confidence of 0.999. The final local refinement uses Point-to-Plane ICP with a correspondence distance of 0.4 voxel sizes. Open3D 0.19.0 provides the geometric processing and registration primitives [25].

C. Implementation Structure and Evaluation Metrics

The implementation uses a small modular Python/Open3D architecture so that preprocessing, algorithms, evaluation, visualization, and mesh construction can be exchanged independently. The registration layer contains the five pairwise methods behind a common result interface, while the reconstruction layer contains Poisson/BPA reconstruction, cleanup, topology analysis, simplification, and export. This separation was important for the experimental workflow because the same preprocessing and evaluation code could be reused for synthetic and realistic datasets.

a) Software architecture.: The code base separates the importable implementation under

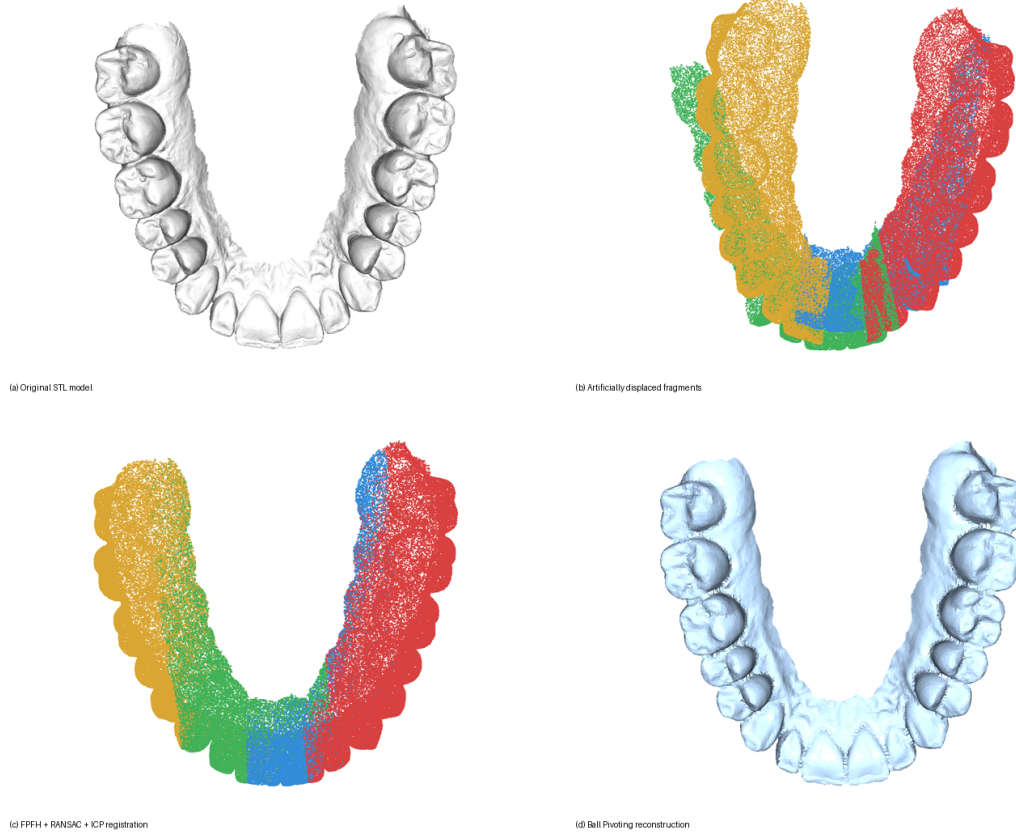


Figure 19: Representative realistic-validation workflow using Model 1 upper jaw. The original STL surface is sampled, split into overlapping fragments, displaced by known rigid transformations, registered with FPFH + RANSAC + Point-to-Plane ICP, and reconstructed with Ball Pivoting. Quantitative evaluation was performed on ten jaw meshes.

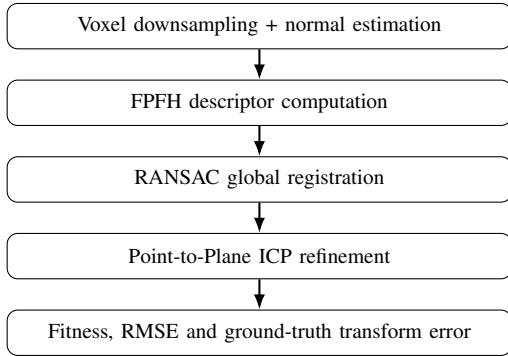


Figure 20: Final pairwise registration pipeline after realistic validation.

`src/registration3d` from experiment drivers, diagnostic scripts, and report-visualization utilities. Within the core package, registration algorithms follow a strategy-style interface: Point-to-Point ICP, Point-to-Plane ICP, GICP, FPFH+RANSAC+ICP, and FPFH+FGR+ICP expose the same `register(...)` operation and return a standardized result object. This

allows the same evaluator and benchmark logic to be reused without algorithm-specific branches. The two feature-based methods additionally share the same FPFH extractor. Multiway registration accepts an interchangeable pairwise registration strategy and produces a merged point cloud, which is deliberately treated as the boundary between registration and surface reconstruction. The mesh stage is similarly separated into reconstruction, cleanup, topology/distance evaluation, simplification, and export.

When ground truth is available, translation error is computed as

$$e_t = \|\mathbf{t}_{est} - \mathbf{t}_{GT}\|_2, \quad (16)$$

and rotation error as

$$e_R = \cos^{-1} \left(\frac{\text{tr}(R_{est} R_{GT}^T) - 1}{2} \right). \quad (17)$$

The realistic robustness study classifies a result as successful when $e_t \leq 1$ model unit and $e_R \leq 1^\circ$. This explicit transform-based criterion avoids the ambiguity of relying only on fitness or inlier RMSE.

D. Synthetic Registration Evaluation

1) *Evaluation Setup:* Synthetic dental-like source and target clouds were generated with known rigid ground truth. Four disturbance variables were varied independently: overlap, rotation, translation, and additive positional noise. Each condition was evaluated in 10 reproducible repetitions. Within a given stress-test series, the same repetition index uses the same dataset seed across all values of the varied parameter, while stochastic registration methods receive reproducible algorithm seeds. This design reduces unintended variation between neighboring conditions and allows direct evaluation of the estimated transformation rather than relying only on correspondence-based metrics.

Experiment	Tested values
Overlap	0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80
Rotation	0°, 5°, 10°, 20°, 30°, 40°, 60°
Translation	0, 1, 2, 4, 6, 8, 12
Noise standard deviation	0, 0.01, 0.025, 0.05, 0.10, 0.20

Table XIV: Controlled synthetic registration experiments. Each parameter value was evaluated with $n = 10$ reproducible repetitions.

The principal metrics were fitness, inlier RMSE, translation error, rotation error, runtime, and success rate. Fitness and inlier RMSE alone were not treated as correctness guarantees: repetitive dental geometry can produce geometrically plausible but semantically incorrect correspondences. Whenever ground truth was available, translation and rotation errors were therefore decisive.

a) *Interpretation of overlap and rotation.:* The updated 10-repeat overlap study removes an apparent non-monotonicity that was present in an earlier exploratory plot. For FPFH + RANSAC + ICP, the observed success rates increase from 0% at overlaps 0.20–0.40 to 40% at 0.50, 90% at 0.60 and 0.70, and 100% at 0.80. FGR shows an even sharper transition, remaining at 0% through 0.50 and reaching 100% from 0.60 onward. This supports the interpretation that, on the synthetic dental-like geometry, the feature-based methods require a sufficiently large common visible region before FPFH correspondences become reliably distinctive.

The rotation series exhibits a notable non-monotonic behavior for Point-to-Plane ICP. An input rotation of 0° does not correspond to a perfectly aligned pair: the series keeps overlap at 0.60, translation magnitude at 3 model units, and Gaussian noise at $\sigma = 0.02$. Under this translation-only displacement, Point-to-Plane ICP converged consistently to an incorrect local solution, with mean translation error 3.5399 model units and mean rotation error 7.336°. For rotations from 5° through 40°, the altered correspondence configuration instead led to a favorable

convergence basin, producing 100% success and mean translation errors of approximately 0.017–0.021 model units. At 60°, one of ten runs failed, reducing success to 90%. This result is interpreted as a dataset-specific local-minimum effect: it does not imply that adding rotation generally improves ICP, but rather that the correspondence landscape of the repetitive synthetic geometry changes with the applied pose.

GICP shows a similarly non-monotonic rotation response, although for a slightly different reason. It is still a local iterative method and therefore remains sensitive to the nearest-neighbor correspondences induced by the initial pose. In addition, its residual is weighted by local covariance information, so a change in relative orientation alters both the correspondences and the covariance-weighted objective. On the repetitive synthetic dental-like geometry, different input rotations can therefore move the optimizer between substantially different convergence basins. The observed fluctuations are interpreted as correspondence and local-minimum effects of this particular dataset, not as a monotonic relationship between larger rotations and better or worse GICP performance.

2) Synthetic Findings and Provisional Selection:

The synthetic experiments highlighted the expected trade-off. Local ICP variants were fast and highly accurate whenever the initial alignment remained inside their capture range, but they failed for larger displacements. Feature-based global registration handled larger initial pose differences but was more sensitive to low overlap, repetitive geometry, and noise.

Under the original synthetic benchmark, FPFH + FGR + Point-to-Plane ICP provided the most attractive overall compromise between global capture range, precision after local refinement, and runtime. It was therefore selected *provisionally* for the first multiway baseline. This was not treated as a universal ranking; the later realistic validation was explicitly intended to challenge this choice.

A four-fragment synthetic multiway test confirmed that FGR + ICP could produce a precise chain under the controlled test conditions. The maximum node translation error was approximately 0.0047 and the maximum rotation error approximately 0.035°, while the merged cloud contained 22,022 points. Additional multi-start and cycle-consistency experiments showed, however, that high fitness can still occur for incorrect dental alignments. This observation motivated the second validation stage on realistic jaw geometry.

3) *Synthetic Multiway Baseline and Robustness Analysis:* The synthetic multiway dataset contains four partially overlapping fragments with 13,793, 15,302, 16,193, and 15,069 points. The controlled acquisition model again follows $\mathbf{x}_i = A_i \mathbf{x}_{world}$, so the exact pose of fragment i in the global reference frame is A_i^{-1} . For the provisional FGR baseline, only sequential odometry edges were used. Loop closures

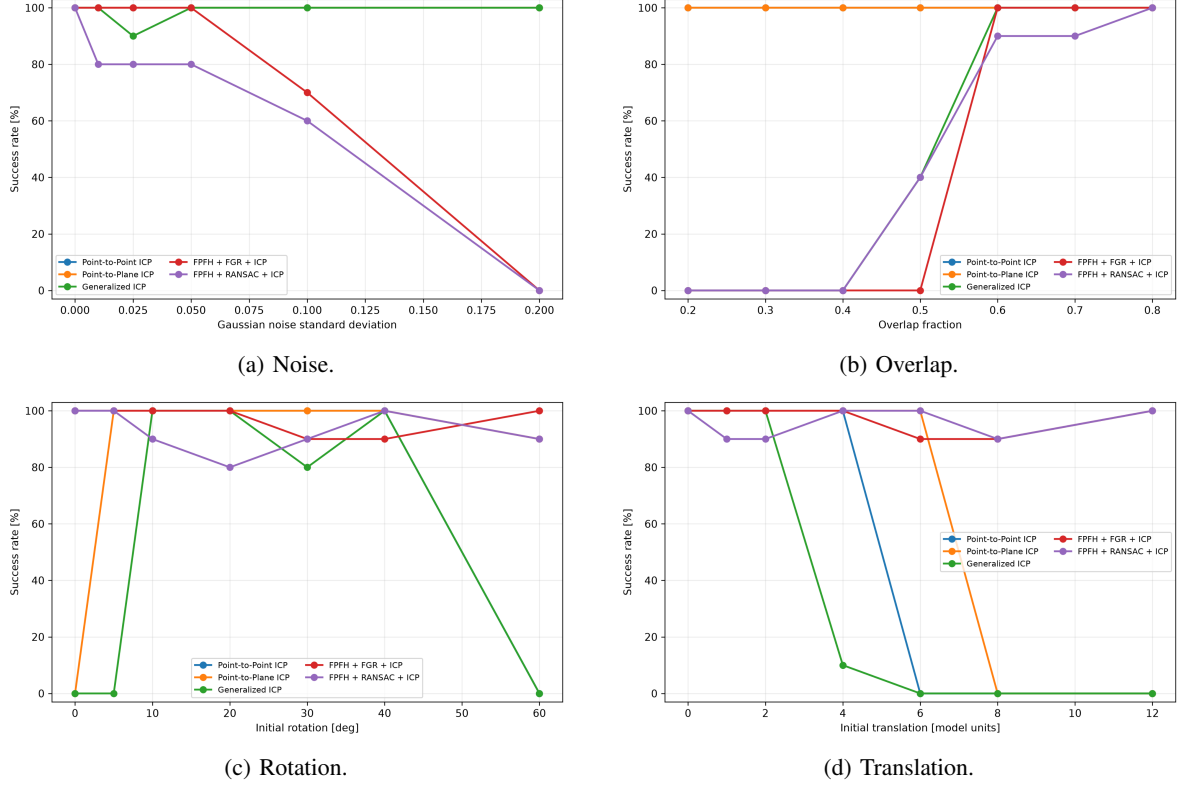


Figure 21: Success-rate curves from the controlled synthetic pairwise experiments ($n = 10$ repetitions per parameter value). The curves should be interpreted as empirical behavior on the specific dental-like geometry rather than as universal monotonic capture-range characteristics.

Stress case	P2P ICP	P2Plane ICP	GICP	RANSAC + ICP	FGR + ICP
Favorable initialization	Very strong	Very strong	Very strong	Strong	Strong
Large translation	Capture-range failures	Capture-range failures	Capture-range failures	Robust global recovery	Robust global recovery
Low overlap	Can converge locally if initialized well	Can converge locally if initialized well	Can remain locally accurate	Ambiguous feature matches	Ambiguous feature matches
High rotation	Fails at the largest tested angle	Non-monotonic local-minimum behavior	Strongly non-monotonic	Large capture range	Large capture range
Strong synthetic noise	Stable only with favorable initialization	Stable only with favorable initialization	Stable only with favorable initialization	Global feature matching degraded	Global feature matching degraded

Table XV: Qualitative behavior observed in the controlled synthetic stress tests. The table summarizes failure modes rather than claiming a universal algorithm ranking.

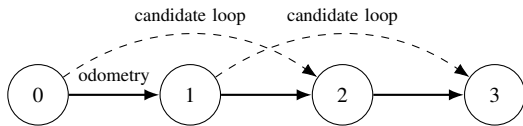


Figure 22: Synthetic four-fragment pose-graph concept. Solid edges form the evaluated sequential baseline; dashed loop candidates were examined only in the optional robustness analysis.

were deliberately disabled to measure the selected pairwise method without global correction.

Odometry edge	Fitness	Inlier RMSE
0 → 1	0.7397	0.2499
1 → 2	0.6904	0.2493
2 → 3	0.6676	0.2466

Table XVI: Synthetic four-fragment FGR + ICP multiway baseline.

The resulting pose errors were 0 for fragment 0, 0.00424 / 0.00636° for fragment 1, 0.00468 / 0.03513° for fragment 2, and 0.00268 / 0.02626° for fragment 3. The merged point cloud contained 22,022 points. Thus, under controlled synthetic conditions,

sequential accumulation remained small.

A separate robustness analysis generated 20 FGR and 20 RANSAC hypotheses with deterministic seeds, clustered similar transformations, and ranked clusters by median RMSE. Candidate loop closures were also checked for cycle consistency. This procedure could recover correct odometry in ambiguous synthetic cases, but it was intentionally kept separate from the baseline algorithm comparison: cycle consistency can reject some inconsistent hypotheses, yet it cannot prove that a globally self-consistent chain is geometrically correct. The realistic validation therefore remained necessary.

E. Validation on Realistic Dental Geometry

1) *Five-Model Dataset Construction*: The realistic validation was expanded to five distinct dental model sets, each containing an upper- and lower-jaw STL mesh. The ten meshes have bounding-box diagonals between 79.696 and 92.084 model units; the ratio between the largest and smallest diagonal is only 1.155, so the same registration parameters can be used without rescaling. Every jaw surface was uniformly sampled to 120,000 points and split into four partially overlapping fragments along the first PCA axis with a nominal overlap of 60%.

Known rigid acquisition transformations were applied using

$$\mathbf{x}_i = A_i \mathbf{x}_{\text{world}}, \quad (18)$$

so that the exact pairwise ground-truth transformation is

$$T_{i \rightarrow j}^{GT} = A_j A_i^{-1}. \quad (19)$$

The STL format itself does not encode physical units. All distances are therefore reported in model units.

Property	Five-model validation suite
Dental model sets	5
Jaw meshes	10 (5 upper, 5 lower)
Surface samples per jaw	120,000
Fragments per jaw	4
Adjacent pairwise cases	30
Nominal fragment overlap	60%
Bounding-box diagonal range	79.696–92.084 model units

Table XVII: Realistic five-model validation dataset.

The experimental protocol deliberately separates geometric diversity from stochastic repetition. The zero-noise benchmark contains 30 distinct adjacent fragment pairs, while the nonzero-noise conditions repeat these same base pairs with three independently seeded perturbations. This distinction prevents repeated perturbations from being presented as additional independent dental geometries. The same fragmented jaws are then reused for the RANSAC multiway experiment and for surface reconstruction, so registration and mesh quality can be evaluated along one consistent end-to-end path.

2) Pairwise Comparison Across 30 Fragment Pairs:

The same five registration algorithms and preprocessing parameters were applied to the adjacent fragment pairs $0 \rightarrow 1$, $1 \rightarrow 2$, and $2 \rightarrow 3$ for every jaw. This yields 30 geometrically distinct zero-noise pairwise cases. Success requires translation error ≤ 1 model unit and rotation error $\leq 1^\circ$.

The five-model experiment makes the distinction between precision and robustness particularly clear. FGR + ICP and GICP have very small median errors when they converge correctly, yet their much larger mean errors reveal severe outliers. RANSAC + ICP combines similarly low successful-case error with no failures in the 30 evaluated adjacent-fragment cases. The 100% observed rate corresponds to 30/30 cases and is reported as a benchmark result rather than as evidence of universal reliability.

Runtime therefore represents a real but acceptable trade-off rather than a hidden cost. Point-to-Plane ICP and GICP are fastest at approximately 0.13–0.14 s per evaluated pair, whereas RANSAC + ICP requires about 0.68 s and FGR + ICP about 0.89 s on the benchmark system. The selected RANSAC pipeline is roughly five times slower than the fastest local methods, but those local methods fail on 20–27% of the zero-noise cases. Since a failed global alignment propagates directly into the subsequent multiway reconstruction, robustness was prioritized over sub-second differences in pairwise runtime.

3) *Noise Robustness Across Five Models*: Additive isotropic Gaussian positional noise was applied with standard deviations 0, 0.01, 0.025, 0.05, 0.10, and 0.20. At $\sigma = 0$, each of the 30 distinct pairwise cases was evaluated once. At every nonzero level, three independent noise realizations were generated for each case, yielding 90 trials per algorithm and noise level. Within a given realization, all algorithms receive exactly the same noisy point clouds.

σ	P2P	P2Plane	GICP	FGR	RANSAC
0.000	50.0	80.0	73.3	73.3	100.0
0.010	51.1	76.7	72.2	88.9	100.0
0.025	53.3	78.9	76.7	83.3	100.0
0.050	52.2	76.7	71.1	77.8	100.0
0.100	41.1	75.6	74.4	70.0	100.0
0.200	7.8	71.1	77.8	33.3	84.4

Table XIX: Success rate [%] in the five-model noise study. The zero-noise row contains 30 unique cases; every nonzero row contains 90 noisy trials per algorithm.

RANSAC + ICP remained at 100% observed success through $\sigma = 0.10$. At $\sigma = 0.20$, 76 of 90 trials succeeded, corresponding to 84.44% and a Wilson interval of approximately 75.57–90.50%. At the same level FGR dropped to 33.3%, while GICP and Point-to-Plane ICP reached 77.8% and 71.1%, respectively.

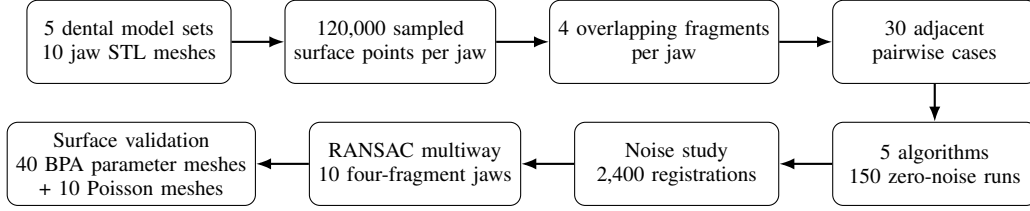


Figure 23: Five-model evaluation protocol. The same ten jaw geometries are propagated from controlled virtual fragmentation through pairwise comparison, noise robustness, sequential multiway registration, and surface-reconstruction validation.

Method	Success	Mean e_t	Median e_t	Mean e_R [°]	Median e_R [°]	Runtime [s]
Point-to-Point ICP	50.0%	2.395165	0.226672	8.768677	0.899657	0.283623
Point-to-Plane ICP	80.0%	1.842892	0.013040	6.905271	0.070498	0.131327
Generalized ICP	73.3%	1.632389	0.001382	8.141821	0.008669	0.141021
FPFH + FGR + ICP	73.3%	3.977360	0.001573	10.316597	0.010267	0.891728
FPFH + RANSAC + ICP	100.0%	0.001921	0.001509	0.009483	0.009741	0.681506

Table XVIII: Pairwise registration summary across 30 adjacent fragment pairs from ten jaw meshes. Large mean-to-median gaps for several competing methods are caused by catastrophic registration failures.

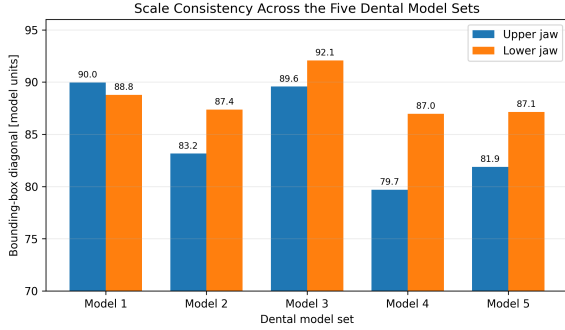


Figure 24: Bounding-box diagonal for all ten realistic jaw meshes. The relatively narrow range (79.696–92.084 model units) supports using one common registration parameterization, while also showing that the validation does not intentionally span extreme anatomical scale differences.

The RANSAC mean translation and rotation errors at $\sigma = 0.10$ were only 0.02285 model units and 0.0898° . At $\sigma = 0.20$, the mean errors rose to 0.4002 model units and 2.749° , whereas the corresponding medians remained much smaller at 0.1634 and 0.522° . This mean-to-median separation indicates that the strongest-noise condition is dominated by a subset of catastrophic failures rather than by a uniform loss of precision.

A second non-monotonic effect appears for FGR at very low noise: its observed success rate rises from 73.3% at $\sigma = 0$ to 88.9% at $\sigma = 0.01$ before decreasing again as the perturbation grows. This should not be interpreted as evidence that additive noise generally improves FGR. A plausible dataset-specific explanation is that small perturbations slightly change local neighborhoods and therefore FPFH

descriptors, which can disrupt ambiguous feature correspondences produced by repetitive dental geometry and occasionally move the global optimization toward a more favorable hypothesis. At larger noise levels, descriptor degradation dominates and performance declines. Because the 90 nonzero-noise trials are repeated perturbations of the same 30 base pairs, this pattern is treated as a descriptive observation rather than as a statistically independent demonstration of a beneficial-noise effect.

The expanded realistic validation therefore confirms the final pairwise choice:

FPFH $\rightarrow$ RANSAC $\rightarrow$ Point-to-Plane ICP.

F. Multiway Registration

The selected RANSAC + ICP pipeline was applied sequentially to the four fragments of every jaw. Pairwise transformations were chained into the reference coordinate system of fragment 0 without loop-closure correction, so the experiment directly measures accumulated odometry error. For pairwise transformation $T_{i \rightarrow i+1}$ and global pose P_i ,

$$P_{i+1} = P_i T_{i \rightarrow i+1}^{-1}. \quad (20)$$

Because all acquisition transformations are known, each chained pose can be compared directly with ground truth.

Metric	Mean	Median	Maximum
Translation error [model units]	0.003175	0.003059	0.008239
Rotation error [°]	0.013212	0.013768	0.022660

Table XX: Global pose error over 30 non-reference poses from ten sequential four-fragment registrations.

The pairwise middle edge $1 \rightarrow 2$ remains the most difficult odometry step across the model suite: its

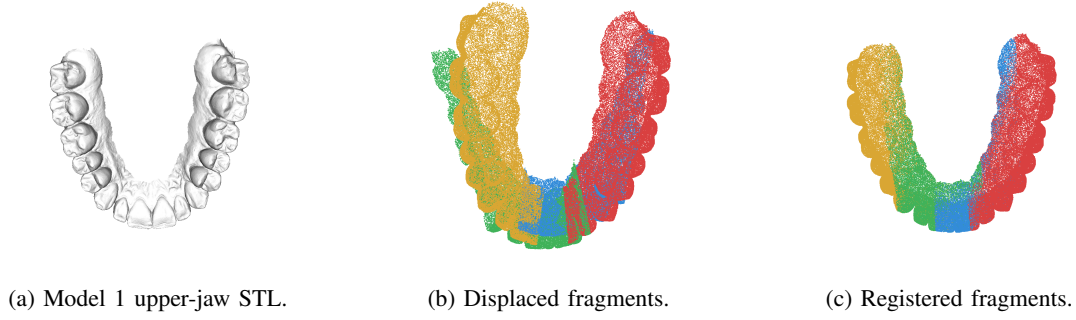


Figure 25: Representative qualitative example from Model 1. The quantitative registration experiments use all five model sets and all ten jaw meshes.

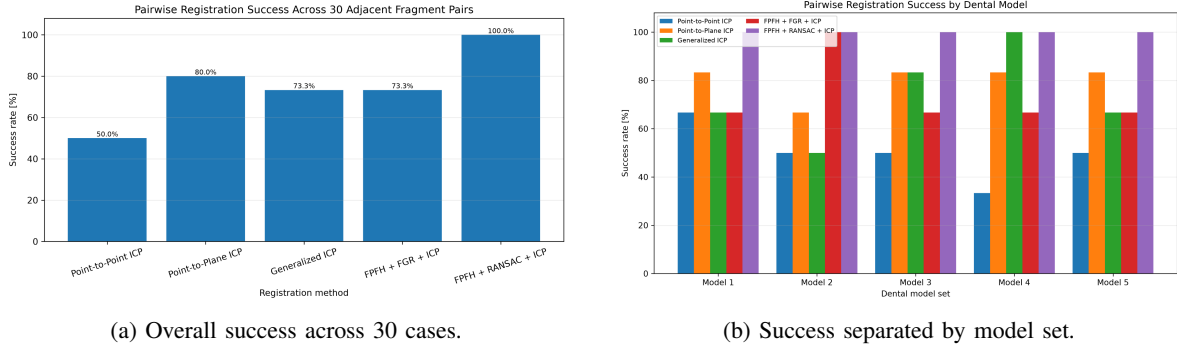


Figure 26: Five-model zero-noise pairwise validation. RANSAC + ICP achieved 30/30 successful registrations and 100% success in every individual model set.

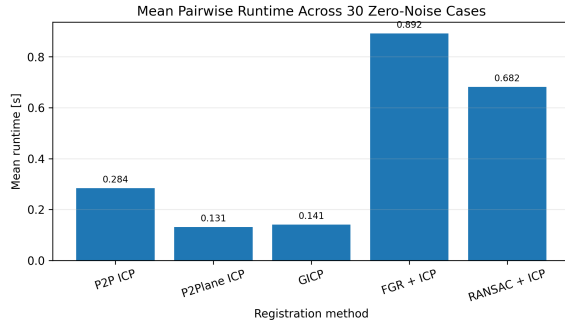


Figure 27: Mean runtime per zero-noise adjacent-pair registration across the 30 realistic cases. The selected RANSAC pipeline is slower than the local ICP variants but remains below FGR in this benchmark while providing substantially higher robustness.

mean pairwise rotation error is 0.01310° , compared with 0.00766° for $0 \rightarrow 1$ and 0.00758° for $2 \rightarrow 3$. Even after sequential accumulation, however, the worst observed global pose error remains only 0.008239 model units in translation and 0.02266° in rotation. The ten merged point clouds form the inputs to the surface-reconstruction validation.

G. Surface Reconstruction

A point cloud contains discrete surface samples but no explicit connectivity [26]. The model-construction stage therefore estimates a triangular surface from the registered points. Two methods were compared: Poisson Surface Reconstruction [27] and the Ball Pivoting Algorithm [28].

Poisson reconstruction solves for an implicit surface and tends to produce smooth, globally connected geometry. This is advantageous for completeness, but missing measurements may be bridged by surfaces that are not actually supported by the input points. Ball Pivoting constructs triangles locally by rolling virtual balls with specified radii over the oriented point cloud. It is therefore more conservative but may leave open boundaries when the selected radii are too small.

Normals are estimated using a radius equal to three times the mean nearest-neighbor distance and at most 50 neighbors. Poisson is evaluated at depth 9 with the lowest 2% density vertices removed. BPA uses multiple radii relative to the mean point spacing.

For mesh evaluation, 100,000 points are sampled uniformly from the reconstructed surface. Distances are measured in both directions: input/reference $\rightarrow$ reconstructed mesh and reconstructed mesh $\rightarrow$ input/reference. The second direction is important because it reveals unsupported surface patches. The

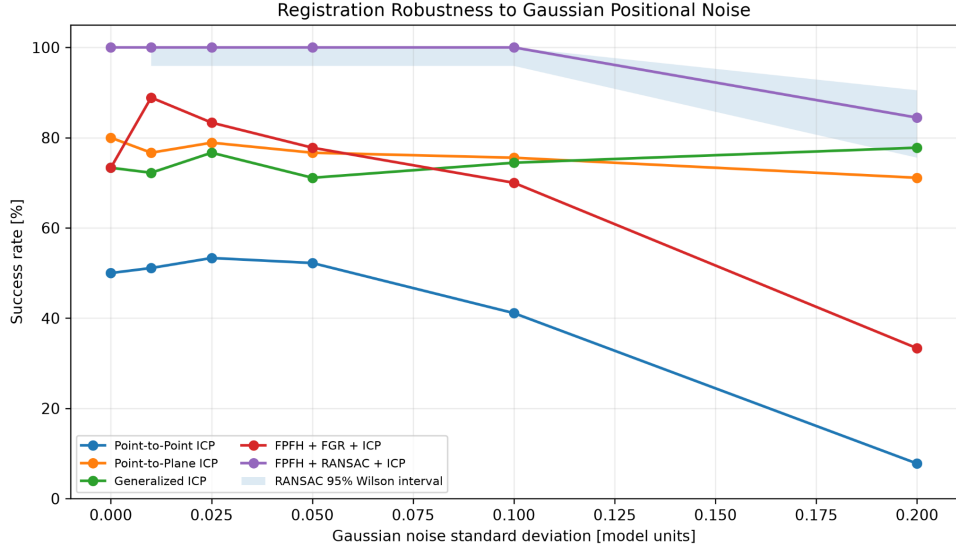
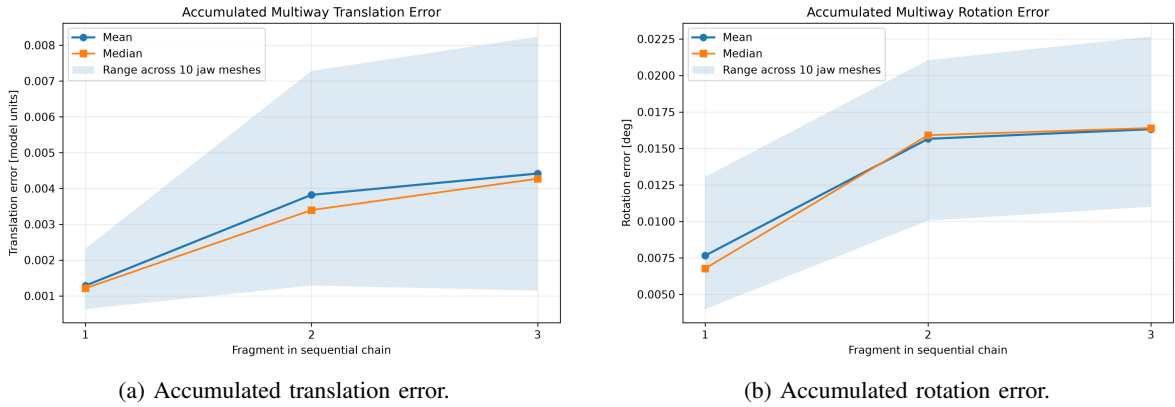


Figure 28: Success rate under additive Gaussian positional noise across five dental model sets. The shaded band shows the 95% Wilson interval for RANSAC + ICP at nonzero noise levels. Because repeated noise realizations share the same 30 underlying fragment pairs, the intervals are used as descriptive finite-sample uncertainty indicators rather than formal population-level confidence statements.



(a) Accumulated translation error.

(b) Accumulated rotation error.

Figure 29: Sequential multiway error across all ten jaw meshes. Lines show mean and median; shaded regions show the full range across the ten jaws.

symmetric RMSE combines both directions:

$$\text{RMSE}_{\text{sym}} = \sqrt{\frac{\text{RMSE}_{A \rightarrow B}^2 + \text{RMSE}_{B \rightarrow A}^2}{2}}. \quad (21)$$

This is a sampling-based surface approximation rather than an exact point-to-triangle Hausdorff distance.

H. Synthetic Mesh Evaluation

1) *Poisson versus Ball Pivoting*: On the synthetic merged cloud containing 22,022 points, Poisson produced a dense surface but also a very large mesh-to-cloud error and self-intersections. The cleaned BPA reconstruction was much closer to the observed geometry.

Metric	Poisson	BPA
Triangles	151,430	40,020
Connected components	17	1
Self-intersections	Yes	No
Cloud $\rightarrow$ mesh RMSE	0.122208	0.076404
Mesh $\rightarrow$ cloud RMSE	2.741374	0.115709
Symmetric RMSE	1.940369	0.098046

Table XXI: Synthetic surface-reconstruction comparison after BPA cleanup.

2) *BPA Parameter Study and Cleanup*: Four BPA radius-factor sets were compared. The smallest set (1.0, 1.5, 2.0) achieved the lowest symmetric RMSE on the synthetic geometry. Although it initially produced 82 connected components, 99.04% of all triangles belonged to one dominant component; the

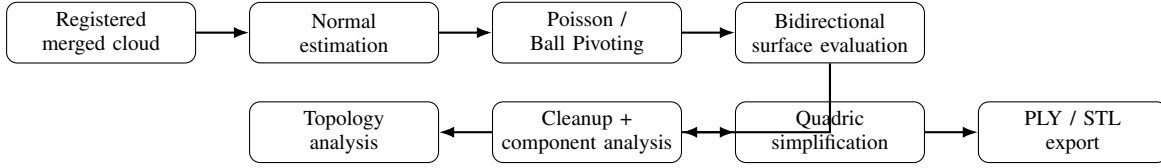


Figure 30: Surface-reconstruction and model-processing workflow. Algorithm selection is based on geometry and topology rather than visual smoothness alone.

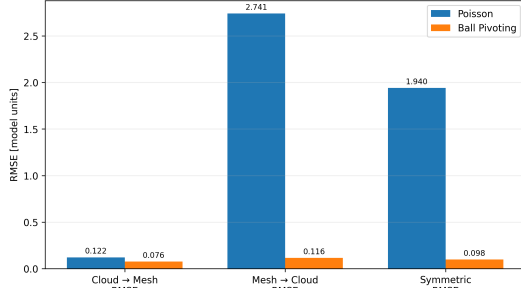


Figure 31: Synthetic geometric error comparison between Poisson and Ball Pivoting.

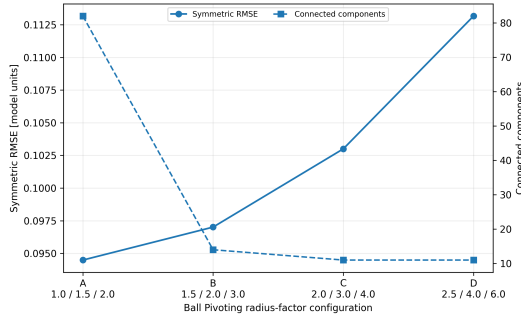


Figure 32: Synthetic BPA radius-factor study.

remaining components were tiny satellites. Removing components below 250 triangles yielded one connected component with 40,020 triangles.

A topology inspection identified no genuine non-manifold edges and no self-intersections, but the surface remained non-watertight, non-vertex-manifold, and globally non-orientable. An attempted vertex-splitting repair removed the non-manifold-vertex condition but introduced self-intersections. The repair was therefore rejected instead of forcing a topologically cleaner but geometrically worse result.

3) *Mesh Simplification*: Quadric decimation [29] was tested at target sizes of 30,000, 20,000, 10,000, and 5,000 triangles. The 30k variant provided the best compromise: it reduced the triangle count by approximately 25% while increasing the symmetric RMSE by only 0.171%.

After removing one isolated triangle, the final synthetic export contained 16,649 vertices and 29,998 triangles. The symmetric RMSE changed from 0.098070

Topology property	Full-resolution BPA candidate
Vertices / triangles	21,689 / 40,020
Boundary edges	7,982
Non-manifold edges	0
Non-manifold vertices	1,204 (5.55%)
Self-intersections	0
Vertex manifold	No
Watertight	No
Orientable	No
Orientation-conflict edges	420
Affected triangles	784 (1.96%)

Table XXII: Detailed topology diagnosis of the cleaned synthetic BPA candidate.

Property	Original	After vertex splitting
Vertices	21,689	22,961
Triangles	40,020	40,020
Non-manifold vertices	1,204	0
Vertex manifold	No	Yes
Self-intersections	No	Yes
Orientable	No	No
Symmetric RMSE	0.098070	0.098070

Table XXIII: Rejected topology-repair attempt. Removing the non-manifold-vertex condition introduced self-intersections and was therefore not accepted.

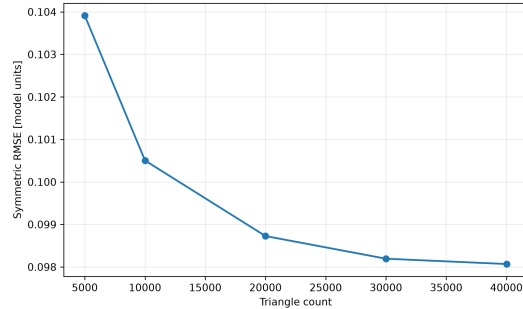


Figure 33: Synthetic mesh simplification study.

for the full-resolution cleaned BPA mesh to 0.098237 after simplification and final cleanup, an increase of only 0.171%. Both PLY and STL exports could be reloaded successfully. PLY reloaded with 16,649 indexed vertices and 29,998 triangles; STL retained the same triangle count but reported more vertices because STL repeats triangle vertices instead of storing the shared indexed connectivity used by PLY. This synthetic export pipeline demonstrates cleanup,

Variant	Triangles	Reduction	Components	Sym. RMSE
Original	40,020	0.0%	1	0.098070
30k	29,999	25.0%	2	0.098195
20k	20,000	50.0%	19	0.098727
10k	10,000	75.0%	78	0.100504
5k	4,999	87.5%	46	0.103913

Table XXIV: Synthetic simplification study. The 30k variant preserves geometry while stronger simplification rapidly increases fragmentation.

simplification, serialization, and integrity checking independently of the later realistic-geometry validation.

The remaining topological limitations are reported rather than hidden by aggressive repair. For visualization and geometric comparison the open BPA surface is acceptable, but applications requiring a closed volume would need a specialized downstream repair stage. This distinction is important for DentaVision: the current contribution targets faithful surface reconstruction, not guaranteed watertight solid modeling.

I. Validation of Surface Reconstruction on Five Dental Models

1) *BPA Radius Re-Optimization Across Ten Jaws:* The realistic BPA parameter study was repeated on all ten registered jaw clouds. Four radius-factor configurations were evaluated against the original STL surfaces using symmetric RMSE, boundary-edge count, component structure, reconstructed-to-reference area ratio, and self-intersections.

The same trade-off appears in every model set. Moving from A to B reduces the mean boundary-edge count from 3694.1 to 678.4, while mean symmetric RMSE increases only from 0.113276 to 0.115989. Configurations C and D close only modestly more boundaries than B but increase geometric error by approximately 9.4% and 24.2% relative to B. Configuration B is therefore retained as the final realistic BPA setting.

2) *Poisson versus Final BPA Across Ten Jaws:* Poisson reconstruction and the selected BPA configuration B were then evaluated against all ten original STL surfaces. BPA was more accurate on every model set and produced no self-intersections. Poisson generated fewer open boundaries, but this apparent completeness was achieved by adding large unsupported surface regions.

The directional evaluation is central to this result. Poisson covers much of the reference geometry reasonably well, with a mean reference-to-reconstruction RMSE of 0.1366 model units. In the opposite direction, however, its mean RMSE rises to 2.0807 model units, compared with only 0.1140 for BPA. The mean Poisson surface-area ratio of 1.415 indicates approximately 41.5% more reconstructed area than the reference on average, whereas BPA remains close to

the reference at 0.972. All ten Poisson meshes contain self-intersections; none of the ten BPA meshes do.

The final realistic reconstruction choice is therefore:

Ball Pivoting with radius factors (1.5, 2.0, 3.0).

J. Discussion: Synthetic Screening versus Realistic Validation

The two-stage evaluation shows why broad synthetic screening and multi-model realistic validation serve different purposes. The synthetic generator isolates overlap, rotation, translation, and noise, which makes failure modes interpretable. The five-model suite then tests whether those conclusions survive substantial changes in actual dental geometry. The second stage both revised and strengthened the provisional choices: RANSAC replaced FGR as the final global registration method, while BPA was confirmed and its realistic radius configuration changed from A to B.

A second lesson concerns metric choice. FGR often has excellent median transform errors despite catastrophic failures, so median precision alone would hide robustness problems. Likewise, Poisson can look visually smooth and has relatively few boundary edges, yet its reconstruction-to-reference error and inflated surface area reveal unsupported geometry. The final decisions therefore combine transform error, success rate, bidirectional surface distance, boundary structure, connected components, surface-area ratio, and self-intersection tests rather than relying on a single metric.

K. Reproducibility and Integration into the Shared Pipeline

All experiments are deterministic where possible. The synthetic stress tests use 10 repetitions per condition with controlled dataset-seed ranges and reproducible algorithm seeds; the same repetition index is reused across values within a stress-test series so that the intended parameter change dominates the comparison. Open3D seed 42 is used for surface sampling. The five-model zero-noise benchmark assigns deterministic algorithm seeds to every model/jaw/pair combination. In the noise experiment, deterministic per-model, per-jaw, per-level, and per-repetition seeds generate the noisy clouds, and every algorithm receives the same perturbed geometry. Intermediate outputs are stored as PLY meshes/point clouds and CSV or NumPy files containing metrics and poses.

The module boundaries also define the hand-off to the surrounding DentaVision project. The image-acquisition and 3-D reconstruction stage provides point-cloud fragments. The present registration stage aligns these fragments and exports a merged point cloud. Surface reconstruction then produces an explicit mesh suitable for downstream visualization or later processing. Generated test fragments and intermediate

Mesh	Vertices	Triangles	Components	Vertex manifold	Self-int.	Watertight	Sym. RMSE
Cleaned full-resolution BPA	21,689	40,020	1	No	No	No	0.098070
Final 30k export	16,649	29,998	1	No	No	No	0.098237

Table XXV: Final synthetic mesh integrity check. Simplification reduces complexity substantially without creating self-intersections or materially changing geometric error.

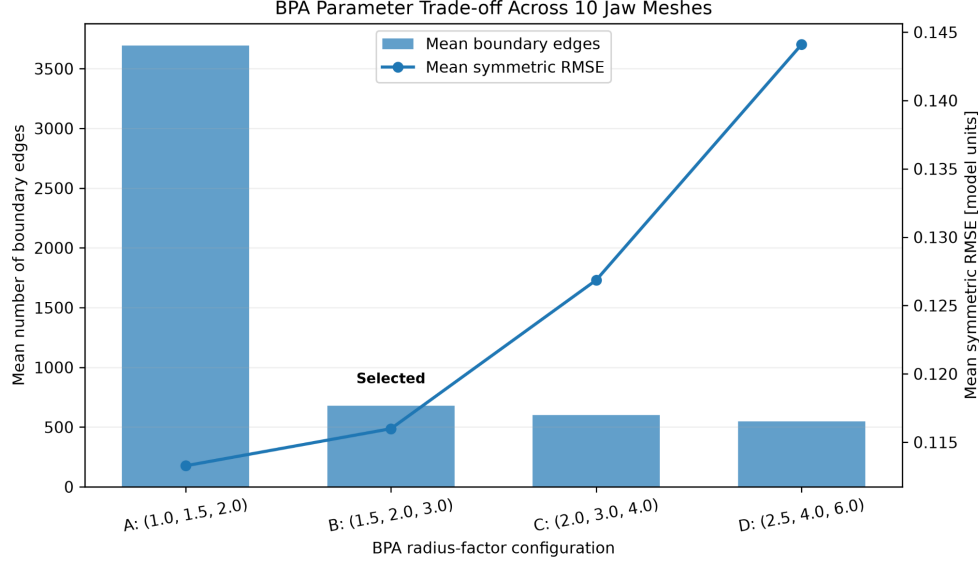


Figure 34: BPA parameter trade-off across ten jaw meshes. Configuration A minimizes geometric error but leaves many open boundaries. Configuration B is selected as the best compromise between boundary closure and fidelity.

Configuration	Mean sym. RMSE	Mean boundary edges	Mean components	Largest comp. frac.	Mean area ratio	Self-int.
A (1.0, 1.5, 2.0)	0.113276	3694.1	6.0	0.998650	0.939826	0/10
B (1.5, 2.0, 3.0)	0.115989	678.4	3.7	0.998970	0.972137	0/10
C (2.0, 3.0, 4.0)	0.126863	600.3	3.2	0.999159	0.971531	0/10
D (2.5, 4.0, 6.0)	0.144096	548.1	4.3	0.998966	0.971059	0/10

Table XXVI: Aggregate realistic BPA parameter study. Configuration B increases mean symmetric RMSE by only about 2.4% relative to A while reducing mean boundary edges by about 81.6%.

Method	Mean sym. RMSE	Median sym. RMSE	Ref.→Rec. RMSE	Rec.→Ref. RMSE	Mean area ratio	Self-int.
BPA	0.115989	0.115584	0.117964	0.113965	0.972137	0/10
Poisson	1.475913	0.999702	0.136558	2.080730	1.414758	10/10

Table XXVII: Aggregate surface-reconstruction validation across ten jaw meshes. The large Poisson reconstruction-to-reference error and area ratio reveal unsupported reconstructed surface regions.

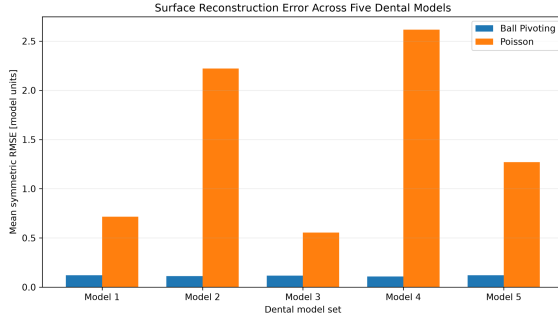
outputs can be reproduced from the source STL models and are therefore not required to be versioned in the shared repository.

For the final repository organization, reusable implementation code remains under `src/registration3d`, while executable studies are grouped under a top-level `experiments/` directory. Report-figure generation and diagnostic utilities are kept separately in `visualization/` and `diagnostics/`. Historical single-model experiment drivers are archived rather than mixed with the final five-model study. This organization

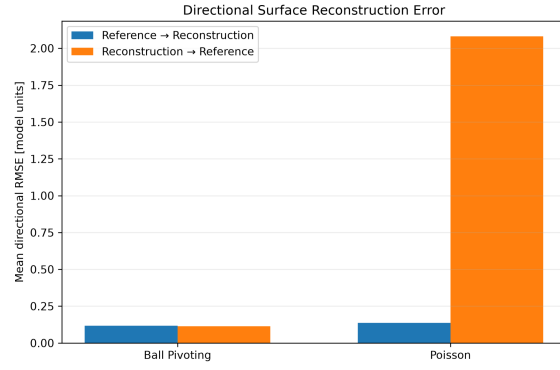
keeps the core package importable while making the experimental provenance of the reported numbers explicit. The principal five-model drivers are the pairwise benchmark, noise benchmark, sequential multiway benchmark, BPA parameter study, and BPA-versus-Poisson mesh validation.

L. Final Method Selection and Limitations

The final method selection differs from the provisional synthetic choice because the realistic validation exposed failure modes that were not sufficiently represented by the first synthetic benchmark.

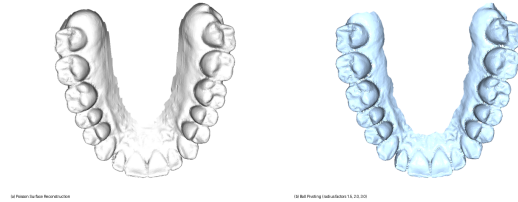


(a) Symmetric RMSE by model set.

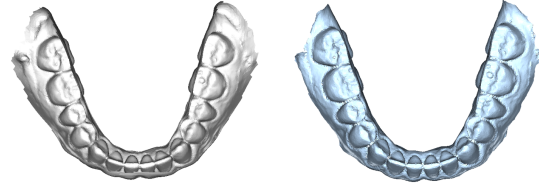


(b) Mean directional RMSE.

Figure 35: Surface-reconstruction generalization across five dental model sets. BPA is consistently more accurate, while Poisson’s dominant error occurs in the reconstruction-to-reference direction, indicating unsupported added geometry.



(a) Representative upper-jaw comparison.



(b) Representative lower-jaw comparison.

Figure 36: Representative qualitative comparison from Model 1. Poisson appears smooth and closed, but the ten-jaw quantitative evaluation shows substantially larger unsupported surface error than BPA.

Decision	Synthetic evidence	Five-model realistic evidence	Final interpretation
Global registration	FGR offered a strong accuracy/runtime compromise in the controlled benchmark	RANSAC solved all 30 zero-noise adjacent pairs and stayed at 100% through $\sigma = 0.10$	Select RANSAC + ICP for robustness
Multiway chaining	Small error in four-fragment synthetic baseline	Maximum global error across 30 non-reference poses: 0.008239 model units and 0.02266°	Sequential four-fragment accumulation remains small
Surface reconstruction	BPA strongly outperformed Poisson in bidirectional error	BPA outperformed Poisson on every model set; 0/10 versus 10/10 self-intersecting meshes	Retain BPA
BPA radii	Configuration A minimized synthetic RMSE	B reduced boundary edges by 81.6% versus A for only 2.4% higher mean RMSE	Use configuration B on realistic dental geometry

Table XXVIII: How five-model validation revised or confirmed the provisional synthetic conclusions.

Stage	Final choice	Key evidence
Pairwise	FPFH + RANSAC + P2Plane ICP	30/30 zero-noise successes; strongest noise robustness
Multiway	Sequential four-fragment chaining	Explicit pairwise-error accumulation without loop-closure correction
Surface	Ball Pivoting	Lower bidirectional error; 0/10 self-intersecting meshes
BPA radii	(1.5, 2.0, 3.0)	81.6% fewer boundary edges than A for 2.4% higher mean RMSE
Simplification	≈30k triangles	25.04% triangle reduction for 0.171% RMSE increase in synthetic study

Table XXXI: Final design decisions after synthetic screening and five-model realistic validation.

1) Threats to Validity:

a) *Internal validity.*: The experiments were designed to make algorithmic differences dominate over incidental randomness. All methods within one noisy trial receive exactly the same perturbed point clouds, dataset and algorithm seeds are deterministic, and pairwise transformations can be evaluated against exact virtual ground truth. Nevertheless, stochastic global registration can still exhibit seed-dependent outcomes. The reported benchmark therefore characterizes the implemented configuration and seed-controlled protocol rather than every possible random realization. The Wilson intervals shown for RANSAC

Experiment driver	Purpose
<code>realistic_model_suite_pairwise_benchmark.py</code>	Five-algorithm comparison on 30 adjacent fragment pairs
<code>realistic_model_suite_noise_benchmark.py</code>	Shared-noise robustness evaluation over 2,400 registrations
<code>realistic_model_suite_multiway_ransac.py</code>	Sequential four-fragment RANSAC multiway evaluation
<code>realistic_model_suite_bpa_parameter_study.py</code>	BPA radius trade-off across ten jaws
<code>realistic_model_suite_mesh_validation.py</code>	Final BPA-versus-Poisson surface comparison

Table XXIX: Principal final experiment drivers after repository cleanup. All are executed from the project root through the Poetry environment.

Artifact	Role in pipeline
Fragment PLY files	Pairwise and multiway registration input
Estimated pose arrays	Reproducible multiway transformation state
Merged point cloud	Interface between registration and mesh reconstruction
Evaluation CSV/JSON	Quantitative experiment results
Final PLY mesh	Indexed geometry for processing and visualization
Final STL mesh	Interoperable triangular surface export

Table XXX: Principal artifacts exchanged or exported by the registration/model-construction stage.

visualize finite-sample uncertainty only; the three nonzero-noise repetitions share the same 30 base fragment pairs and are not treated as 90 independent dental geometries.

b) Construct validity.: Correctness is operationalized by transform error when ground truth is available, using thresholds of one model unit and one degree in the realistic registration study. These thresholds are deliberately more informative than fitness or inlier RMSE alone, because repetitive dental surfaces can support plausible but incorrect correspondence sets. Surface quality is likewise assessed bidirectionally instead of relying only on appearance or watertightness.

However, the mesh-distance evaluation is sampling based and therefore approximates, rather than exactly computes, point-to-triangle surface distance. In addition, STL does not encode physical units, so absolute distances are reported in model units and should not be interpreted as millimeters without verified source metadata.

c) External validity.: The five realistic model sets are STL surfaces that are sampled and fragmented in software; they are substantially more diverse than the synthetic dental-like generator but are not actual intraoral scanner captures. Real acquisitions may contain occlusions, outliers, strongly nonuniform point density, reflections, motion artifacts, and scanner-specific systematic error. The added perturbation is isotropic Gaussian positional noise rather than a calibrated sensor-noise model. Furthermore, the ten jaws have intentionally similar overall scale and do not systematically cover severe malocclusions, missing teeth, restorations, pathological anatomy, or deliberately varied acquisition conditions. Generalization to those clinically relevant geometry classes therefore remains untested. Finally, the realistic multiway experiment evaluates four-fragment sequential chaining without loop closures; substantially longer acquisition trajectories may require pose-graph optimization and explicit loop-consistency checks.

M. Conclusion

A complete registration and model-construction pipeline was implemented and evaluated from multiple dental point-cloud fragments to reconstructed triangular surfaces. Five pairwise registration algorithms were first compared under controlled synthetic changes in overlap, rotation, translation, and noise. The synthetic benchmark initially favored FPFH + FGR + Point-to-Plane ICP, but the subsequent five-model validation exposed catastrophic FGR outliers and changed the final selection to FPFH + RANSAC + Point-to-Plane ICP.

Across five distinct dental model sets comprising ten jaw meshes and 30 adjacent zero-noise fragment pairs, the selected RANSAC pipeline achieved 30/30 successful registrations with mean translation and rotation errors of 0.001921 model units and 0.00948°. Under added Gaussian positional noise it maintained 100% observed success through $\sigma = 0.10$ and achieved 84.44% at $\sigma = 0.20$. Sequential four-fragment multiway registration across all ten jaws yielded a maximum global pose error of only 0.008239 model units and 0.02266°.

For surface reconstruction, BPA configuration B (1.5, 2.0, 3.0) provided the best realistic trade-off between geometric fidelity and boundary closure. Across all ten jaw meshes, BPA achieved a mean symmetric RMSE of 0.115989 compared with 1.475913 for Poisson. Poisson’s mean reconstruction-to-reference

RMSE reached 2.080730 and its mean area ratio 1.414758, indicating substantial unsupported geometry; all ten Poisson meshes contained self-intersections, whereas none of the BPA meshes did.

The main outcome is therefore not only a final algorithm choice but a reproducible selection strategy: controlled synthetic experiments expose specific failure modes, while multi-model realistic geometry determines which methods generalize sufficiently well to be carried forward into the DentaVision pipeline.

VI. MEDICAL TOUCHSCREEN GRAPHICAL USER INTERFACE INTEGRATION

Authors: Irmak Damla Özdemir

VII. INTRODUCTION

Automating parts of the intraoral scanning procedure with the help of a robotic arm is a theme, motivated by the goal of reducing dependence on the operator and making the procedure more comfortable for the patient. A system built around this idea needs some way for an operator to interact with it: something that shows the state of the procedure, lets the operator choose what needs to be scanned, and turns information about the patient into values that the robotic arm can use to reach the correct position.

DentaVision Control Panel is a touchscreen graphical user interface built with Qt Creator that functions as a control panel for a dental scanning procedure. The interface tracks the patient's head position and orientation in real time through a camera and estimates the target coordinates that a robotic arm would need to reach in order to position its scanning tool at a selected tooth. The system brings together three main parts in a single control panel: a dental chart using FDI numbering that lets the operator select the tooth to be scanned, a pipeline that uses computer vision to detect the patient's face and mouth region and continuously estimate head position and orientation from the camera feed, and a stage that converts the detected head pose and the selected tooth into a target pose estimation in the coordinate space used by the robotic arm.

The panel itself is limited to producing this target pose; it does not compute the joint level motion of the robotic arm and does not perform any part of the scan.

At the current stage, *DentaVision Control Panel* runs as a standalone prototype: the tracking, stabilization, and target estimation described above are fully implemented and aims to produce real values from a live camera feed. However, the interface is not yet connected to a physical robotic arm or to the simulation. Therefore, the control panel does not know the details of any robotic platform.

A. Purpose

The graphical user interface described is intended to be used by healthcare personnel during an oral scanning procedure and it mainly serves two goals: to give the personnel supervising the procedure clear visual feedback on its state, and to help protect the patient by checking that the patient's head stays stable before and during scanning.

To support this second goal, the system relies on the head tracking and target estimation described earlier to flag whether the patient's head is stable enough for the scan to proceed. Command buttons tied to each stage of the procedure keep this process under the operator's control rather than letting it run on its own. Alongside this, the interface also reserves a panel for manual adjustment of the robotic arm's position, so that healthcare personnel can step in and correct the position themselves whenever it is judged necessary.

B. Scope

The control panel is developed with the expectation that, at a later stage, the target pose it computes could be handed to a robotic system that resolves it into joint motion through inverse kinematics. At its current stage, values produced by manual jog operations on the Joint Control panel are only printed to the terminal rather than sent to any robotic hardware.

The scope of this work is limited to the software and algorithm level: the interface logic, the head tracking pipeline, and the target coordinate calculation described above, without the hardware integration mentioned above.

The head position estimate produced by the system does not rely on a full 3D pose estimation method, and does not use a camera calibration matrix. Instead, it is derived from a simplified geometric relationship between the 2D landmark positions returned by MediaPipe Face Mesh. Likewise, the offset used to obtain the target position for a given tooth comes from a fixed, generic dental arch template rather than a measurement taken from the individual patient.

Finally, the system assumes that the patient remains at a fixed distance from the camera throughout the procedure, using a chair and headrest arrangement that keeps this distance constant; the conversion from pixel distances to real-world distances is therefore based on a single scale constant calibrated for that one fixed distance, rather than a distance measured continuously.

C. Terminology

1) *FDI Tooth Numbering*: The system identifies teeth using the two-digit FDI (Fédération Dentaire Internationale) notation. The first digit denotes the quadrant, and the second digit denotes the tooth's position within that quadrant, counted from the midline outward. Table XXXII lists the four quadrants, and

Table XXXIII lists the tooth name corresponding to each position digit; concatenating a quadrant digit with a position digit yields the full FDI number and, in combination, the tooth's full name (e.g. 26 = Quadrant 2 + Position 6 = Upper Left First Molar).

Table XXXII: FDI quadrants

Quadrant	Jaw	Side
1	Maxilla (upper)	Right
2	Maxilla (upper)	Left
3	Mandibula (lower)	Left
4	Mandibula (lower)	Right

Table XXXIII: FDI tooth positions (second digit)

Position	Tooth
1	Central Incisor
2	Lateral Incisor
3	Canine
4	First Premolar
5	Second Premolar
6	First Molar
7	Second Molar
8	Third Molar

2) X , Y , Z and RX , RY , RZ : They represent a complete description of an object's position and orientation in three-dimensional space, expressed as six numbers: (x, y, z, r_x, r_y, r_z) .

The first three values, x , y , and z , describe where the object is located along three perpendicular axes of a fixed reference frame. Together they behave like a 3D address: moving along x shifts the object left or right, along y shifts it up or down, and along z shifts it in depth, toward or away from a reference point.

The last three values, r_x , r_y , and r_z , describe how the object is rotated, independently of where it is. These follow the common aviation-style convention:

- **RX / pitch:** Rotation that tilts the object up or down, like nodding.
- **RY / yaw:** Rotation that turns the object left or right, like shaking your head "no".
- **RZ / roll:** Rotation that tilts the object side to side, like tilting your head toward one shoulder.

Together, these six values fully define both where something is and how it is angled, without needing any further information. This is the standard way a target pose is communicated between the different parts of the system throughout this report.

3) *Maxilla and Mandibula*: *Maxilla* and *Mandibula* are the anatomical terms for the upper and lower jaw.

4) *Facial Landmarks and Face Mesh*: A *facial landmark* is a specific, identifiable point on a face, for instance, the corner of the mouth, the tip of the nose or the edge of an eye, detected automatically from a camera image. A *face mesh* is the complete set of such landmarks arranged over the face.

5) *Inverse Kinematics (IK)*: *Inverse kinematics* is the process of determining the joint angles a robotic arm must take in order to place its end effector at a desired pose. It is the counterpart to *forward kinematics*, which computes the resulting pose from a given set of joint angles. Since a single pose can often be reached through more than one joint configuration, and because some poses may lie outside the arm's physical reach, inverse kinematics is generally a more involved problem than forward kinematics, and typically requires a dedicated solver.

VIII. SYSTEM OVERVIEW

A. Architecture

DentaVision Control Panel is organized as a processing pipeline that turns a live camera feed into an absolute Tooth Target position. At every frame, the system moves through four stages:

- **camera input acquisition,**
- **facial and mouth detection,**
- **geometric computation,**
- **panel display.**

Each stage is implemented as a separate module aiming for single responsibility, so that the detection logic knows nothing about robot motion, and the display layer knows nothing about how a coordinate was derived.

Figure 38 shows this structure as it is implemented in the codebase. At the top, `main` creates and wires together the five controllers: `camera_controller`, `dental_chart_controller`, `tooth_target_controller`, `joint_motion_controller`, and `tooth_badge_controller`. `camera_controller` is the only one that depends on the vision package, while the remaining controllers depend on core for teeth data, tooth targets, and phase state, and on ui for the widgets they update. Both vision and core are leaf packages in this structure: they are depended on by the controllers above them but do not themselves depend on any other package, which keeps detection and computation logic independent of how the panel is built. Inside ui, most modules form a single chain centered on `mainwindow`, with a few of them reaching directly into core for shared enums or into assets.

B. Data Flow

The flow starts with the raw image captured by the camera. Each frame passes through the stage that detects the face and mouth region; the output of this stage is the patient's current head position and orientation. This estimate is written to a single head position object shared across the system and is updated on every new frame. At the same stage, the reliability of this estimate is also checked: the head must stay close enough to the target reference point, and it must not move too much over several

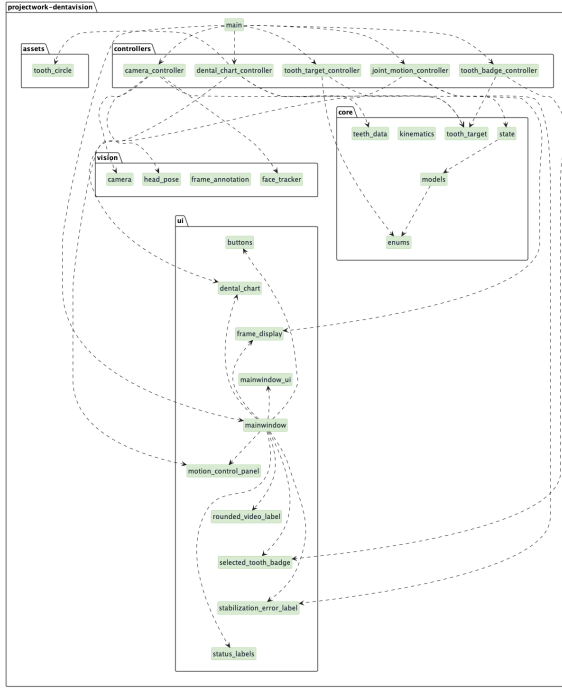


Figure 37: Class Diagram of DentaVision Control

consecutive frames. Only when both conditions are met does the system enter the stabilized state, and no action, including tooth selection or starting a scan, is allowed while the head position is not stabilized.

Once the head position is stabilized, the operator selects a tooth through the interface. The FDI label of the selected tooth (see subsection VII-C1) is then combined with the current head position: the tooth's precomputed angular and depth offsets, together with its fixed geometric position on the dental arch, are added to the current head position to produce the final Cartesian target the robotic arm needs to reach. This target is both displayed numerically on the panel and passed on to the motion control stage.

In the motion control stage, this target is taken as a baseline pose. Once the conditions required to use this panel are met (see subsection IX-C), the operator can apply small corrections on top of this baseline through the joystick and rotation controls in the interface. At this stage, the system tracks a small set of states through a ScanFlow (idle, aligning, scanning, paused) to determine when alignment is complete, when scanning can begin, and when it should be paused; these phases are triggered by stabilization and by operator interaction, and their status is shown on the panel.

In the final step, the computed Cartesian target must be converted into the joint angles that would move the physical robot; this conversion is performed through inverse kinematics (see subsection VII-C). This step is not part of the current system: the data flow

described here ends at the point where the Cartesian target is produced and made available to that step (see section XIV).

C. Dependencies and System Requirements

The panel's logic was implemented in Python, with the interface designed using Qt. Camera capture and face detection rely on the Python computer vision libraries introduced earlier, OpenCV and MediaPipe. The main components used throughout the project are listed in Table XXXIV.

Table XXXIV: Dependencies and Development Tools Used in Control Panel

Component	Purpose
Python	Used to implement the application
PySide6	GUI framework (Qt for Python bindings)
MediaPipe	Face and mouth landmark detection
OpenCV	Camera capture, color conversion, and overlay drawing
Qt Creator	Visual design and layout editing of UI

D. Operating Environment and Physical Setup Assumptions

The correct operation of DentaVision Control Panel relies on a set of assumptions about the physical setup it is used in:

- The control panel is designed as a touchscreen, and healthcare personnel interact with its components by touch.
- A working webcam or USB camera is required; since MediaPipe can run on a CPU, a GPU is not required.
- The patient sits in a fixed dental chair, with the head held in place using a dental head stabilizer. The head position stays fixed throughout the examination and does not move up, down, or side to side.
- The height of the chair and its distance from the camera are set so as not to affect the reliability of the vision pipeline, and this distance is treated as constant.
- The camera used by the panel is mounted at a fixed position and does not move.
- The camera and the control panel may be part of the same device, or an external camera integrated into the patient chair may be used instead.
- The lighting in the room is expected to be suitable for the camera's field of view, or additional lighting should be added manually to the area the camera covers.

As noted in the Scope, this setup keeps the distance between the camera and the patient fixed. In practice, this means that `HeadPosition.z` is not measured from the camera and is instead kept at a constant baseline value, and it is what allows the `CM_PER_PIXEL`

calibration constant, which expresses how many centimeters a single camera pixel corresponds to, to be determined from a single measurement. If the chair or the camera is moved, this calibration needs to be repeated.

IX. USER INTERFACE DESIGN

The UI design of the control panel was guided by a user flow (Figure 38) built with the medical environment and the personnel's user experience in mind, and the implementation was kept as close to this flow as possible.

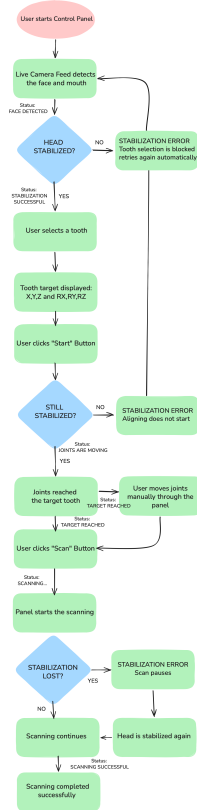


Figure 38: User Flow Diagram (Excalidraw)

Initially, a simpler user flow was used as a basis to create the first version of the design in Figma (Figure 86), and this design was then refined through iteration and further optimized during implementation.

In this context, the following points were taken into account in the design:

- Touch targets, such as buttons and the cells of the dental chart, are sized and spaced for use with a finger, since the panel is operated as a touchscreen rather than with a mouse and keyboard.
- The state of the procedure is communicated mainly through short text labels and color, so the operator can tell whether a face is detected, whether the head is stabilized, and whether scanning is active or paused, at a glance.

- Controls that are not valid in the current phase are disabled rather than hidden. For example, tooth selection is blocked while the head is not stabilized, and the joint motion panel is disabled while scanning is active, so the operator cannot trigger an action the procedure does not allow at that moment.
- The layout is divided into separate, fixed regions for the dental chart, the camera feed, the motion control panel, and the status indicators, so that each part of the procedure has a predictable place on the screen instead of competing for the same space.
- The tooth target and tooth information are shown near the dental chart right after a tooth is selected, and joint control panel is right below this section, so that the operator can see the target values and make adjustments in place.

The design was built by creating a Qt Creator UI file, using layouts, frames, QLabels, and similar widgets (Figure 87 and 88). Some components, such as the dental chart, frame display and joystick widget, were implemented directly in a Python file under *ui* folder instead, since their behavior was too complex to define declaratively within Qt Creator.

A. Main Window Layout

The main window measures 1024x600 pixels and divides the workflow into three panels, meant to be read from left to right and top to bottom. On the left, from top to bottom, are the selected tooth indicator, the tooth targeting values for the selected tooth, and a motion control module that lets personnel manually control joint movement. The center holds the dental chart panel, used to select the tooth to be targeted. On the right are two panels: the live camera feed and a status panel showing the state of the system. At the top of the layout are the control panel's title and the three-stage Start/Scan/Pause button. The first version of the main window was created in Figma (Figure 39).

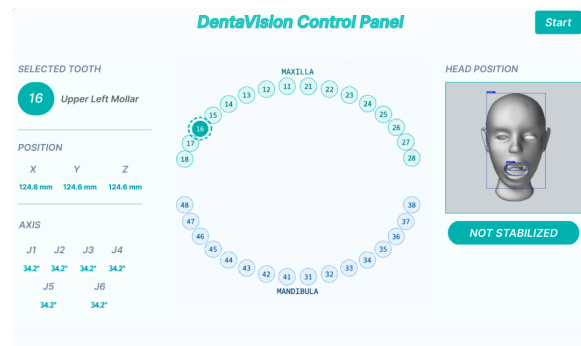


Figure 39: First Version of MainWindow

This layout was later revised based on supervisor feedback and the project's evolving requirements, with

the goal of matching the actual use case and real-world usability more closely. Compared to the first version, manual joint control was implemented, from a set of raw numeric axis readouts to a joystick widget paired with a small set of directional buttons, which is closer to how an operator would actually want to nudge the robotic arm during alignment. The selected tooth and tooth target fields were also changed to show an explicit empty state, such as "No tooth selected" and dashes in place of coordinate values, instead of example values, so the panel reflects what the operator actually sees when the panel is started. Finally, the camera feed and the system status were separated into two distinct areas rather than a single combined block, so that face and head detection status stays visible even when the camera feed itself is empty or not yet running. The resulting layout is shown in the current version of the main window (Figure 40).

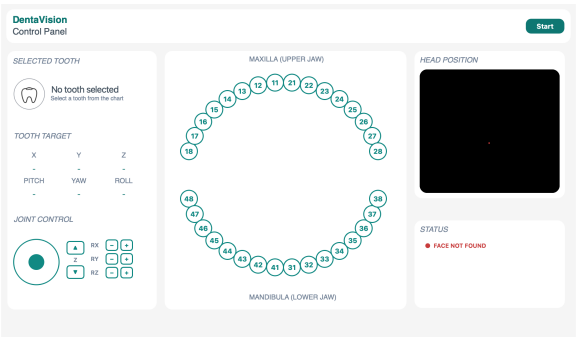


Figure 40: Last Version of MainWindow

B. Tooth Target Panel

This panel displays, under "Selected Tooth", a default tooth icon when no tooth is selected (Figure 41). Once a tooth is selected from the dental chart, this icon is replaced by the tooth's FDI number together with its name (Figure 42).

Below the Selected Tooth section is the Tooth Target area, which shows the target coordinates computed for the selected tooth: X, Y, Z for position, and pitch, yaw, roll for rotation. While no tooth is selected, all fields are shown as dashes rather than as zero values (Figure 43); this is an intentional choice, since showing "0.0" in place of a missing value could be mistaken by the operator for an actual computed measurement, whereas a dash makes clear that no value has been computed yet. Once a tooth is selected, these fields update immediately to show the computed target values together with their unit (Figure 44). The values shown here were originally in millimeters, and were later changed to meters to match the units expected by the PyBullet robot simulation (Figure 45).

SELECTED TOOTH



Figure 41: No Tooth Selected Badge

SELECTED TOOTH



Figure 42: Tooth Selected View

TOOTH TARGET

X	Y	Z
-	-	-
PITCH	YAW	ROLL
-	-	-

Figure 43: Tooth Target Landing/Default View

TOOTH TARGET

X	Y	Z
-42.0 mm	0.6 mm	18.6 mm
PITCH	YAW	ROLL
25.0°	-1.4°	1.9°

Figure 44: Tooth Target View with 'mm' Unit

TOOTH TARGET

X	Y	Z
-0.0400 m	0.0004 m	0.0186 m
PITCH	YAW	ROLL
-6.5°	-67.0°	4.4°

Figure 45: Tooth Target View with 'm' Unit

C. Manual Joint Control Panel

The panel is built around a joystick, used to adjust position along X and Y by dragging its center point away from the middle of the circle. Next to it, a pair of arrow buttons labeled Z is used to move up or down along the vertical axis. To the right of these, three rows of plus/minus button pairs, labeled RX, RY, and RZ, are used to adjust rotation around each of the three axes independently (Figure 46). Each adjustment made through these controls is applied on top of the baseline pose described earlier in Data Flow Section, is only printed to the terminal rather than sent to a physical robot (Figure 89).

As also shown in the state diagram (Figure 57), this panel is enabled while the system is in the *aligning* or *paused* states, and is locked during *scanning*, since a manual correction at that point could interfere with the ongoing scanning motion and put the patient's safety at risk.

JOINT CONTROL

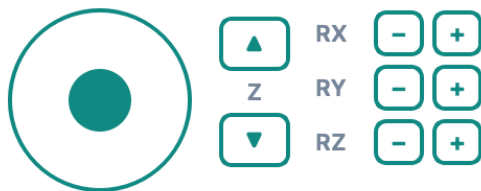


Figure 46: Joint Control Panel

D. Dental Chart Panel

The dental chart panel arranges teeth not as a plain list or table, but on an elliptical layout that resembles the geometry of a simplified dental arch, the curve of the upper and lower jaw (Figure 47). The panel includes *maxilla* (upper jaw) and *mandibula* (lower jaw) labels to make the arch easier to read. This layout is intended to help personnel locate a tooth more quickly and intuitively than scanning a numbered list would.

When a tooth is clicked, the tooth selection callback checks whether the patient's head position is stable enough; if it is, the selected tooth's name and FDI number appear as a badge (Figure 48). If the head position is not stable enough, the system blocks the selection before it happens, so that no tooth target is computed from an unreliable head position while the patient's head is still moving. In this case, a message is printed to the terminal (see section XIII), while on the panel itself the selection is simply not allowed, and the "head is not stabilized" status lets personnel know why.

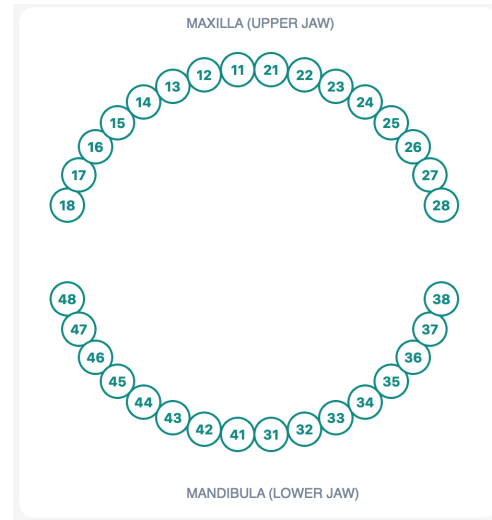


Figure 47: Dental Arch Landing View

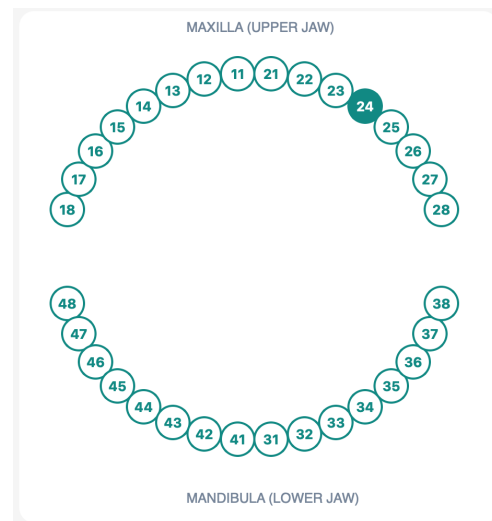


Figure 48: Dental Arch Tooth Chosen View

E. Camera Panel and Status Indicators

The Camera Panel consists of a live video feed with bounding boxes highlighting the detected face and mouth, with rounded corners framing the detected regions (Figure 49). In addition, a red point is

displayed on the screen to indicate the target position at which the patient's mouth should be centered. This visual reference is intended to assist in monitoring the stability of the patient's head position. A default positional tolerance of 15 pixels is defined around the target point; the patient's mouth is therefore considered to remain within the acceptable stabilization range as long as its detected position stays within this tolerance.

Depending on the detection state, the labels FACE NOT FOUND (Figure 50), FACE DETECTED, and HEAD IS NOT STABILIZED (Figure 51) or STABILIZATION SUCCESSFUL (Figure 52) are shown in the status area, supported by color coding.

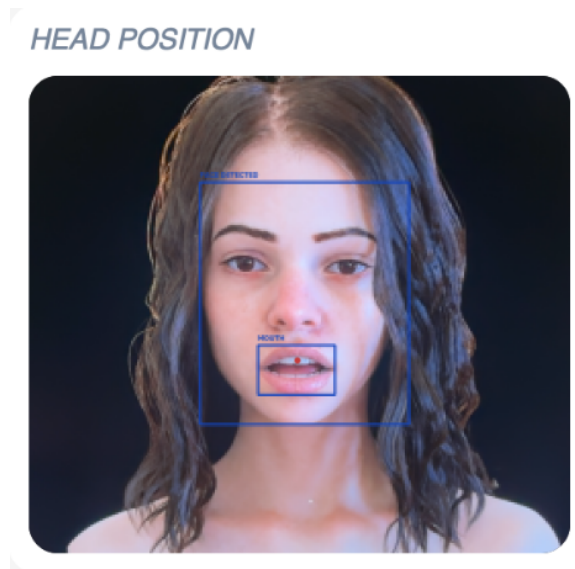


Figure 49: Camera View with a Human Face Model generated by AI. (Status: Head is not stabilized)

After the 'Start' button is clicked the status shows JOINTS ARE MOVING (Figure 53), followed by TARGET REACHED (Figure 54) once the joints arrive at the target, in this case a 2 seconds buffer to simulate the delay. The operator then makes any necessary manual adjustments and presses the button again, now labeled 'Scan', to begin scanning; if this proceeds without interruption, the status shows SCANNING... (Figure 55), followed by SCANNING SUCCESSFUL (Figure 56).

The status section is located directly below the camera panel and is intended as an indicator that helps personnel follow the progress of the procedure and monitor the patient's safety. The sequence of states shown by the status area is as follows:

- 1) By default, the status area starts in red with the FACE NOT FOUND label (Figure 50).
- 2) Once a face is detected in the camera feed, the label changes to green FACE DETECTED. (Figure 51)

- 3) If the patient's head is detected but has not yet stayed within the tolerance around the target point for the required number of consecutive frames, the label changes to HEAD IS NOT STABILIZED (Figure 51), signaling that the patient needs to be repositioned.
- 4) Once the head remains within tolerance for the required number of consecutive frames, the label switches to STABILIZATION SUCCESSFUL (Figure 52), and tooth selection becomes possible.
- 5) After a tooth is selected and 'Start' button is pressed, the label changes to JOINTS ARE MOVING (Figure 53) while the system waits for the joints to reach the computed target.
- 6) Once the target is reached, the label updates to TARGET REACHED (Figure 54).
- 7) When the button, now labeled 'Scan', is pressed to begin scanning, the label changes to SCANNING... (Figure 55).
- 8) If scanning completes without the head losing stabilization, the label finally shows SCANNING SUCCESSFUL after 2 seconds of delay (Figure 56).

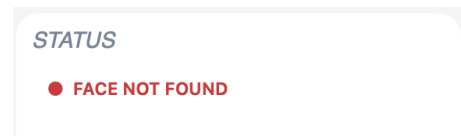


Figure 50: (Status 1) FACE NOT FOUND



Figure 51: (Status 2) HEAD IS NOT STABILIZED



Figure 52: (Status 3) STABILIZATION SUCCESSFUL

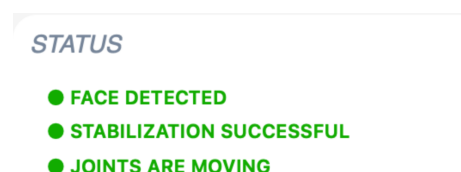


Figure 53: (Status 4) JOINTS ARE MOVING

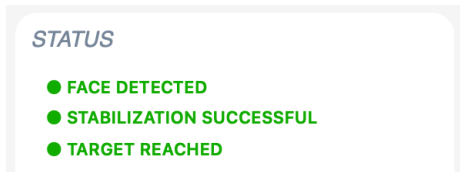


Figure 54: (Status 5) TARGET REACHED

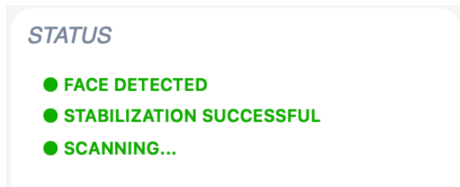


Figure 55: (Status 6) SCANNING...

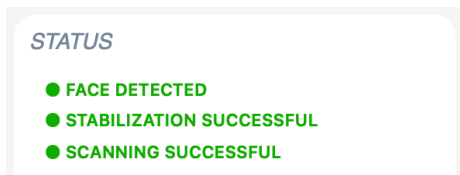


Figure 56: (Status 7) SCANNING SUCCESSFUL

F. Buttons

A single button is located in the top layer of the control panel. This button initially appears as the 'Start' button (Figure 58). When the button is pressed, the same button turns into the 'Scan' button (Figure 60), and provided the patient's head is stabilized, the joints begin moving toward the target. Until the target is reached, pressing the button again has no effect, which is reported through the terminal message "Cannot scan yet: target not reached" (see section XIII). Once the joints reach the target, pressing the Scan button turns it into the Pause button (Figure 62).

Once the procedure has started, if the patient's head stabilization is lost at any point, the button is disabled and shown in its faded state instead of its normal color, together with a stabilization error indicator. Depending on the phase the system was in when stabilization was lost, this faded state is shown either on the 'Start' button (Figure 59) or on the 'Scan' button (Figure 61); the button becomes usable again once stabilization is restored.

The full transition between these states is shown in Figure 57.

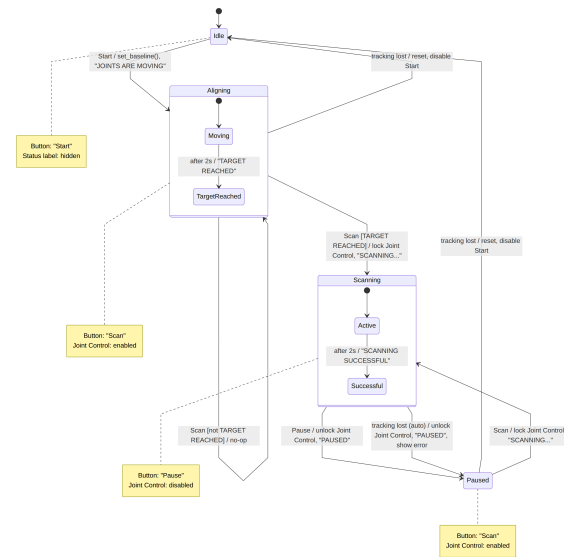


Figure 57: State Diagram of Button Transformation created by Claude AI



Figure 58: Start Button View



Figure 59: Start Button Faded View



Figure 60: Scan Button View



Figure 61: Scan Button Faded View



Figure 62: Pause Button View

X. VISION AND HEAD TRACKING

1) *Head Detection*: Each frame captured from the camera is first flipped horizontally to produce a mirror image, and its color space is converted from BGR to RGB, since the face tracking model expects RGB input. This frame is then passed to MediaPipe's Face Mesh solution, which, if it finds a face in the image, returns a dense set of landmarks spread across the face. The system is configured to track at most one face at a time, so even if multiple faces are detected, only the first result is used.

Two separate regions are extracted from the resulting set of landmarks. The first is the overall bounding box of the face, built from the minimum and maximum x and y coordinates across all landmark points. The second is a narrower mouth bounding box, obtained by selecting only the landmark indices belonging to the lip region, taken from MediaPipe's own lip topology. The left and right corner points of the mouth are also kept separately, to be used later in the orientation estimate.

2) *Frame Annotation*: Once the face and mouth have been located, this information is drawn directly onto the frame: a rectangle labeled "FACE DETECTED" is placed around the face bounding box, and a separate rectangle labeled "MOUTH" is placed around the mouth bounding box. These drawings are applied directly to the pixel data; the frame is then converted into a format the interface can display and shown in the video panel together with a fixed red dot marking the target. This red dot represents the fixed reference position the patient's mouth needs to be aligned with.

3) *Head Position and Orientation Estimation*: Once annotation is complete, the center of the mouth bounding box is computed and compared against a fixed target point defined in advance on the screen. This target point is horizontally centered and positioned slightly below the vertical center of the frame. The pixel-wise difference between the mouth center and this target point is converted into a position estimate in real-world units (x , y) using a known pixel-to-millimeter ratio; this value indicates how far, and in which direction, the patient's head has drifted from the target.

Orientation is estimated using two separate cues. The angle between the mouth's left and right corner points gives how much the head is tilted sideways (roll, r_z). How far the mouth center has shifted relative to the center of the overall face bounding box is used as a ratio to estimate how much the head is turned (yaw, r_y) and how much it is tilted forward or backward (pitch, r_x), and is converted into degrees using a predefined scale factor. The resulting six values (x , y , z , r_x , r_y , r_z) are written to a single head position object shared across the system.

4) *Stabilization*: Whether the estimated position is considered reliable is decided through a two-stage check. The first stage is a single-frame check: if the pixel distance between the mouth center and the target point (red dot) is within the set tolerance 15 px, that frame is considered "on target". However, a single frame appearing on target momentarily does not mean the head has actually settled.

For this reason, a second stage separately tracks how much the head position changes between consecutive frames. On each new frame, the current position is compared to the previous one; if the difference stays below a set threshold, a "stable frame" counter is incremented, and if it exceeds the threshold, the counter is reset. Once the counter reaches a predefined value, corresponding to roughly one second's worth of consecutive frames, the head position is considered to have actually settled.

The system only marks the final state as "stabilized" when both "on target" check and the multi-frame "motion has stopped" check are satisfied at the same time; this state is reflected in the corresponding label on the interface and is used as a precondition for actions such as tooth selection and starting a scan. If face tracking is lost for any reason, the system starts evaluating again from a clean stabilization window.

A. Future Implementations

The current system covers everything up to the computation of the target pose; passing this target on to a physical robotic arm or to a robot simulation such as PyBullet, and resolving it into joint angles through inverse kinematics, is planned as a future implementation step. Once this step is added, it would only require forwarding the Cartesian target already produced by the control panel to that layer.

Head position estimation currently relies on a simplified 2D landmark geometry; in a later stage, this could be replaced with a full 3D pose estimation based on solvePnP and a camera calibration matrix, to improve the accuracy of the position and orientation estimates.

The dental arch template used in the tooth target calculation is currently a fixed, generic geometric template; replacing it in the future with patient measurements, obtained through an initial scan or calibration step, could bring the computed target positions closer to the patient's actual anatomy.

Similarly, the pixel to millimeter conversion currently relies on a single scale constant, based on the assumption that the patient stays at a fixed distance from the camera. If this distance needs to vary, a calibration method that continuously measures distance from the frame would need to replace this fixed constant.

XI. CALCULATION OF TOOTH TARGET

The Tooth Target represents the target position, X, Y, Z, and the current head orientation, RX, RY, RZ, that the robotic arm would need to reach for a selected tooth. The calculation is split across two files: the `compute_tooth_target()` function performs the actual computation, while `tooth_target_controller.py` manages when this result is computed and how it is written to the panel.

The `compute_tooth_target(tooth_number, head_position)` performs a simple addition:

```
x = head_position.x + dx,  
y = head_position.y + dy,  
z = head_position.z + dz
```

Here, `dx` and `dy` are read from the `TOOTH_OFFSETS_MM` dictionary, and `dz` from the `TOOTH_DEPTH_OFFSETS_MM` dictionary, based on the tooth number. Both dictionaries are stored in core as teeth data. This data is derived from a fixed dental arch model and assumes that each tooth has a fixed position relative to the center of the mouth.

The data coming from head tracking and the fixed data coming from tooth anatomy (`dx`, `dy`, `dz`) are combined at this point. The Tooth Target is not a value that is computed once and stays fixed on the screen; it is recomputed in two different situations:

- **When a tooth is selected:** Clicking a tooth triggers the `show_target_for(tooth)` method. This method takes the result of `compute_tooth_target()` and writes it to the X, Y, and Z labels as meter values.
- **Whenever the head position is updated:** Each time the camera loop computes a new head position for a frame, `CameraController` triggers a callback. If a tooth is currently selected and the head is still stabilized, `show_target_for()` is called again and the panel is updated with the new values.

This second point matters: the Tooth Target is not a single snapshot taken at the moment of the click; it is a live value, recomputed on every frame for as long as a tooth remains selected and the head stays stabilized. If the head moves slightly, while still staying within the stabilization tolerance, the meter values shown on the panel are updated accordingly.

The two groups of values, position and orientation, are shown together on the same panel, but their sources and the logic behind their calculation are independent of each other:

The RX, RY, and RZ rotation values are produced in the vision module by

`estimate_head_orientation()`, using three different approaches for the three angles:

RZ, the roll value, is computed directly from the angle between the pixel coordinates of the two mouth corners (landmarks 61 and 291). In other words, however much the mouth line deviates from horizontal, that is, however much the head is tilted to the side, this angle is converted directly into degrees using `atan2`.

RY (yaw) and RX (pitch) are computed independently of each other, both relative to the center of the detected face box. The horizontal offset of the mouth center from the face box center, normalized by half the width of the box, gives a ratio used to estimate yaw; the vertical offset, normalized by half the height of the box, gives a separate ratio used to estimate pitch. Each ratio is then scaled by a fixed factor of 45 degrees, so that a mouth center sitting at the edge of the face box corresponds to an estimated angle of about 45 degrees in that direction.

A. Future Implementations

The most direct improvement to this calculation would be replacing the yaw and pitch approximations described above with a proper 3D pose estimation based on *solvePnP* and a camera calibration matrix, so that RX and RY are computed as true angles rather than being inferred from a shift in 2D landmark positions. This implementation does not cover this real life head position and orientation estimation, which is planned as a future step.

XII. MANUAL JOINT CONTROL MECHANISM

When the operator or personnel first presses 'Start' with a tooth selected, `MotionController.set_baseline(x, y, z, rx, ry, rz)` is called with the tooth target computed for that tooth at that moment. This stores the baseline position and rotation, and resets any offsets accumulated from a previous baseline back to zero. This baseline is set once, at the start of the alignment phase; selecting a different tooth afterward does not automatically recompute it.

The panel is enabled while the system is in the aligning or paused phase, since these are the states where a baseline exists and the joints are not currently in a scan, and it is disabled during scanning to prevent a manual correction from conflicting with an ongoing scan.

The joystick and the Z and rotation buttons apply corrections to the baseline differently. Moving the joystick reports its current horizontal and vertical position directly; this position is scaled by a fixed range of 10 mm and added to the baseline X and Y, so the resulting value reflects where the joystick currently is. Pressing a Z or rotation button, on the other hand, adds a fixed step, 1 mm for Z and 5

degrees for rotation, to a running offset that is kept from one press to the next, so repeated presses in the same direction accumulate.

Each of these updates is printed to the terminal Figure 89: a joystick movement is reported as, for example, "Joystick: x=3.2 mm, y=-1.5 mm", a Z adjustment as "Z: 2.0 mm", and a rotation adjustment as, for example, "RX: -6.5°". As stated in the Scope, none of these values are sent to a physical robot, since that step has not been implemented.

A. Future Implementations

Unlike the joystick, whose displacement is inherently bounded by the fixed 10 mm range it is scaled by, the Z and rotation offsets accumulate with every button press without an upper bound, since the underlying step functions only add to a running total. As a future implementation, a maximum offset from the baseline could be added for these two controls as well, so that repeated presses cannot move the target further than a range considered safe for a manual correction.

Two further additions could support this: a reset to baseline control that sets the accumulated Z and rotation offsets back to zero without waiting for a new baseline to be set, useful if the operator has made too large a correction; and showing the accumulated offset next to the baseline value, so the operator can see how far the current pose has moved from the originally computed tooth target rather than only seeing the combined result. Once the robotic integration described earlier is implemented, these same offsets would need to be included in the pose is sent to that layer, so manual adjustments remain part of the same system rather than being handled separately.

XIII. ERROR HANDLING AND EDGE CASES

When a Camera object is created, the backend passed to VideoCapture is selected automatically based on the operating system: `cv2.CAP_AVFOUNDATION` on macOS, `cv2.CAP_DSHOW` on Windows, and `cv2.CAP_V4L2` on Linux as the default for any other platform. This selection is made by `_default_backend()` using `platform.system()`, so the same code opens the camera correctly across operating systems without requiring a backend argument to be passed manually, though one can still be supplied explicitly if needed.

Selecting the correct backend does not guarantee that the camera can be opened. If `cv2.VideoCapture` still fails, for example because no camera is connected or the operating system has not granted camera permission, `Camera.__init__` raises a `RuntimeError` with the message "Failed to open camera. Check camera permissions and availability.". This exception

is not caught anywhere between Camera and the main window, so a failure at this point stops the application from starting rather than being reported through the interface.

Once the camera has started successfully, a later read failure is handled differently. If `read_frame()` returns `None` during operation, for instance because the camera is disconnected, `CameraController._update_frame()` calls `stop()`, which releases the camera, clears the video label and sets its text to "Camera off" (Figure 63), resets the face and stabilization indicators, and, if a scan was active, sets `_scanning_active` to `False`, hides the scanning label, and prints "Scanning Paused".

If no face is present in a frame, the face status is set to not detected, the stabilization state is set to "hidden", `head_position.detected` is set to `False`, and a "Face not found" warning is drawn on the frame (Figure 50); no tooth target is computed for that frame.

Several operator actions are only valid under specific conditions, and the corresponding controller reports why the action did not happen instead of performing it silently:

- Selecting a tooth or pressing Start when no face is detected: "Cannot {action}: face not detected" (Figure 67).
- Selecting a tooth or pressing Start when the head is not stabilized: "Cannot {action}: head is not stabilized" (Figure 66).
- Pressing Start while idle with no tooth selected: "Cannot start: no tooth selected" (Figure 65).
- Pressing Start again before the joints have reached the target: "Cannot scan yet: target not reached" (Figure 69).

If tracking is lost while the system is idle or aligning, `on_tracking_lost()` prints "Tracking lost", (Figure 68) then clears the tooth selection, the tooth information, and the tooth target, shows the stabilization error label (Figure 64), disables the 'Start' button, which returns to its faded state (Figure 59), resets its text, stops the target reached timer, hides the "joints are moving" label, re-enables the motion control panel, and resets the phase to idle.

If tracking is lost while a scan is in progress, two things happen within the same frame update. First, `CameraController` detects that scanning is active but the face is no longer detected or stabilized, sets `_scanning_active` to `False`, hides the scanning label, and prints "Scanning Paused" (Figure 70). Immediately after, the same state transition triggers `on_tracking_lost()` in the main window, which prints "Stabilization lost during scan — pausing automatically", calls `_enter_paused()` (which

sets the scanning label to its paused state, re-enables the motion control panel, and relabels the 'Start' button back to its faded Scan state, Figure 61), shows the stabilization error label, disables the 'Start' button, and clears the tooth target (Figure 64).

Once tracking is restored, `on_tracking_restored()` hides the stabilization error label and re-enables the Start button; the system does not resume a paused scan automatically, and pressing the 'Start' button again is required to continue.

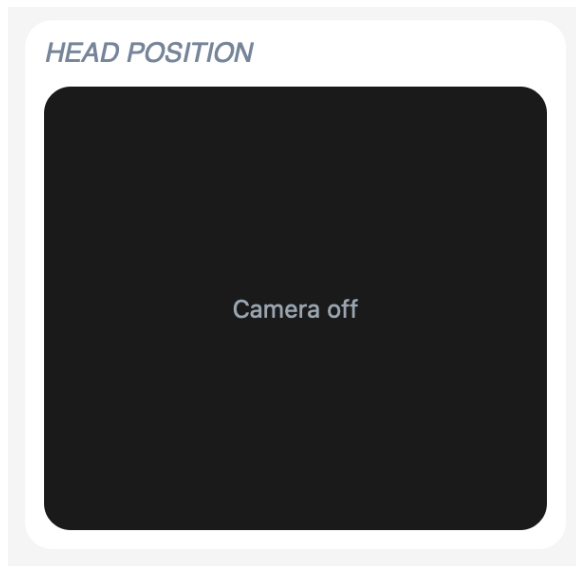


Figure 63: Video Panel After the Camera Feed Is Lost

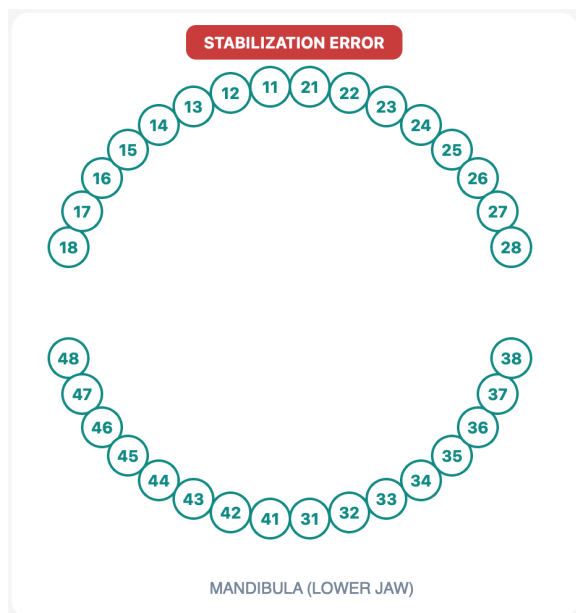


Figure 64: Stabilization Error

```
WARNING: All log messages before absl::InitializeLog() is
I0000 00:00:1789563875.029009 1 gl_context.cc:344] C
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
Cannot start: no tooth selected
```

Figure 65: Start Button Pressed With No Tooth Selected

```
Tracking lost
Tracking lost
Tracking lost
Cannot select tooth: head not stabilized
```

Figure 66: Tooth Selection Attempted With Head Not Stabilized

```
Tracking lost
Tracking lost
Tracking lost
Tracking lost
Tracking lost
Tracking lost
Cannot select tooth: face not detected
```

Figure 67: Tooth Selection Attempted With No Face Detected

```
INFO: Created TensorFlow Lite XNNPACK delegate
Tracking lost
Tracking lost
Tooth 22 is selected
Tracking lost
Tracking lost
```

Figure 68: Tracking Lost Error

```
Joints are moving...
Cannot scan yet: target not reached
Cannot scan yet: target not reached
Cannot scan yet: target not reached
Cannot scan yet: target not reached
Cannot scan yet: target not reached
```

Figure 69: Scan Target Not Reached Error

```
Cannot scan yet: target not reached
Scanning paused for joint re-calibration
Scanning paused for joint re-calibration
Tracking lost
```

Figure 70: Scan Paused

XIV. LIMITATIONS AND ASSUMPTIONS

This section brings together the assumptions and limitations described throughout the report.

- **Fixed calibration:** The conversion from pixel distances to real world distances is calibrated for a single fixed patient to camera distance.

- **Approximate head orientation:** The orientation values are not measured with a full 3D pose estimation method together with a camera calibration matrix.
- **Unmeasured depth:** The head position's Z value is not measured from the camera; it is kept at a fixed baseline, consistent with the fixed distance assumption above.
- **Generic dental arch model:** The offsets used to compute a tooth target come from a fixed, generic dental arch template.
- **No robotic hardware connection:** The system does not connect to a physical robotic arm or to a robot simulation. The computed target pose, and any manual joint corrections, are only printed to the terminal.
- **No inverse kinematics:** Converting the computed target pose into joint angles is not implemented; this step is left for future work.

A. UI Demonstration

The sequence of captured states of the running application is demonstrated from an unstabilized head position to a tooth target that has been reached and is ready for scanning.

Figure 71 shows the panel before the head position has stabilized. No tooth has been selected yet, the Tooth Target panel displays empty values, and the dental chart is overlaid with a stabilization error, matching the HEAD IS NOT STABILIZED status. The 'Start' button remains enabled, but starting a scan at this point is blocked by the same guard described in section XIII.

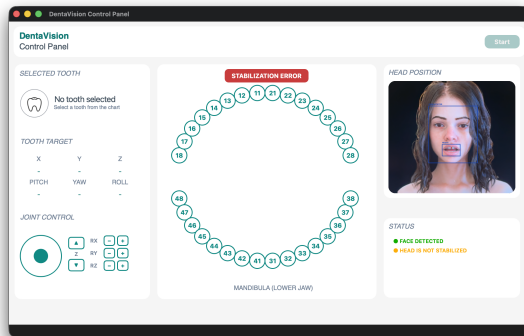


Figure 71: Control panel before stabilization, with no tooth selected and a stabilization error shown over the dental chart.

Once the head position stabilizes, the error banner disappears and the status list reports FACE DETECTION and STABILIZATION SUCCESSFUL, as shown in Figure 72. No tooth has been selected yet, so the Tooth Target panel still shows no values, and the Start button remains the only available action.

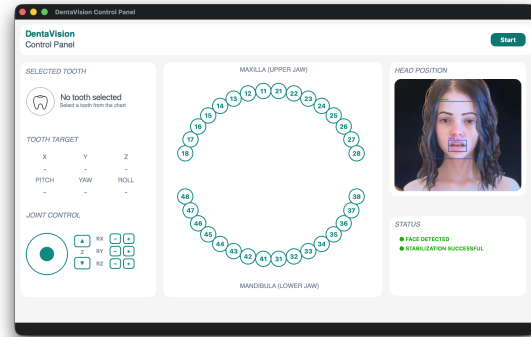


Figure 72: Control panel after stabilization succeeds, with no tooth selected.

After selecting tooth 24 from the dental chart, the Tooth Target panel populates with the computed position and orientation values for that tooth, following the calculation described in section XI. The status list now also reports TARGET REACHED, and the top-right button changes from Start to Scan, matching the transition described in the Buttons subsection. Figure 73 shows this state within the application window, and Figure 74 shows the state where scanning phase is completed successfully.

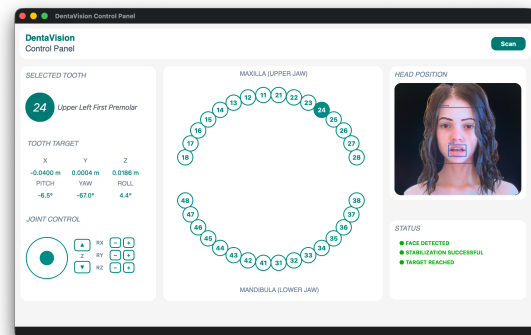


Figure 73: Tooth 24 selected, with its target reached and the Scan button now available.

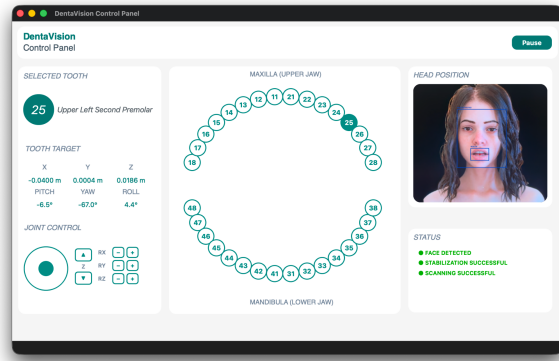


Figure 74: Tooth 24 selected, with its target reached and the Scan is successful

XV. CONNECTION TO THE ROBOT ARM SIMULATION

The separate simulation of a UR5e robot arm in PyBullet is controlled through text commands, one of which moves the arm to a given position and orientation. To check whether the Tooth Target output could drive this arm, `tooth_target.py` was extended with `compute_tooth_target_robot_frame()`, which converts the computed target into meters and adds it on top of a fixed base position the robot can already reach, `ROBOT_BASE_POSITION_M = (0.850, -0.000, 0.380)`, and `get_demo_command()`, which formats this result, together with whether the tooth belongs to the upper or lower jaw, into the `jaw` and `m` commands the simulation's console expects. This command is printed automatically whenever a tooth is selected, so it can be copied directly into the simulation terminal.

This process was carried out for an actual tooth selection in the panel, using tooth 14 as an example. With the patient's head stabilized, tooth 14 is selected on the DentaVision Control Panel, which computes and displays its target position and orientation as usual (Figure 75). While the tooth remains selected, `get_demo_command()` is called on every update and prints the corresponding `jaw` and `m` commands to the panel's terminal (Figure 76).

One of these printed commands is then copied by hand and pasted into the UR5e simulation's own console (Figure 77). Confirming the command there first shows a preview of the resulting arm position (Figure 78), and pressing `y` to confirm moves the simulated arm into this position (Figure 79).

With this change, the arm moves in roughly the direction and by roughly the distance that the Tooth Target estimation gives. This is however not the same as a real connection between the two systems. The two coordinate frames were never matched to each other, so the arm is not actually moving toward the patient's tooth, only moving by an offset that happens

to carry the right numbers on top of a fixed starting point.

TOOTH TARGET

X	Y	Z
-0.0251 m	0.0053 m	0.0111 m
PITCH	YAW	ROLL
-2.0°	-37.9°	2.7°

Figure 75: Tooth 14 Target Position and Orientation Computed by the System

```
Tracking lost
Tracking lost
Tracking lost
Tooth 14 is selected
jaw 14 upper
m 0.8227 0.0058 0.3911 -2.1 -38.0 2.6
jaw 14 upper
m 0.8227 0.0061 0.3911 -2.1 -37.9 2.5
jaw 14 upper
m 0.8229 0.0058 0.3911 -2.1 -37.8 2.5
jaw 14 upper
m 0.8229 0.0061 0.3911 -2.1 -37.7 2.4
```

Figure 76: Tooth 14 Target Position and Orientation Printed in the Terminal

```
UR5e Demo
Moves: 'm x y z [rx ry rz]' (RRT, Default-Orientierung 0 0 0)
Tool-Offset: 'o x y z' (z: B. o 0 0 0.15)
Geschw.: 's 0.5' (global, Default 0.5)
Reset: 'e' (nach manuellem Ziehen)
Render: 'render' (Cycles-Render in Blender)
Gebiss: 'jaw <nr> [upper|lower]' (z: B. jaw 3 upper)
Start: 'start <Aussein|Oben|Innen>' (Startposition anfahren)
Waypoints: '+/-' naechster/vorheriger Waypoint

> m 0.8227 0.0058 0.3911 -2.1 -38.0 2.6
Ausfuehren? [Y/n] y
X 0.823 Y 0.006 Z 0.391 pa -14° li -35° el +81° w1 +95° w2 -77° w3 +8°
[ok] (0.823, 0.006, 0.391) rx=-2 ry=-38 rz=3 (0 mm)
>
```

Figure 77: Tooth 14 Command Pasted into the UR5e Simulation Terminal

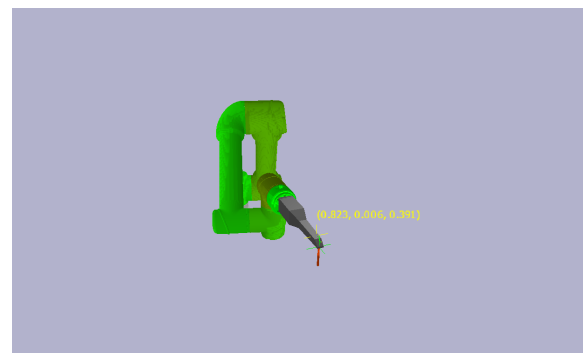


Figure 78: Preview of Tooth 14 Target Position and Orientation on the UR5e Robot Arm Simulation

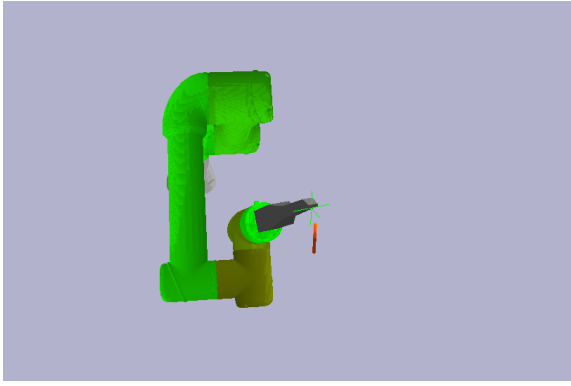


Figure 79: Tooth 14 Target Position and Orientation on the UR5e Robot Arm Simulation After the Command is Confirmed

LIST OF FIGURES

1	Basic UR5e implementation sitting on a table in PyBullet [5].	2	17	Dense stereo reconstruction of the Middlebury Plastic ThirdSize image pair using StereoSGBM. The left image shows the estimated disparity map, while the right image shows the corresponding colored 3D point cloud reconstructed from the disparity.	29
2	The Sirona Omnicam scanner device and the STL model used in the URDF description	2	18	Dense stereo result and 3D reconstruction obtained with IGEV on the Middlebury Plastic stereo pair. The left image shows the estimated disparity map, while the right image shows the corresponding colored 3D point cloud reconstructed from the disparity.	31
3	Description of the complete UR5e model	2	19	Representative realistic-validation workflow using Model 1 upper jaw. The original STL surface is sampled, split into overlapping fragments, displaced by known rigid transformations, registered with FPFH + RANSAC + Point-to-Plane ICP, and reconstructed with Ball Pivoting. Quantitative evaluation was performed on ten jaw meshes.	33
4	A position mirrored from PyBullet to Blender	3	20	Final pairwise registration pipeline after realistic validation.	33
5	Render pair example	3	21	Success-rate curves from the controlled synthetic pairwise experiments ($n = 10$ repetitions per parameter value). The curves should be interpreted as empirical behavior on the specific dental-like geometry rather than as universal monotonic capture-range characteristics.	35
6	Component architecture of the simulation environment.	5	22	Synthetic four-fragment pose-graph concept. Solid edges form the evaluated sequential baseline; dashed loop candidates were examined only in the optional robustness analysis.	35
7	Console view after program start	7	23	Five-model evaluation protocol. The same ten jaw geometries are propagated from controlled virtual fragmentation through pairwise comparison, noise robustness, sequential multiway registration, and surface-reconstruction validation.	37
8	Scanner at the tcp_pos and tcp_ori_deg of a start position	8	24	Bounding-box diagonal for all ten realistic jaw meshes. The relatively narrow range (79.696–92.084 model units) supports using one common registration parameterization, while also showing that the validation does not intentionally span extreme anatomical scale differences.	37
9	Scanner at W_min with the orientation of the start anchor	9	25	Representative qualitative example from Model 1. The quantitative registration experiments use all five model sets and all ten jaw meshes.	38
10	Scanner at W_max with the orientation of the end anchor	9			
11	Scan workflow for one start position. . .	10			
12	Inferred data flow of the Omnicam scanning pipeline.	14			
13	Sparse 3D reconstruction obtained from the Palm Desert image pair. The reconstructed points capture recognizable scene structure, including the main hillside, while remaining distributed throughout the surrounding scene.	22			
14	End-to-end OpenMVG/OpenMVS reconstruction workflow.	24			
15	Comparison between the sparse reconstruction generated by OpenMVG (left) and the dense point cloud generated by OpenMVS (right).	25			
16	Dense stereo reconstruction pipeline using StereoSGBM.	29			

26	Five-model zero-noise pairwise validation. RANSAC + ICP achieved 30/30 successful registrations and 100% success in every individual model set. . . .	38	43	Tooth Target Landing/Default View . . .	49
27	Mean runtime per zero-noise adjacent-pair registration across the 30 realistic cases. The selected RANSAC pipeline is slower than the local ICP variants but remains below FGR in this benchmark while providing substantially higher robustness.	38	44	Tooth Target View with 'mm' Unit . . .	49
28	Success rate under additive Gaussian positional noise across five dental model sets. The shaded band shows the 95% Wilson interval for RANSAC + ICP at nonzero noise levels. Because repeated noise realizations share the same 30 underlying fragment pairs, the intervals are used as descriptive finite-sample uncertainty indicators rather than formal population-level confidence statements.	38	45	Tooth Target View with 'm' Unit	50
29	Sequential multiway error across all ten jaw meshes. Lines show mean and median; shaded regions show the full range across the ten jaws.	39	46	Joint Control Panel	50
30	Surface-reconstruction and model-processing workflow. Algorithm selection is based on geometry and topology rather than visual smoothness alone.	39	47	Dental Arch Landing View	50
31	Synthetic geometric error comparison between Poisson and Ball Pivoting. . .	40	48	Dental Arch Tooth Chosen View	50
32	Synthetic BPA radius-factor study. . .	40	49	Camera View with a Human Face Model generated by AI. (Status: Head is not stabilized)	51
33	Synthetic mesh simplification study. . .	40	50	(Status 1) FACE NOT FOUND	51
34	BPA parameter trade-off across ten jaw meshes. Configuration A minimizes geometric error but leaves many open boundaries. Configuration B is selected as the best compromise between boundary closure and fidelity.	42	51	(Status 2) HEAD IS NOT STABILIZED	51
35	Surface-reconstruction generalization across five dental model sets. BPA is consistently more accurate, while Poisson's dominant error occurs in the reconstruction-to-reference direction, indicating unsupported added geometry.	43	52	(Status 3) STABILIZATION SUCCESSFUL	51
36	Representative qualitative comparison from Model 1. Poisson appears smooth and closed, but the ten-jaw quantitative evaluation shows substantially larger unsupported surface error than BPA. . . .	43	53	(Status 4) JOINTS ARE MOVING . . .	51
37	Class Diagram of DentaVision Control .	47	54	(Status 5) TARGET REACHED	52
38	User Flow Diagram (Excalidraw)	48	55	(Status 6) SCANNING...	52
39	First Version of MainWindow	48	56	(Status 7) SCANNING SUCCESSFUL	52
40	Last Version of MainWindow	49	57	State Diagram of Button Transformation created by Claude AI	52
41	No Tooth Selected Badge	49	58	Start Button View	52
42	Tooth Selected View	49	59	Start Button Faded View	52
			60	Scan Button View	52
			61	Scan Button Faded View	52
			62	Pause Button View	52
			63	Video Panel After the Camera Feed Is Lost	56
			64	Stabilization Error	56
			65	Start Button Pressed With No Tooth Selected	56
			66	Tooth Selection Attempted With Head Not Stabilized	56
			67	Tooth Selection Attempted With No Face Detected	56
			68	Tracking Lost Error	56
			69	Scan Target Not Reached Error	56
			70	Scan Paused	56
			71	Control panel before stabilization, with no tooth selected and a stabilization error shown over the dental chart.	57
			72	Control panel after stabilization succeeds, with no tooth selected.	57
			73	Tooth 24 selected, with its target reached and the Scan button now available. . . .	57
			74	Tooth 24 selected, with its target reached and the Scan is successful	58
			75	Tooth 14 Target Position and Orientation Computed by the System	58
			76	Tooth 14 Target Position and Orientation Printed in the Terminal	58
			77	Tooth 14 Command Pasted into the UR5e Simulation Terminal	58
			78	Preview of Tooth 14 Target Position and Orientation on the UR5e Robot Arm Simulation	58
			79	Tooth 14 Target Position and Orientation on the UR5e Robot Arm Simulation After the Command is Confirmed	59

80	wxPropView environment used during the investigation of the Sirona Omnicam camera through the Impact Acquire SDK.	62
81	Ghidra environment used during the reverse-engineering analysis of the CEREC software binaries.	62
82	C++ UDP/GVSP implementation output (1/4).	62
83	C++ UDP/GVSP implementation output (2/4).	63
84	C++ UDP/GVSP implementation output (3/4).	63
85	C++ UDP/GVSP implementation output (4/4).	63
86	Figma Control Panel UI Design Layout	63
87	Qt Creator UI File	63
88	Qt Creator Object Inspector	64
89	Terminal Output from a Manual Joint Adjustment	64

REFERENCES

- [1] *Bits2Bites Dataset*, [Online; accessed 15. Sep. 2026], Sep. 2026. [Online]. Available: <https://ditto.ing.unimore.it/bits2bites>
- [2] Universal Robots A/S, *UR5e technical specification*, https://www.universal-robots.com/media/1807465/ur5e_e-series_datasheets_web.pdf, Payload 5 kg, reach 850 mm, pose repeatability ± 0.03 mm per ISO 9283, 2021.
- [3] E. Coumans and Y. Bai, *PyBullet, a python module for physics simulation for games, robotics and machine learning*, <http://pybullet.org>, 2016–2021.
- [4] Blender Online Community, *Blender: A 3d modelling and rendering package*, <http://www.blender.org>, 2024.
- [5] josepdaniel, *Ur5-bullet*, <https://github.com/josepdaniel/ur5-bullet>, 2020.
- [6] Dentsply Sirona, *Sirona dental cad/cam system CEREC AC with CEREC Omnicam: Operating instructions*, <https://www.dentsplysirona.com/content/dam/master/product-procedure-brand-categories/cerec/product-categories/hardware/intraoral-scanners/ifu/archived/CER-IFU-CEREC-AC-Omnicam-US-EN-6422252-2019-02-11.pdf>, 2019.
- [7] C. R. Garrett, *Pybullet planning*, <https://pypi.org/project/pybullet-planning/>, 2018.
- [8] M. Stöth, *Pybullet & blender jaw rendering simulation*, <https://github.com/dentavision-intraoralscannerroboter/ur5e-bullet>, 2026.
- [9] H. Hirschmuller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, Feb. 2008, ISSN: 1939-3539. DOI: 10.1109/TPAMI.2007.1166 Accessed: Aug. 25, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/4359315>
- [10] G. Xu, X. Wang, X. Ding, and X. Yang, *Iterative Geometry Encoding Volume for Stereo Matching*, arXiv:2303.06615 [cs.CV], Mar. 2023. DOI: 10.48550/arXiv.2303.06615 Accessed: Aug. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2303.06615>
- [11] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” en, *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, Nov. 2004, ISSN: 1573-1405. DOI: 10.1023/B:VISI.0000029664.99615.94 Accessed: Aug. 25, 2026. [Online]. Available: <https://doi.org/10.1023/B:VISI.0000029664.99615.94>
- [12] R. Szeliski, *Computer Vision: Algorithms and Applications* (Texts in Computer Science), en. Cham: Springer International Publishing, 2022, ISBN: 978-3-030-34371-2 978-3-030-34372-9. DOI: 10.1007/978-3-030-34372-9 Accessed: Aug. 25, 2026. [Online]. Available: <https://link.springer.com/10.1007/978-3-030-34372-9>
- [13] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, ISSN: 1063-6919, Jun. 2016, pp. 4104–4113. DOI: 10.1109/CVPR.2016.445 Accessed: Aug. 25, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/7780814>
- [14] OpenCV, *Depth Map from Stereo Images*, 2026. Accessed: Aug. 25, 2026. [Online]. Available: https://docs.opencv.org/4.13.0/dd/d53/tutorial_py_depthmap.html
- [15] OpenCV, *Camera Calibration and 3D Reconstruction*, 2026. Accessed: Aug. 25, 2026. [Online]. Available: https://docs.opencv.org/4.13.0/d9/d0c/group__calib3d.html
- [16] OpenMVG, *Openmvg image datasets*, https://github.com/openMVG/Image_datasets, Palm Desert Micro dataset.
- [17] D. Scharstein and C. Pal, “Learning conditional random fields for stereo,” in *2007 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2007, pp. 1–8. DOI: 10.1109/CVPR.2007.383191
- [18] H. Hirschmuller and D. Scharstein, “Evaluation of cost functions for stereo matching,” in *2007 IEEE Conference on Computer Vision*

- and Pattern Recognition (CVPR), IEEE, 2007, pp. 1–8. DOI: 10.1109/CVPR.2007.383248
- [19] P. J. Besl and N. D. McKay, “A method for registration of 3-d shapes,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992. DOI: 10.1109/34.121791
- [20] Y. Chen and G. Medioni, “Object modelling by registration of multiple range images,” *Image and Vision Computing*, vol. 10, no. 3, pp. 145–155, 1992. DOI: 10.1016/0262-8856(92)90066-C
- [21] A. Segal, D. Haehnel, and S. Thrun, “Generalized-icp,” in *Robotics: Science and Systems V*, 2009. DOI: 10.15607/RSS.2009.V.021
- [22] R. B. Rusu, N. Blodow, and M. Beetz, “Fast point feature histograms (fpfh) for 3d registration,” in *2009 IEEE International Conference on Robotics and Automation*, 2009, pp. 3212–3217. DOI: 10.1109/ROBOT.2009.5152473
- [23] Q.-Y. Zhou, J. Park, and V. Koltun, “Fast global registration,” in *Computer Vision – ECCV 2016*, 2016, pp. 766–782. DOI: 10.1007/978-3-319-46475-6_47
- [24] M. Lyu, J. Yang, Z. Qi, R. Xu, and J. Liu, “Rigid pairwise 3d point cloud registration: A survey,” *Pattern Recognition*, vol. 151, p. 110408, 2024. DOI: 10.1016/j.patcog.2024.110408
- [25] Open3D Project. “Open3d 0.19.0 documentation,” Accessed: Sep. 9, 2026. [Online]. Available: <https://www.open3d.org/docs/release/>
- [26] Z. Huang, Y. Wen, Z. Wang, J. Ren, and K. Jia, “Surface reconstruction from point clouds: A survey and a benchmark,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 9727–9748, 2024. DOI: 10.1109/TPAMI.2024.3429209
- [27] M. Kazhdan, M. Bolitho, and H. Hoppe, “Poisson surface reconstruction,” in *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, 2006, pp. 61–70. DOI: 10.2312/SGP/SGP06/061-070
- [28] F. Bernardini, J. Mittleman, H. Rushmeier, C. Silva, and G. Taubin, “The ball-pivoting algorithm for surface reconstruction,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 5, no. 4, pp. 349–359, 1999. DOI: 10.1109/2945.817351
- [29] M. Garland and P. S. Heckbert, “Surface simplification using quadric error metrics,” in *Proceedings of SIGGRAPH 1997*, 1997, pp. 209–216. DOI: 10.1145/258734.258849

APPENDIX

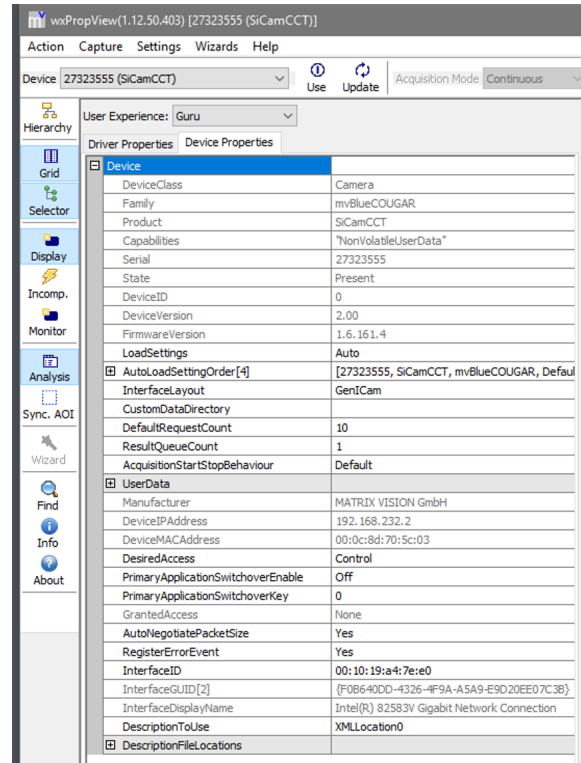


Figure 80: wxPropView environment used during the investigation of the Sirona Omnicam camera through the Impact Acquire SDK.

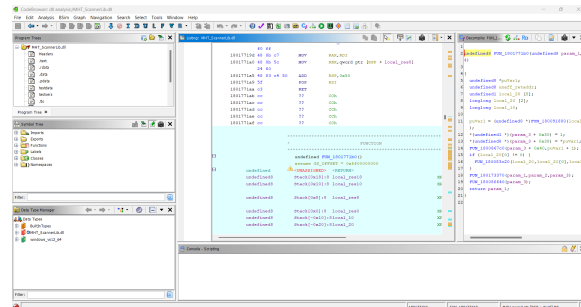


Figure 81: Ghidra environment used during the reverse-engineering analysis of the CEREC software binaries.

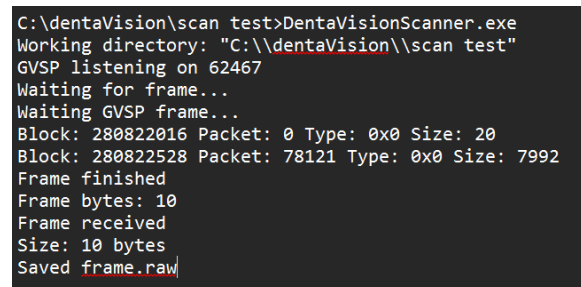


Figure 82: C++ UDP/GVSP implementation output (1/4).

Figure 83: C++ UDP/GVSP implementation output (2/4).

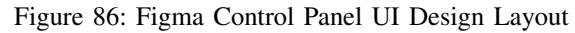


Figure 84: C++ UDP/GVSP implementation output (3/4).

Figure 85: C++ UDP/GVSP implementation output (4/4).



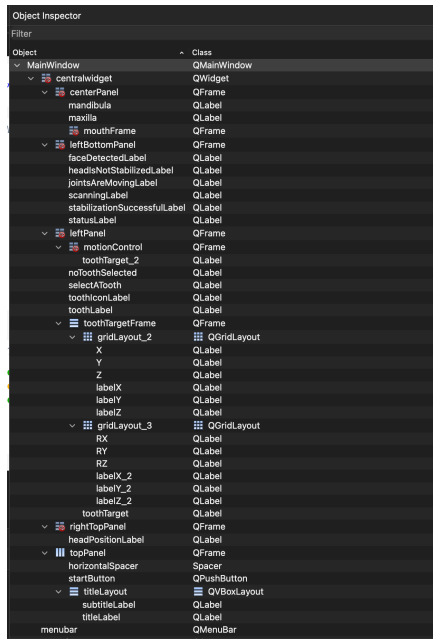


Figure 88: Qt Creator Object Inspector

```
Joystick: x=-0.7 mm, y=6.6 mm
Joystick: x=-3.3 mm, y=3.3 mm
Joystick: x=-3.4 mm, y=2.7 mm
Joystick: x=-3.4 mm, y=2.0 mm
Joystick: x=-3.0 mm, y=0.5 mm
Joystick: x=0.0 mm, y=0.0 mm
Z: 1.0 mm
Z: 2.0 mm
Z: 3.0 mm
RX: 5.0°
RX: 10.0°
RX: 15.0°
RY: -5.0°
RY: -10.0°
RY: -15.0°
RZ: 5.0°
RZ: 10.0°
```

Figure 89: Terminal Output from a Manual Joint Adjustment